AKTU

B. Tech Sem-VI
CS IT & CS Allied

Compiler Design

GATEWAY CLASSES

Unit-1 : Lecture-1

Gateway classes



Topics to be covered

- **Language Translator**
  - Assembler
  - Compiler
  - Interpreter
- **Phases of a compiler**
- **AKTU PYQs**

By Dr. Nidhi Parashar Ma'am
- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified

Language Translator

Language translator, is a computer program that convert source code from one programming language to another language or to machine code (also known as object code). They also find errors during translation.

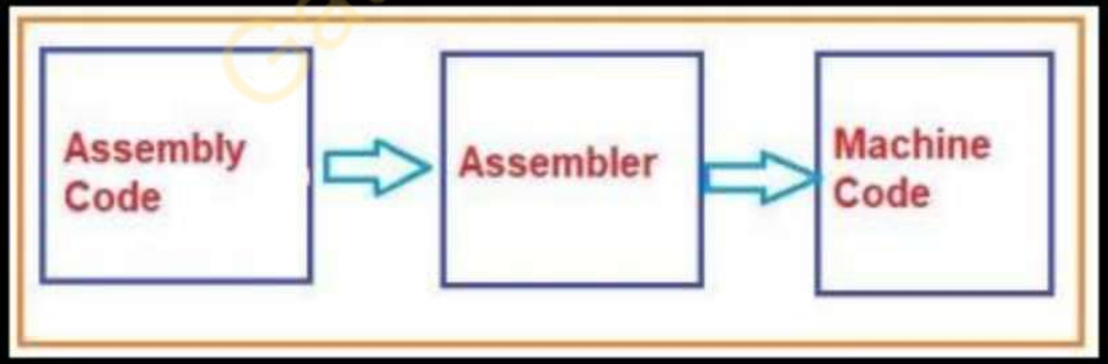
Types of Language Translators:

1. Assembler
2. Compiler
3. Interpreter

Gateway classes

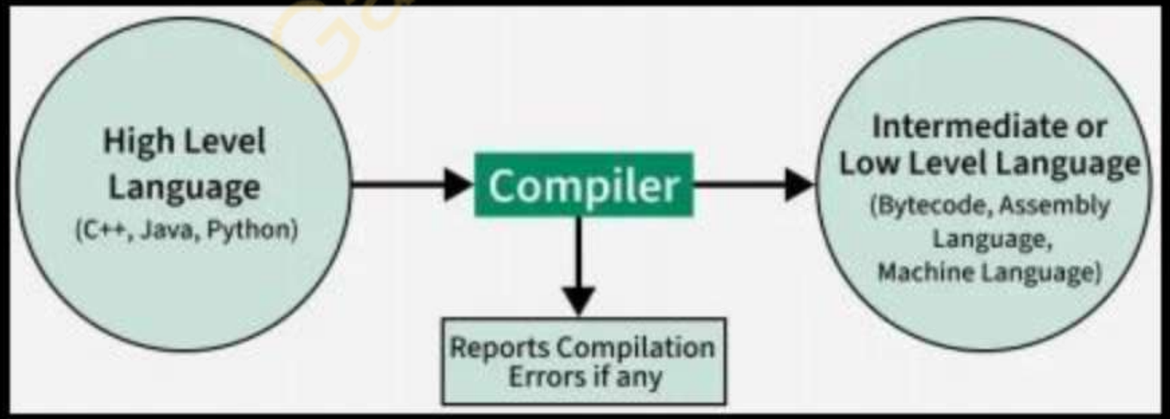
Assembler

The Assembler is used to translate the program written in Assembly language into machine code.



Compiler

- A compiler translates a high-level language program into a form that can be executed by a computer.
- Depending on language and compiler design, compiler may generate machine language directly(visual C++ compiler)(faster compilation), generate assembly language, or intermediate code.
  - In some systems, the compiler first converts the high-level program into **assembly code(Ex. gcc compiler)**, which is then translated into machine code by an **assembler**. It makes debugging easier.
  - In Java or C#, the compiler produces machine independent **intermediate code (bytecode)** that is later converted into machine code by a virtual machine or just-in-time compiler.
- **Compiled programs are faster as t**ranslation is done **only once and** CPU runs **pure machine instructions.**
- Example: If a loop runs 1,000 times, machine code is executed **directly 1,000 times** with no extra translation.

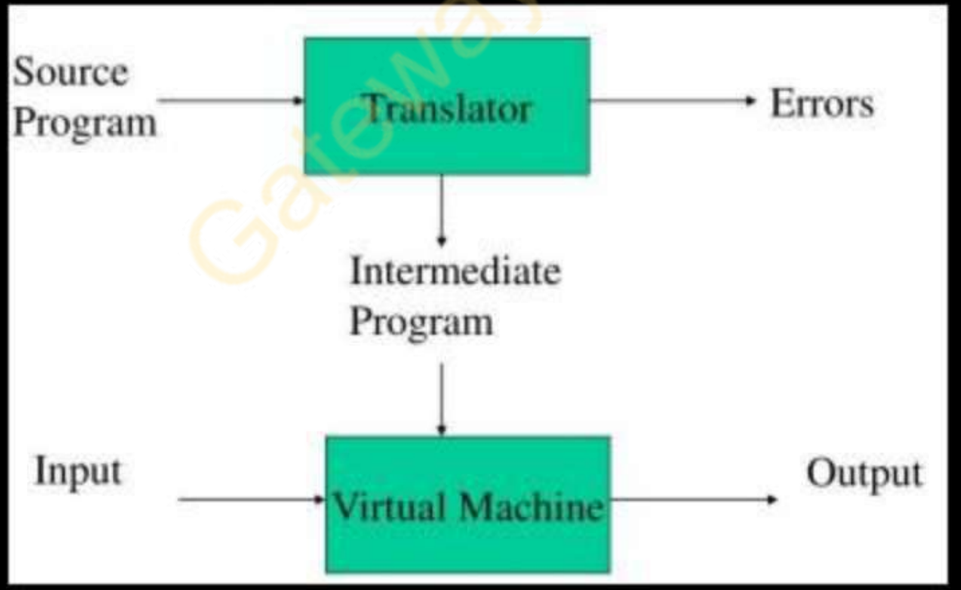


Gateway classes

Java language processors combine compilation and interpretation.

- A Java source program may first be compiled into an intermediate form called bytecodes.
- bytecodes are then interpreted by a virtual machine.
- Benefits:
- bytecodes compiled on one machine can be interpreted on another machine.

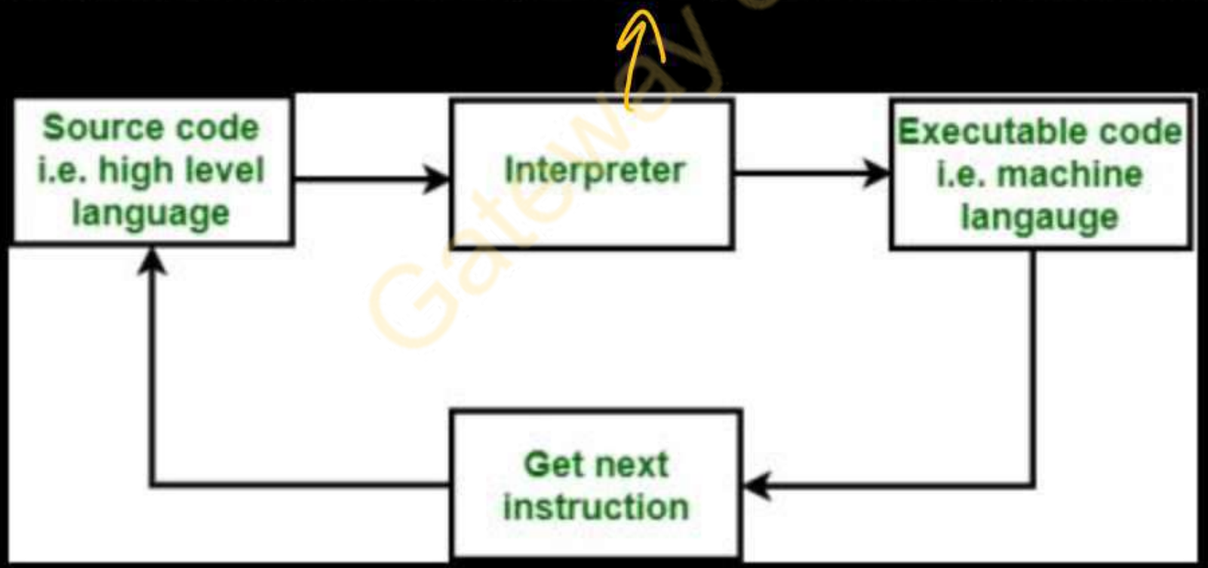
- To achieve faster processing, some Java compilers, called just-in-time compilers translate the bytecodes into machine language immediately at runtime.



Gateway classes

Interpreter:

- Interpreter converts the high-level language program to machine-level language.
- The interpreter checks the source code line-by-line and if an error is found on any line, it stops the execution until the error is resolved.
- Error correction is quite easy but, translation happens **while the program is running**
- The CPU spends time understanding the source code and converting it to machine instructions
- This extra work **slows down execution**
- Example: If a loop runs 1,000 times, the interpreter may **translate the same line 1,000 times**.



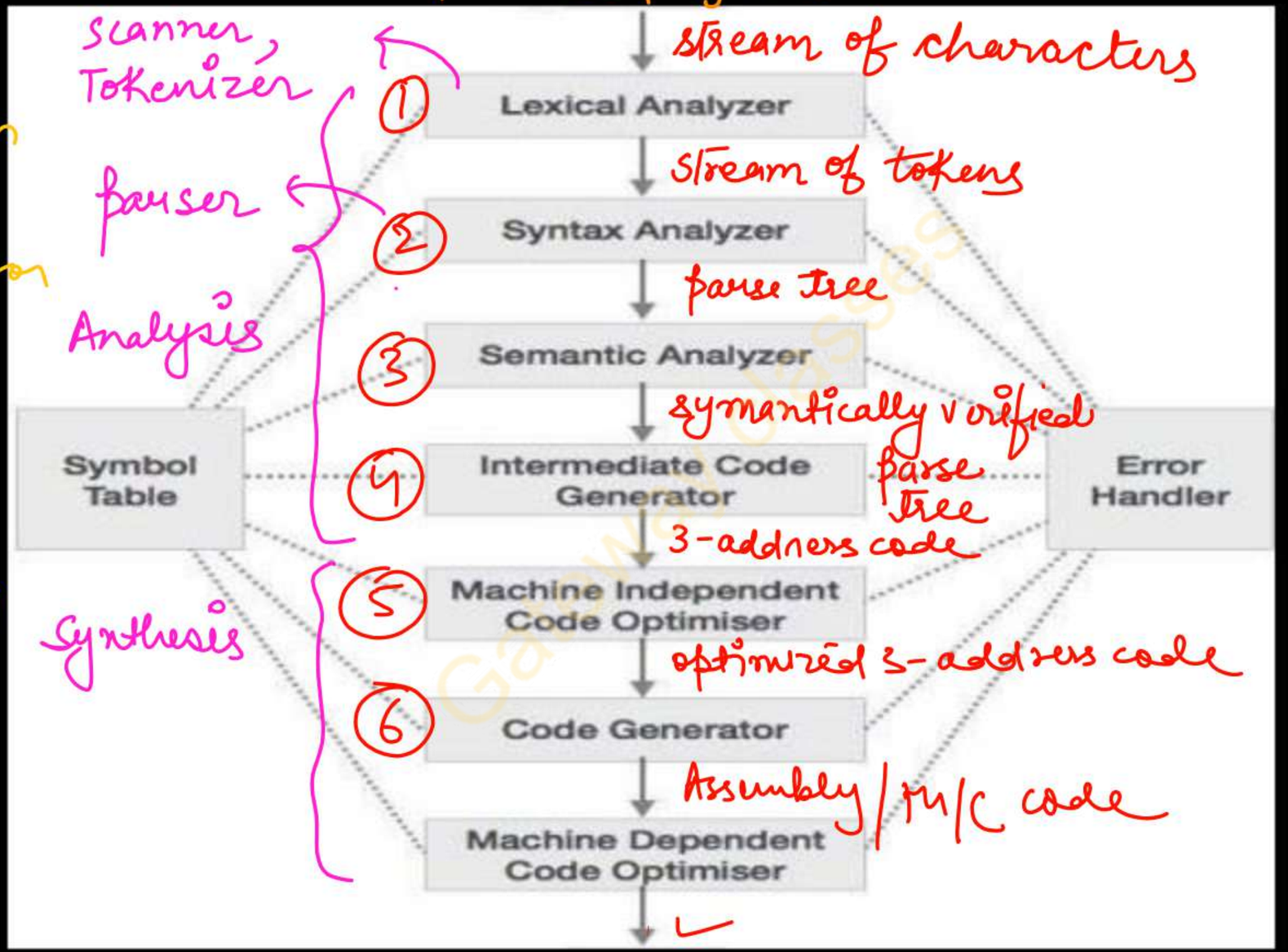
Gateway classes

Difference Between Compiler and Interpreter

Feature	Compiler	Interpreter
Translation Method	Translates entire source code into machine code at once, before execution.	Translates source code line by line during execution.
Execution Speed	Faster execution as the entire code is already converted to executable format.	Slower execution as each line has to be translated and then executed.
Error Handling	Displays all the errors after compiling, which makes debugging complicated.	Displays the errors instantly as the execution takes place line by line, which makes debugging easier.
Program Output	A separate executable file is generated that can run without the source code.	It does not create a separate file, so the program must always be executed through the interpreter.
Memory Usage	More memory is required for storing the compiled file and executable code.	Less memory is required as it runs the source code directly without creating an extra file.
Examples	Commonly used in C, C++, and Java (compiled to bytecode).	Commonly used in Python, Ruby, JavaScript, PHP.

Gateway classes

Phases of a compiler AKTU 2023-24)



HLL program

pre-processor

Pure HLL Program

compiler

Phases of compiler

#include

#define A 25

```
main()
{
   ≡
}
```

Phases of a compiler:

- During compilation, the Source code passes through six phases, each phase processes the code, resulting in machine-understandable code as an output. Each phase transforms one representation of source program to another. In practice, several phases may be grouped together.
- A compiler is made up of six phases:
  1. Lexical Analysis
  2. Syntax Analysis
  3. Semantic Analysis
  4. Intermediate Code Generation
  5. Code Optimization
  6. Code Generation
- The symbol table, which stores information about the entire source program, is used by all phases of compiler.
- Some compilers have a machine-independent optimization phase between the front end and the back end to
- perform transformations on the intermediate representation, so that the back end can produce a better target program.
- Since optimization is optional, one or the other of the two optimization phases may be missing.

Gateway classes

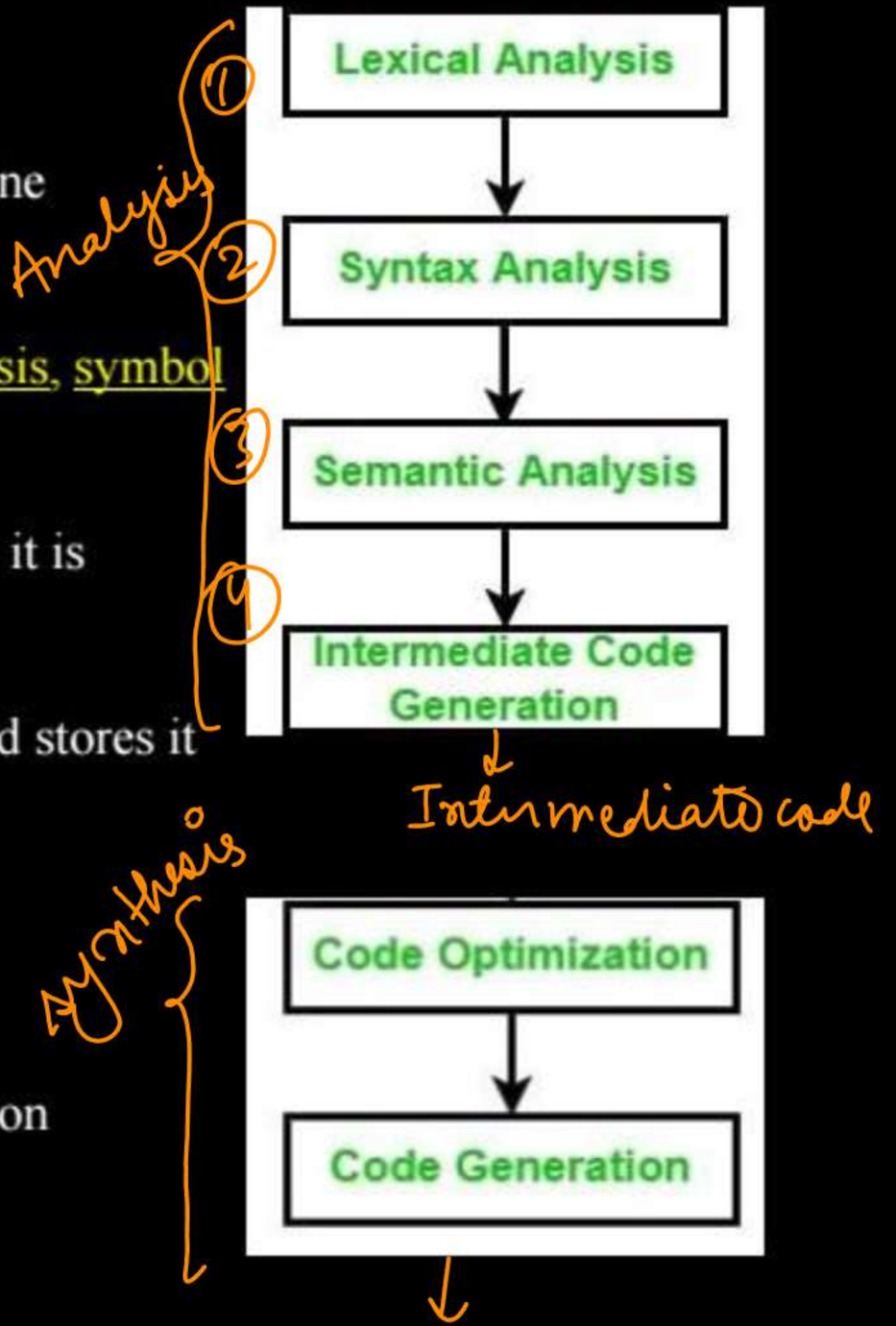
Two parts(Groups)

1. Analysis(Front end of compiler, machine independent):

- Consists of phases that are source language-dependent and target machine independent.
- Generally, consist of lexical analysis, semantic analysis, syntactic analysis, symbol table creation, and intermediate code generation.
- Includes error handling with each of the phases. If any errors are found, it is reported to be fixed.
- Collects important information about variables, data types, functions and stores it in a symbol table, which is later used by the next phase of the compiler.

2. Synthesis (back end of compiler, machine dependent):

- Part of compiler that depend on target machine but not on source.
- In back end, code generation and necessary features of code optimization with error handling and symbol table operations are included.



Gateway classes

Phase-1: Lexical Analysis(Scanning):

- The lexical analyzer scans the source code character by character, grouping these characters into the smallest unit of meaningful data (tokens) based on the language's syntax rules.
- These tokens can represent keywords, identifiers, constants, operators, or punctuation marks.
- It eliminates comments, blank spaces, tabs etc. from the program.
- Errors related to **invalid characters** are detected. If an invalid character or an unknown symbol is found, the analyzer reports errors along with the line number in the source file.
- Creates symbol table entry for each identifier(except data types).

Phase-2: Syntax Analysis or Parsing

- This phase ensures that code follows the correct grammatical rules of programming language like correct use of operators, keywords, and parentheses.
- Verifies that the sequence of tokens produced by the lexical analyzer is arranged in a valid way according to the language's syntax.
- If no error then to represent the structure of the source code , it creates parse tree or syntax tree otherwise generate errors.

Gateway classes

float a, b, c;

a = b + c * 60

↓

Lexical Analyzer. → (1)

(2) Creation of symbol Table

S.No.	V.name	—	—	—
1	a			
2	b			
3	c			

Lexemes	
a	identifier <id, 1>
=	Assignment operator
b	identifier <id, 2>
+	Addition operator
c	Identifier <id, 3>
*	Multiplication operator
60	Integer constant

⟹ Tokenization

a = b + c * 60

↓ Lexical Analyzer

<id 1> = <id 2> + <id 3> * 60

Gateway classes

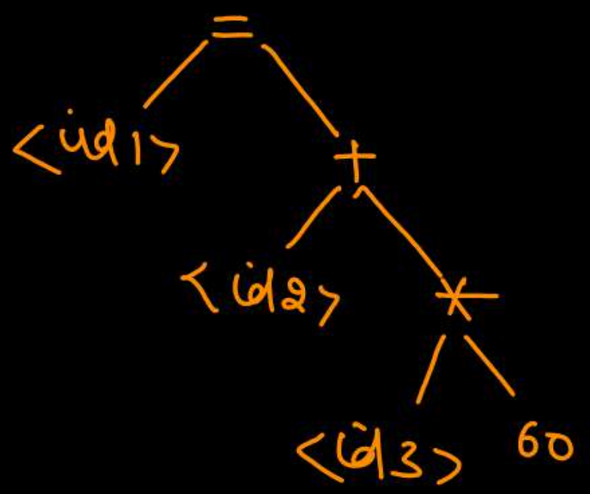
float a, b, c;

a = b + c * 60

↓ lexical Analyzer.

<id1> = <id2> + <id3> * 60

↓ syntax Analyzer.

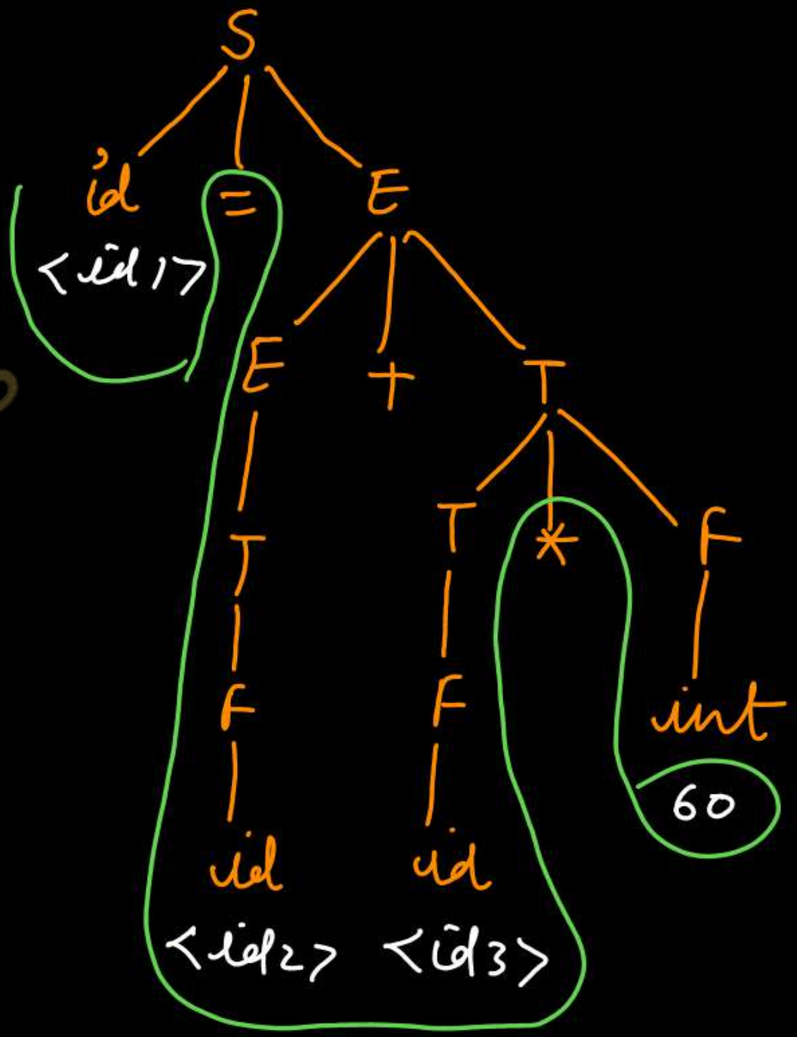


$S \rightarrow id = E$

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id \mid int$

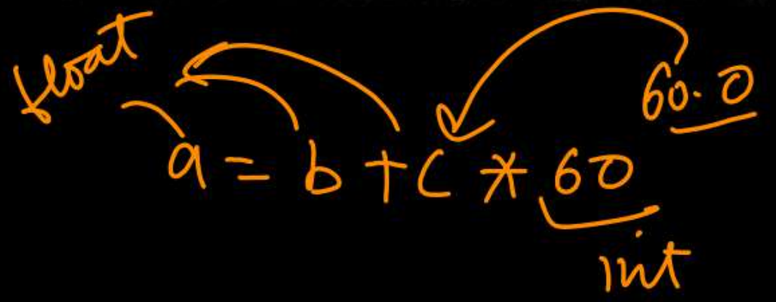


Gateway classes

Phase-3: Semantic Analysis

- Semantic analysis phase ensures the source code makes sense logically.
- It checks whether the program has any semantic errors, such as type mismatches or undeclared variables.
- Both syntax tree of previous phase and symbol table are used to check the consistency of the given code.
- It gathers type information and stores it in symbol table. This type information is subsequently used by compiler during intermediate-code generation.

Multiple declarat



Gateway classes

Semantic Errors:

Errors recognized by semantic analyzer are as follows:

1. Type mismatch: The compiler ensures that operations are performed on compatible data types. For example, many programming language require an array index to be an integer; the compiler must report an error if a floating-point number is used to index an array.
2. Undeclared variables
3. Reserved identifier misuse

float a, b, c;

a = b + c * 60

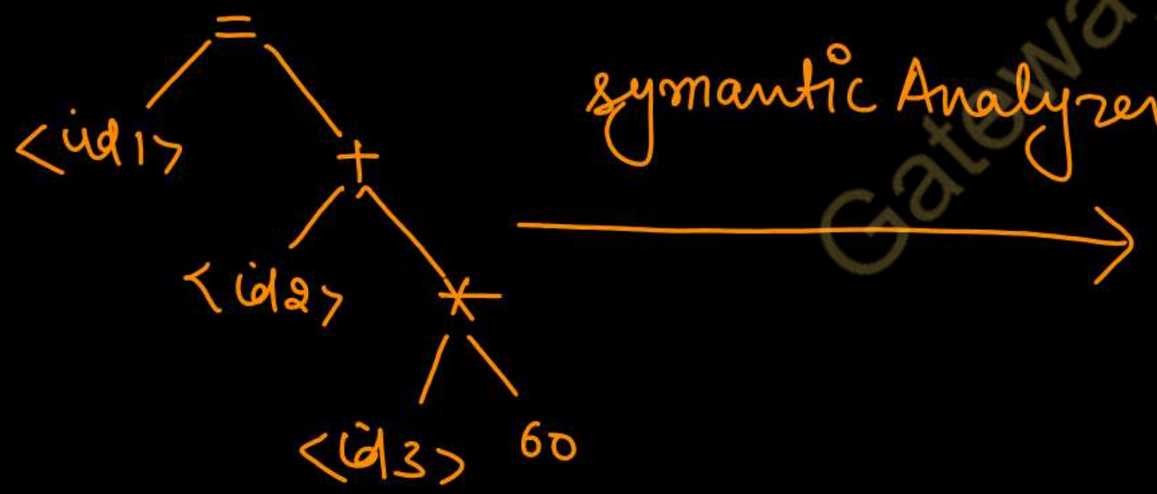
↓ Lexical Analyzer.

<id1> = <id2> + <id3> * 60

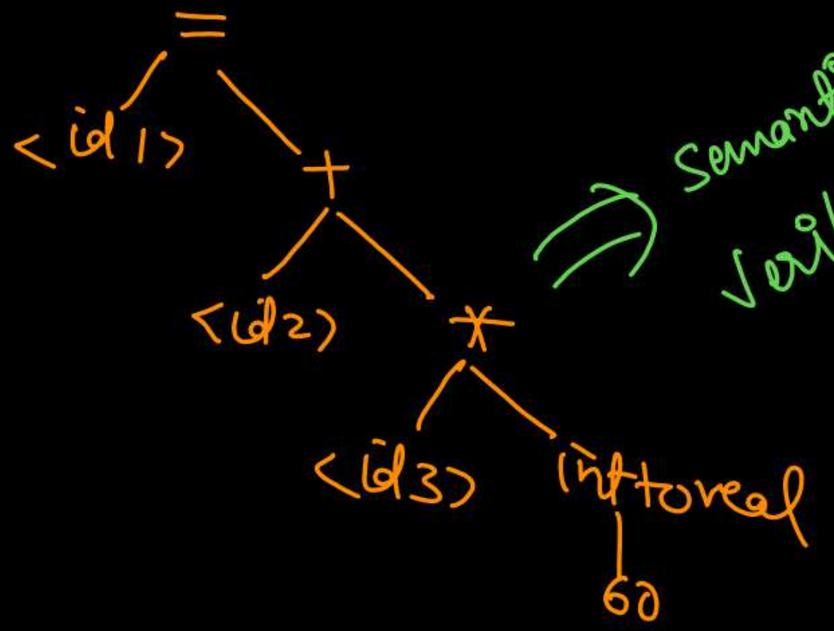
↓ Syntax Analyzer.

S No.	V. name	Type	Scope
1	a	float	global
2	b	float	global
3	c	float	global

adds these info^{n} in Symbol table



Semantic Analyzer



⇒ Semantically Verified Parse tree

Gateway classes

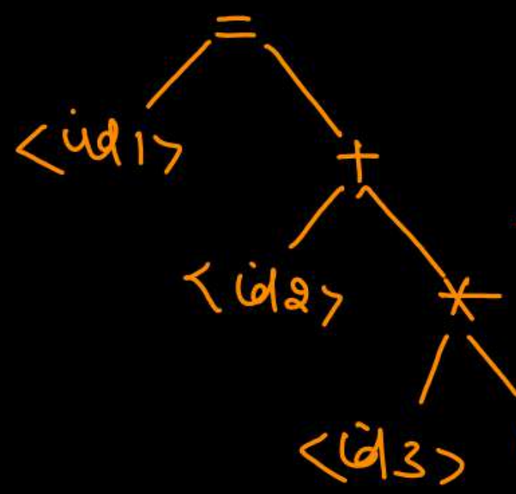
float a, b, c;

a = b + c * 60

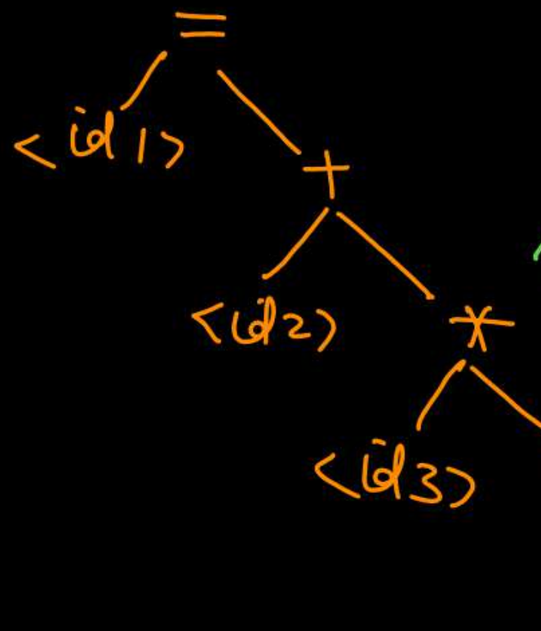
↓ Lexical Analyzer.

<id1> = <id2> + <id3> * 60

↓ Syntax Analyzer.



Semantic Analyzer →



⇒ Semantically verified parse tree

↑ Intermediate code generator

t_1 = Inttoreal(60)
t_2 = id3 * t_1
t_3 = id2 + t_2
id1 = t_3

} 3-address code

Gateway classes

Phase-4: Intermediate Code Generation

- After syntax and semantic analysis of the source program, many compilers generate a machine-like intermediate representation. It is not specific to any particular machine, making it portable and easier to optimize.
- This intermediate representation should have two important properties:
  1. it should be easy to produce.
  2. it should be easy to translate into the target machine.
- In an intermediate form called three-address code, which consists of a sequence of assembly-like instructions with three operands per instruction. Each operand can act like a register.
- The output of the intermediate code generator consists of the three-address code sequence.

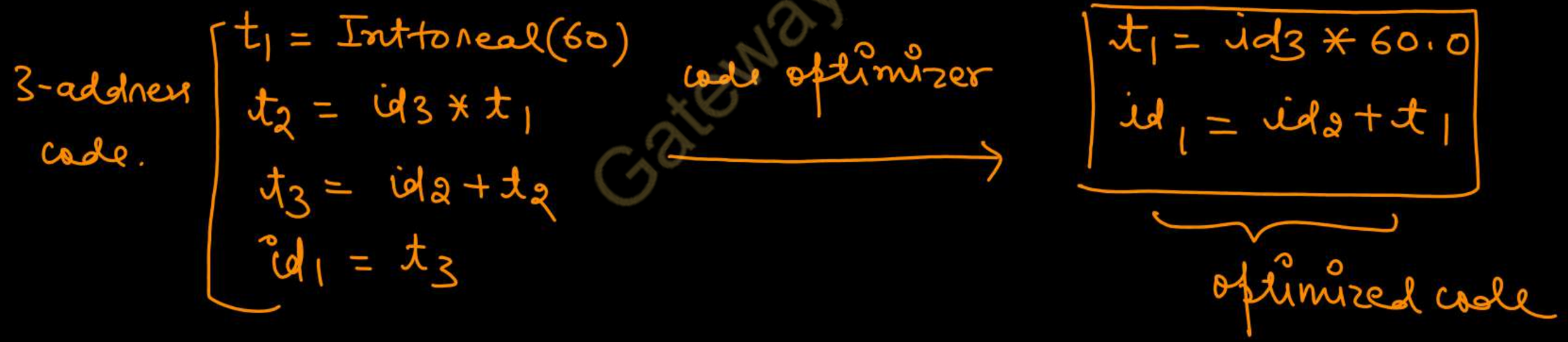
Gateway classes

Phase-5: Code Optimization

Code Optimization is the process of improving the intermediate or target code to make the program run faster, use less memory, or be more efficient, without altering its functionality.

- Involves techniques like removing unnecessary computations, reducing redundancy, and reorganizing code to achieve better performance.

- Optimization is classified broadly into two types Machine-Independent & machine Dependent.



Gateway Classes

Phase-6: Code Generation

- The code generator takes the optimized representation of the intermediate code and maps it to the target machine language.
- The code generator translates the intermediate code into a sequence of (generally) re-locatable machine code.

Symbol Table Management and Error Handling are supporting activities that occur throughout all phases of compilation.

$t_1 = id3 * 60.0$
$id_1 = id_2 + t_1$

code generator →

```
LDF   R1, id3
MULF  R1, R1, #60.0
LDF   R2, id2
ADDF  R2, R1, R2
STF   id1, R2
```

Assembly code

$R_1 \leftarrow id3$
$R_1 \leftarrow R_1 + 60.0$
$R_2 \leftarrow id2$
$R_2 \leftarrow R_1 + R_2$
$id_1 \leftarrow R_2$

Gateway classes

Q: Analyze the role of symbol tables in different phases of compilation. How does symbol table interact with lexical, syntax, and semantic analyzers ? **(AKTU 2024-25)**

- The symbol table is a central data structure used by compiler to store information(Identifier name, Data type, Scope, Memory location) about identifiers.

Compiler Phase	Symbol Table(ST) Operation
Lexical Analysis	Creates symbol table and inserts identifiers with token information
Syntax Analysis	Uses ST for reference; no semantic information is added
Semantic Analysis	Adds type information, scope, and other attributes
Intermediate Code Generation	Uses ST to obtain type and address information
Code Optimization	Uses ST ,no updates are made
Code Generation	Uses ST for memory addresses and data types
Error Handling	Uses ST to report declaration, scope, and type errors

Gateway classes

Error Handling in Phases of Compiler

- Error Handling refers to the mechanism in each phase of the compiler to detect, report and recover from errors without terminating the entire compilation process.

Source Code

↓

Lexical Analysis

- Reads characters and forms tokens, Keeps track of line numbers & column positions
- Detects lexical errors (invalid characters, unknown tokens)
- Tokens + Line & Position Info ——> Error Handler reports lexical errors

↓

Syntax Analysis (Parser)

- Detects syntax errors (missing symbols, wrong structure)
- Error Handler uses token line & position info to reports syntax errors with pointer to error location

↓

Semantic Analysis

- Error Handler reports semantic errors like Type mismatch, Undeclared variables with line & column info
- Example: "Semantic Error: Variable 'b' not declared"

Front End Compilation Continues: - If possible, performs error recovery.

Gateway classes

Error Handling in Phases of Compiler contd..

Intermediate Code Generation

- Detects back-end errors such as unsupported operations

↓

Optimization Phase

-Detects optimization-related errors such as illegal transformations

↓

Code Generation

- Detects code generation errors like
  - Register allocation issues
  - Memory access violations
  - Target architecture constraints

↓

Error Handler reports back-end errors with reference to source or IR location

- Example: "Code Generation Error: Not enough registers for variable 'a'"

↓

Successful code generation if no unrecoverable errors

Otherwise, compilation terminates with errors

Gateway classes

Compiler Design

Unit-1 : DPP-1

1. Explain the complete process of compilation for the statement **(AKTU 2021–22)**

a = b + c * 70

float x, y, z; x = y + z * 60

2. Explain the function of error handler in the phases of compiler. **(AKTU 2020–21)**

3. Mention the role of semantic analysis. **(AKTU 2022–23)**

4. Difference between compiler and assembler. **(AKTU 2017–18)**

5. Explain term language translator and compiler. **(AKTU 2021–22)**

6. What are the various errors that occur in the compilation process. **(AKTU 2021–22)**

7. What are the two parts of compiler? Explain briefly. **(AKTU 2018–19)**

8. Explain the various errors that occur in compilation process **or** explain the syntax phase and lexical phase and tell the errors that come in these phases. **(AKTU 2021–22,2022–23,2020–21)**

9. Explain in detail the process of compilation. Illustrate the output of each phase of compilation of the input

***a* = (*b* + *c*)*(*b* + *c*)* 2** **(AKTU 2024-25,2017-18)**

Gatewayclasses

AKTU

B. Tech Sem-VI

CS IT & CS Allied

GATEWAY CLASSES

Compiler Design

UNIT-1 Lecture-2

Gateway Classes

Todays Target

- Errors during compilation process
- Grouping of compiler phases
- Classification of Compiler
- Bootstrapping
- Cross Compiler
- AKTU PYQ



By Dr. Nidhi Parashar Ma'am

- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified

Q: Demonstrate the process of compilation with help of following input: c = a*b+10 **AKTU 2023-24)**

$C = a + b * 10$

↓

lexical Analyzer

c	identifier <id,1>
=	Assignment operator
a	identifier <id,2>
+	Addition operator
b	identifier <id,3>
*	Multiplication operator
10	Integer constant

⇓

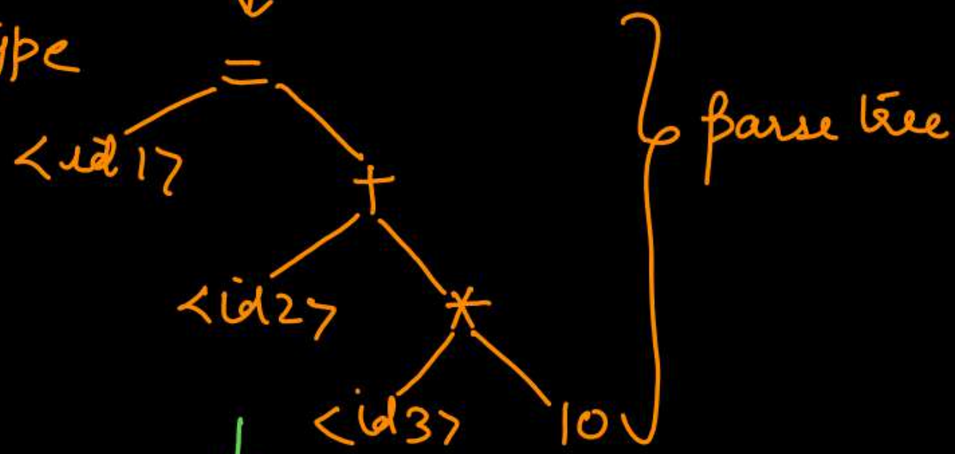
$\langle id_1 \rangle = \langle id_2 \rangle + \langle id_3 \rangle * 10$

S.No.	V.name	Data Type
1.	c	int
2.	a	int
3.	b	int

$\langle id1 \rangle = \langle id2 \rangle + \langle id3 \rangle * 10$

↓

Syntax Analyzer

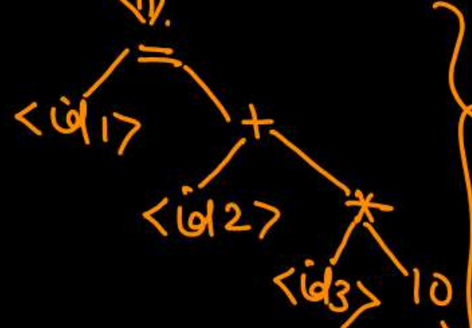


parse tree

↓

Semantic Analyzer

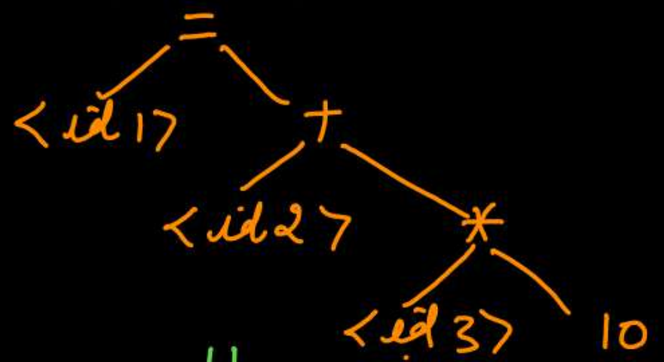
⇓



semantically verified parse tree

Gateway Classes

Q: Demonstrate the process of compilation with help of following input: c = a*b+10 **AKTU 2023-24)**



intermediate code generation

$t_1 = id3 * 10$

$t_2 = id2 + t_1$

$id_1 = t_2$

code optimizer

$t_1 = id_3 * 10$

$id_1 = id_2 + t_1$

optimized code

code generation

```
LOAD R1, id3
MUL R1, R1, #10
LOAD R2, id2
ADD R2, R1, R2
STORE id1, R2
```

Target Assembly code

Gateway Classes

Q: Explain in detail the process of compilation. Illustrate the output of each phase of compilation of the input $a = (b + c)*(b + c)* 2$ **AKTU 2024-25,2017-18)**

$a = (b+c)*(b+c)*2$

↓

1. Lexical Analyzer

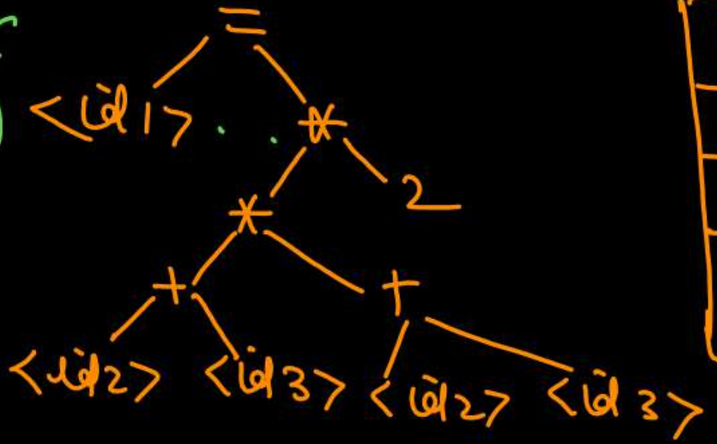
↓

<id1> = (<id2> + <id3>) * (<id2> + <id3>) * 2

↓

2. Syntax Analyzer

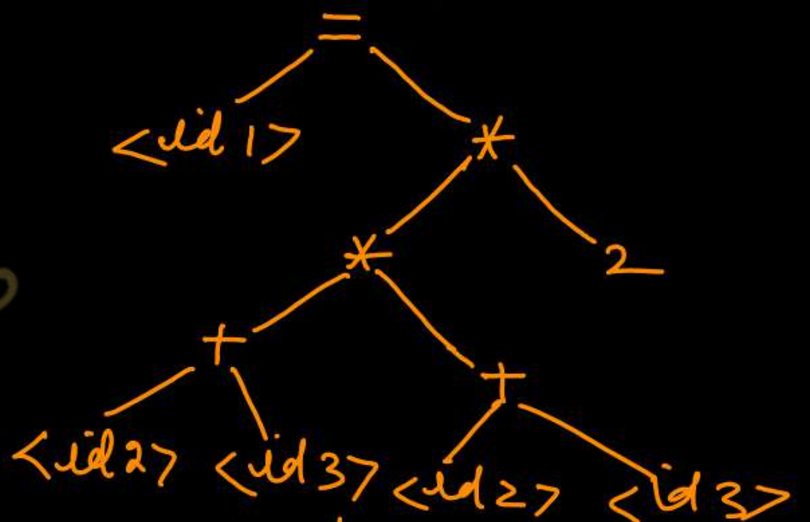
parse tree



Symbol Table

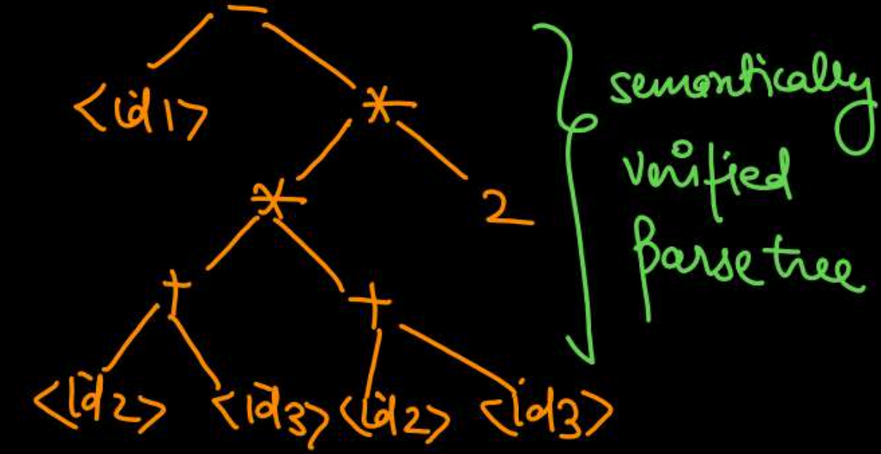
S.No.	V.name
1.	a
2.	b
3.	c

Gateway Classes

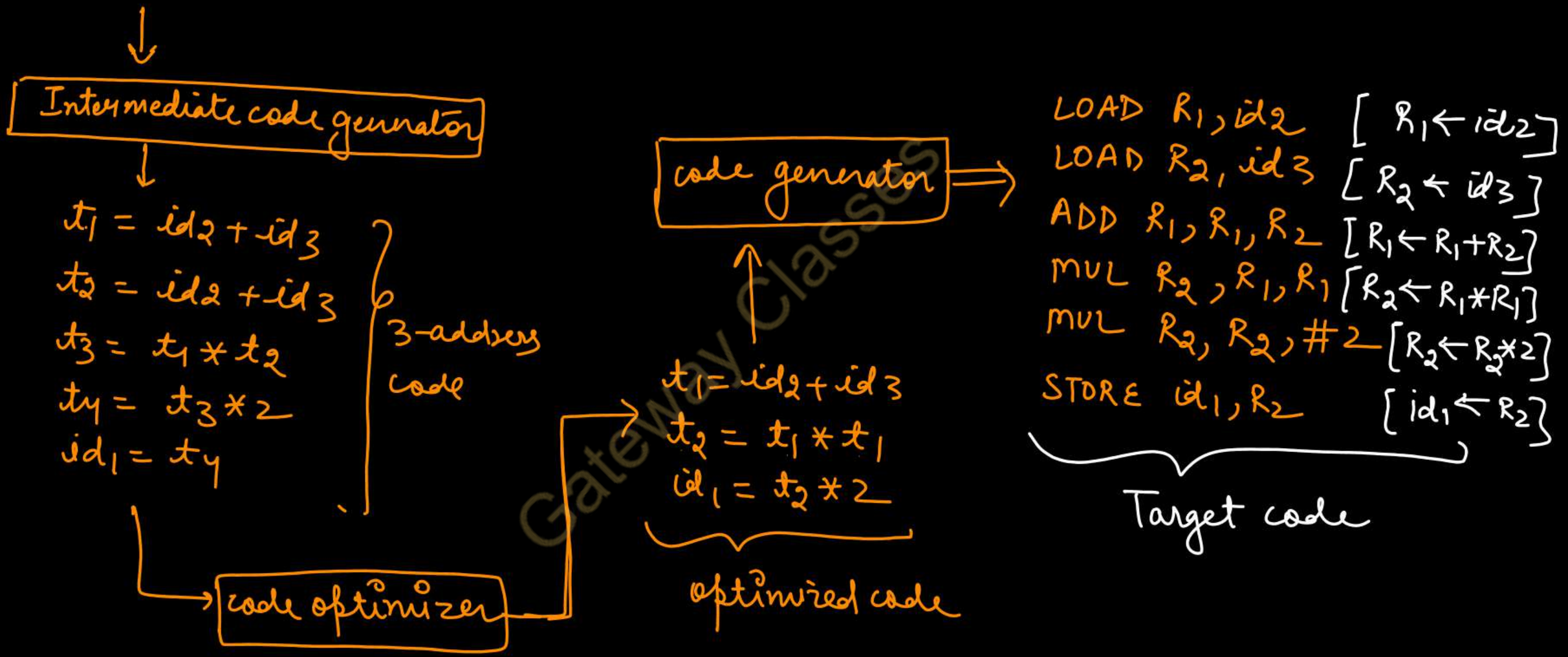


↓

3. Semantic Analyzer



Q: Explain in detail the process of compilation. Illustrate the output of each phase of compilation of the input $a = (b + c)*(b + c)* 2$ **AKTU 2024-25,2017-18)**



Gateway Classes

Q: What are the various errors that may appear in compilation process? **AKTU 2021-22,2020-21)**

Phase	Type of Errors
Lexical Phase Errors	➢ Misspellings of identifiers, keywords, or operators ➢ Missing quotes around text intended as a string.
Syntactic Phase Errors	➢ Unbalanced parenthesis or missing semicolon ➢ Errors in structure ➢ Missing operators
Semantic Phase Errors	These errors occur when the statements are syntactically correct but do not make sense semantically according to the language rules. ➢ Incompatible value assignment ➢ Undeclared variables ➢ Actual arguments not matching formal arguments

Gateway Classes

Types of Lexical Errors:

1. Replacing a character with an incorrect character.

```
int main()
{    int x = 12@34;
     return 0;
}
```

lexical error as '@' doesn't belong within 0-9 range

2. Appearance of illegal characters

```
int main()
{   printf("hello");$
}
```

Lexical error as an illegal character $ appears at end of statement.

3. Spelling Error

```
int main()
{    int 3num= 1234;
}
```

spelling error as identifier cannot start with a number

4. Unmatched string

/* . . . . . */

```
int main()
{/* comment
   printf("hello");
}
```

This is a lexical error since the ending of comment "*/" is not present but the beginning is present.

Gateway Classes

Types of syntactic Errors: (Syntax Analysis)

Errors in structure

```
if (x > y)
        printf("Hello");
else
```

(else without a corresponding statement)

Missing operator

```
int x = 5 10;
// missing operator (+, -, *, /)
```

Unbalanced parentheses

```
if (x > 0
{
// missing ')'
```

Missing punctuation symbols

Such as semicolons, commas, or colons.

Example:

```
int x = 10 // missing semicolon
```

Misspelled keywords

```
whlie (x < 10)
// should be while
```

Gateway Classes

Types of semantic Errors:

1. Incompatible type of operands

```
int a = 5;
int *p;
int x = a + p;   // incompatible types: int and pointer
```

2. Undeclared variables

Occurs when a variable is used without being declared.

Example:

```
int a = 10;
b = a + 5;   // semantic error: 'b' is not declared
```

3. Invalid assignment

Occurs when incompatible types are used in assignment.

Example:

```
int x;
x = "Hello";   // semantic error: assigning string to int
```

4. Mismatch between actual and formal arguments

```
void display(int x)
{
    printf("%d", x);
}
int main()
{
    display(5.5);   // semantic error: float passed to int
    return 0;
}
```

Example (Number of arguments mismatch):

```
void sum(int a, int b) {
    printf("%d", a + b);
}
int main() {
    sum(10);   // semantic error: missing one argument
    return 0;
}
```

Gateway Classes

How will you group the phases of compiler? AKTU 2022-23)

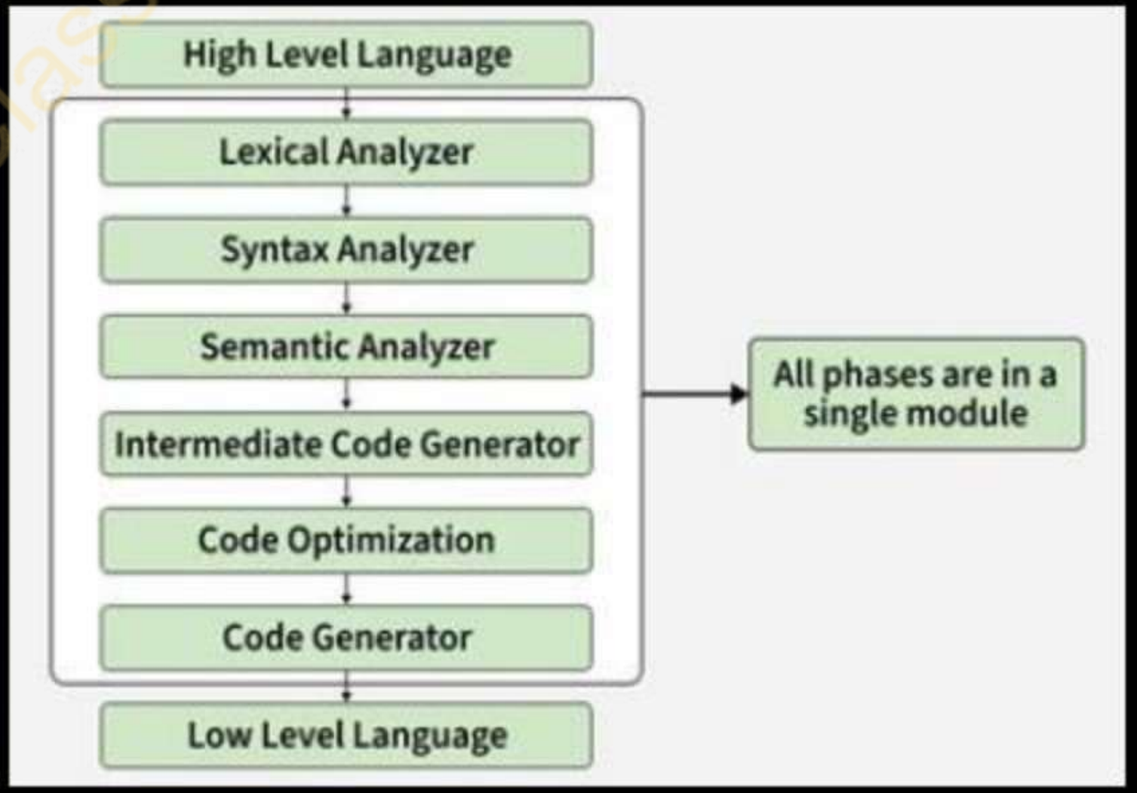
Several phases are grouped together to a pass so that it can read the input file and write an output file.

Compiler Passes

During compilation, a compiler may traverse the source program or its intermediate representation one or more times. Each such traversal is known as a compiler pass.

Types of Compiler Passes

1. Single Pass Compiler
   - It reads the source code only once and completes all compilation steps during this single read.
   - Since it produces output immediately without going back to the code, the compilation is fast.
   - Simple compiler structure
   - Uses less memory
   - Limited optimization and error handling

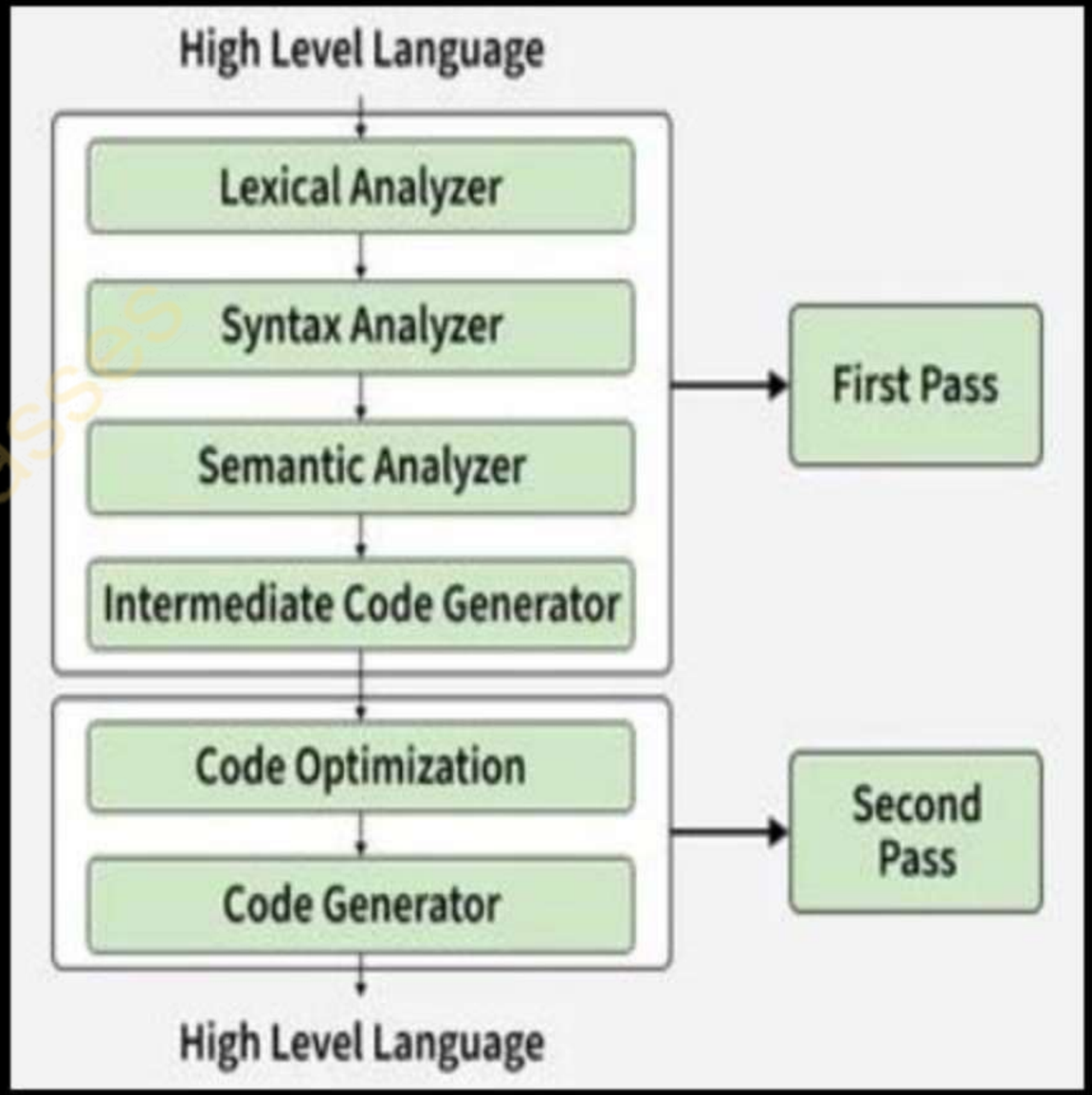


Gateway Classes

Two-Pass compiler *or* Multi-Pass compiler

- A Two pass/multi-pass Compiler processes the *source code* of a program multiple times.
- In multi-pass Compiler, we divide phases into two passes as:

1. **First Pass** referred as
   - Front end
   - Analytic part
   - Platform independent
2. **Second Pass** referred as
   - Back end
   - Synthesis Part
   - Platform Dependent



Gateway Classes

Difference Between One Pass and Two Pass Compiler

One-Pass Compiler	Multi-Pass Compiler
It reads the code only once and translates it at a similar time.	It reads the code multiple times, each time changing it into numerous forms.
Faster.	Slower.
Less efficient code optimization and code generation.	Better code optimization and code generation.
It is also called a "Narrow compiler." It has limited scope.	It is also called a "wide compiler." As they can scan every portion of the program.
There is no intermediate code generation.	The intermediate code is generated during the process.
Example: C and Pascal uses One Pass Compiler.	Example: Modula-2 uses Multi Pass Compiler.

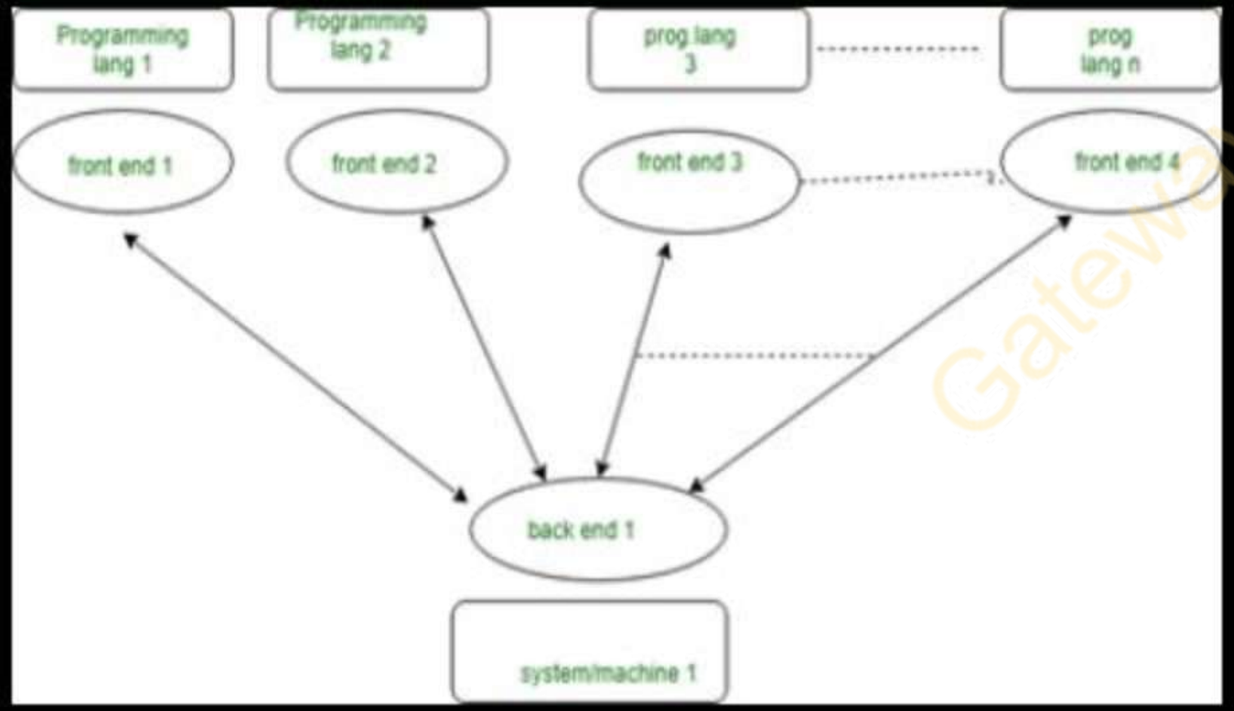
Gateway Classes

Reasons for Grouping Compiler Phases AKTU 2016-17)

1. Modularity and Code Reuse: Grouping compiler phases into Front End (Analysis) and Back End (Synthesis) enables code reuse across different compilers.

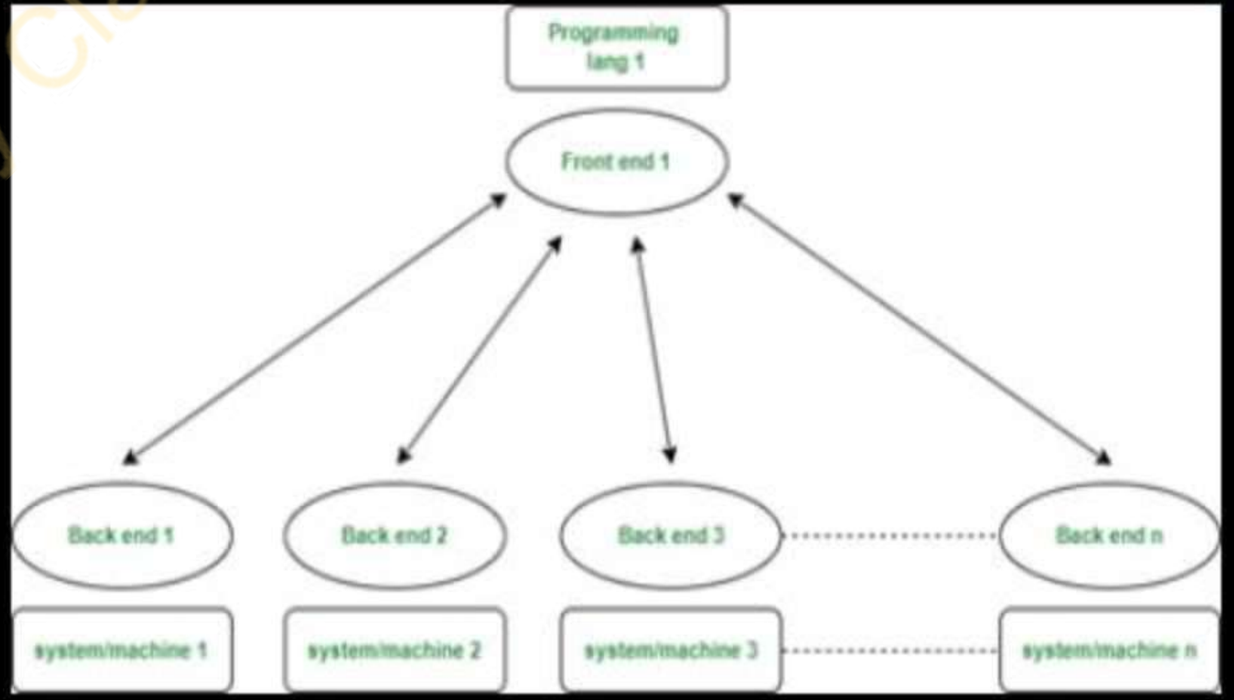
To design a compiler for different programming languages for same machine.

- Front end/first pass for each PL.
- Only one Back end/second pass



To design a compiler for same programming language for different machines.

- One Front end for same PL.
- Different Back end for diff machines



Gateway Classes

2. Simplifies Design and Maintenance:

- Separating compiler functionality into well-defined groups simplifies the overall design.
- Each group can be developed, tested, debugged, and optimized independently, making the compiler easier to maintain and extend.

3. Faster error detection and reporting

Errors can be detected earlier and reported more accurately when phases work together.

Q: Differentiate between phases and passes of a compiler **AKTU 2024-25)**

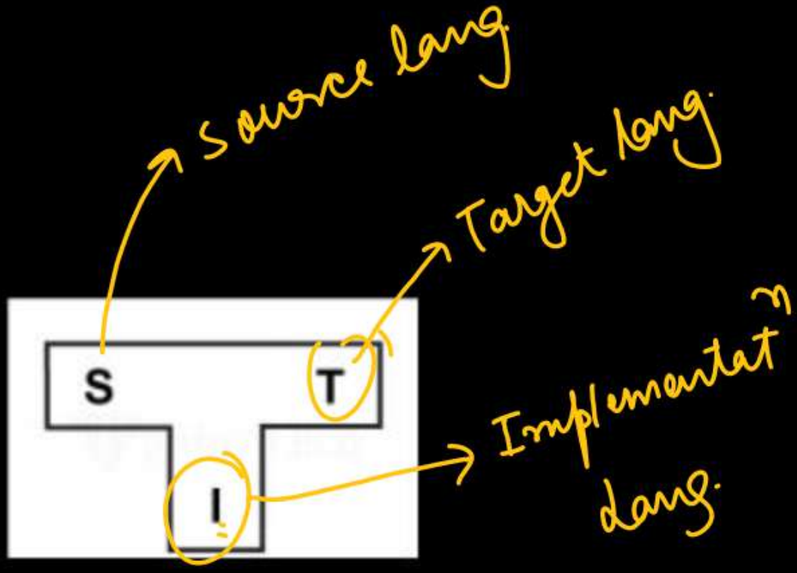
Gateway Classes

1. Phases are the steps in the compilation process while passes are the number of times the compiler traverses through the source code.
2. There are six main phases in the compilation process while there are two types of compilers as single pass and multi-pass compilers. Hence, this is another difference between phases and passes of compiler.

Bootstrapping:

- Bootstrapping is a technique where a basic compiler is used to create a more advanced version of itself. This process helps in building compiler for new programming languages and improving the ones already in use.
- It relies on idea of a self-compiling compiler where each iteration improves the compiler's ability to handle more complex code.
- Self-compiling compiler/ self-hosting compiler : A type of compiler that can compile its own source code or in other words a compiler that is **written in the same programming language that it compiles**.
- A compiler can be characterized by three languages:
  1. Source Language
  2. Target Language
  3. Implementation Language

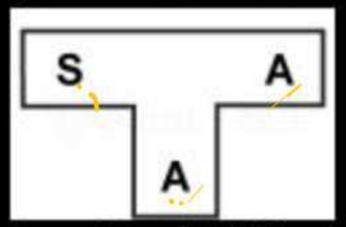
Gateway Classes



- T- diagram shows a compiler ${}^{S}_{I}{}^{T}$ for Source S, Target T, implemented in I.

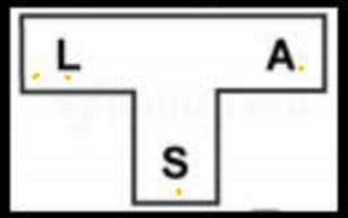
Follow some steps to produce a new language L for machine A:

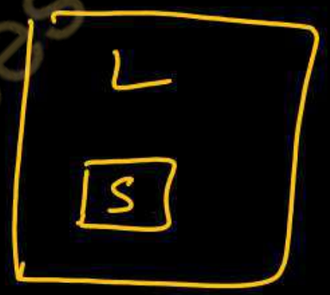
1.Create compiler ${}^{S}_{A}{}^{A}$ for subset S of desired language L ,using language "A" and that compiler runs on machine A.



$\Rightarrow S_A A$

2.Create a compiler ${}^{L}_{S}{}^{A}$ for language L written in a subset of L.

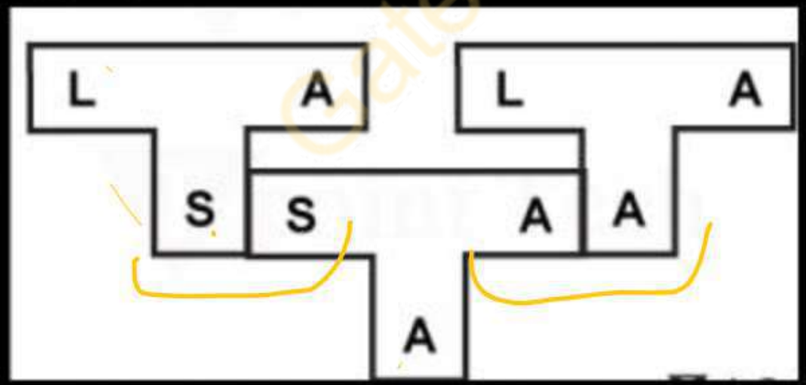




3. **Compile ${}^{L}_{S}{}^{A}$ using the compiler ${}^{S}_{A}{}^{A}$ to obtain ${}^{L}_{A}{}^{A}$.**

${}^{L}_{A}{}^{A}$ is a compiler for language L, which runs on machine A and produces code for machine A.

$$ {}^{L}C^{A}_{S} \rightarrow {}^{S}C^{A}_{A} \rightarrow {}^{L}C^{A}_{A} $$



The process described by the T-diagrams is called bootstrapping

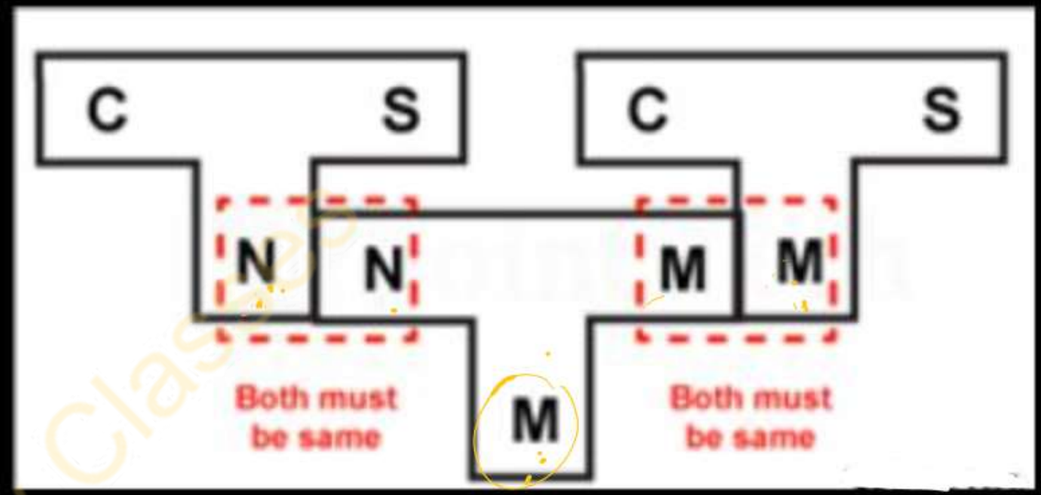
Gateway Classes

Cross compiler

A **cross compiler** is a compiler that runs on one machine but generates code for another machine.

To design a cross compiler which:

- Runs on machine N
- Generates code for machine M



- The compiler:
  - Runs on machine N
  - Takes a C program and Produces S code
  - That S code runs on machine M
- **Because N ≠ M, this is a cross compiler.**

- **C** → Source language (example: C)
- **S** → Target language (output code)
- **N** → Host machine (where compiler runs)
- **M** → Target machine (where program will run)

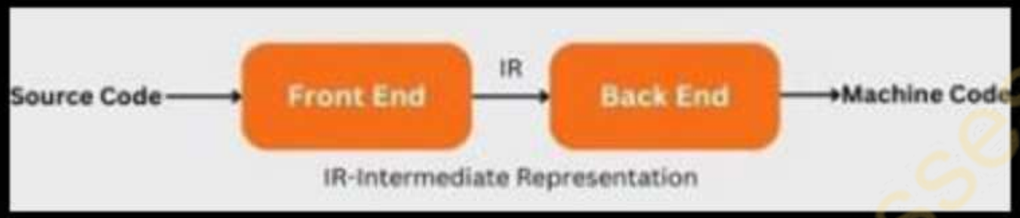
Gateway Classes

Classification of Compiler . AKTU 2018-19)

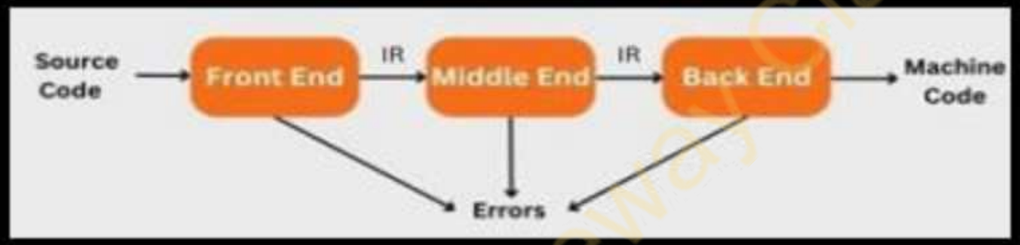
Single-pass Compiler:



Two pass Compiler:



Multi pass Compiler:



Native Compiler: Native compilers generate target code for the same platform on which it runs. For example, Turbo C or GCC compiler. if a compiler runs on a Windows machine and produces executable code for Windows, it is a native compiler.

Cross Compiler: A cross-compiler generates executable code for a platform other than the one on which the compiler is running. For example, if a compiler runs on a Linux machine and produces executable code for Windows, then it is a cross-compiler.

Gateway Classes

Compiler Design
Unit-1 : DPP-2

1. What are the various errors that occur in the compilation process. **(AKTU 2021–22)**
2. Explain the complete process of compilation for the statement **(AKTU 2021–22)**

   a = b + c * 70

   float x, y, z; x = y + z * 60

3. Explain in detail the process of compilation. Illustrate the output of each phase of compilation of the input

   ***a* = (*b* + *c*)*(*b* + *c*)* 2** **(AKTU 2024-25,2017-18)**

4. Discuss about the cross compiler. **(AKTU 2020–21)**
5. What are the classification of compiler? **(AKTU 2018–19)**
6. State any two reasons as to why phases of compiler should be grouped. **(AKTU 2016–17)**
7. What is bootstrapping? Explain in detail. **(AKTU 2018–19)**
8. What is cross compiler? How is bootstrapping of a compiler done to a second machine. **(AKTU 2017–18)**

Gateway Classes

AKTU

B. Tech Sem-VI

CS IT & CS Allied

GATEWAY CLASSES

Compiler Design

UNIT-1 Lecture-3 (Part 1 + Part 2)

Gateway Classes

Todays Target

- Finite Automaton (FA)
- Deterministic Finite Automata
- DFA Construction
- Minimization of DFA
- Non-deterministic Finite Automata
- NDFA to DFA conversion
- AKTY PYQs



By Dr. Nidhi Parashar Ma'am

- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified

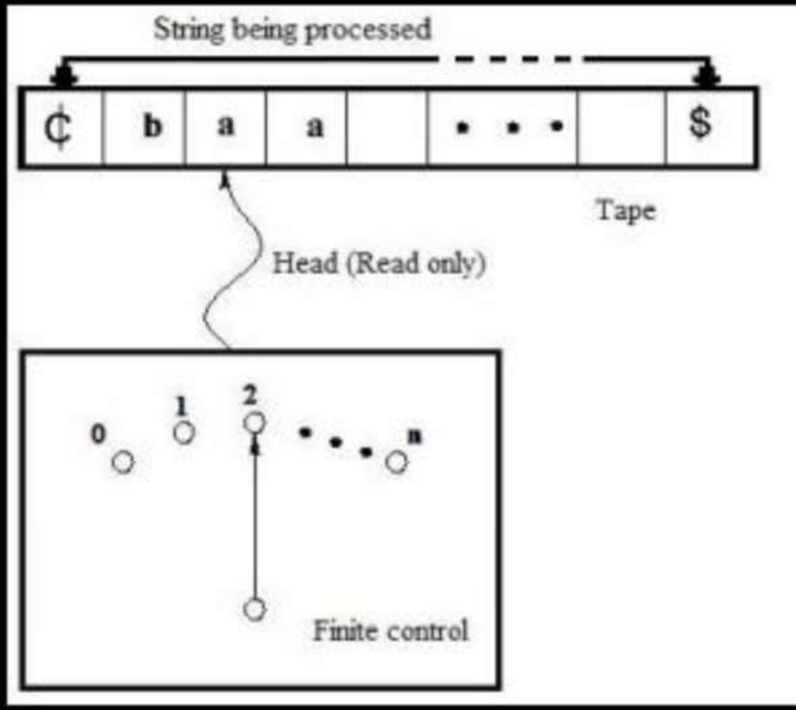
Finite Automaton (FA)

- A **Finite Automaton** is a mathematical model of computation used to recognize **regular languages**.
- It consists of a finite number of states and transitions between those states based on input symbols.
- Memory is limited to the **current state only**. FA have **no stack/ auxiliary memory**.

Components of finite automata:

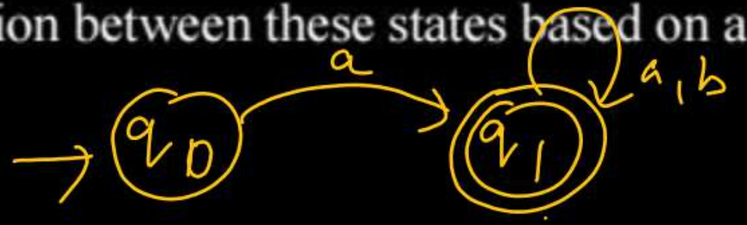
1. **Input tape:** The left-to-right sequence of symbols between two end-markers is the input string to be processed.
2. **Read only Head:** It reads one input symbol at a time and moves only from left to right, starting at the leftmost square of the input tape. When no symbol remains on the right, the machine halts and the automaton terminates.
3. **Finite control:** It controls the operation of the automaton by deciding the next state $\delta(q, a)$ and the movement of the read head based on the current state and input symbol.

Gateway Classes



Formal Definition of Finite Automaton

- A finite automata is an abstract machine used to model computation.
- It consists of a finite number of states and operates on input symbols to transition between these states based on a set of rules.



To represent the finite automata, we need 5 tuples – M=(Q,Σ,δ,q0,F)

- States (Q) – Q is a finite, non-empty set of states. $Q = \{q_0, q_1\}$
- Input Alphabets (Σ) – Σ is finite, non-empty set of input symbols. $\Sigma = \{a, b\}$
- Transition Function (δ) – δ is a set of functions that defines how the automaton transitions from one state to another based on the current state and input symbol. It can be expressed as: $\delta: Q \times \Sigma \rightarrow Q$. $\delta(q_0, a) = q_1$
- Initial State (q0) – q0 is the initial state. → q_0
- Final States (F) – F is a set of one or more final states.

Gateway Classes

Types of Finite Automata

1. Finite Automata without output –
   a) Deterministic Finite Automata (DFA)
   b) Non-Deterministic Finite Automata (NFA or NDFA)
   c) Non-Deterministic Finite Automata with epsilon moves (∈-NFA)
2. Finite Automata with Output –
   a) Moore Machine
   b) Mealy Machine

Deterministic Finite Automata (DFA)

A Deterministic Finite automata is defined as a 5-tuples.

M=(Q,Σ,δ,q0,F) where,

- Q: Finite set called states.
- Σ: Finite set called alphabets.
- δ: $Q \times \Sigma \rightarrow Q$ is the transition function.
- $q0 \in Q$ is the start or initial state.
- F: Final or accept state.

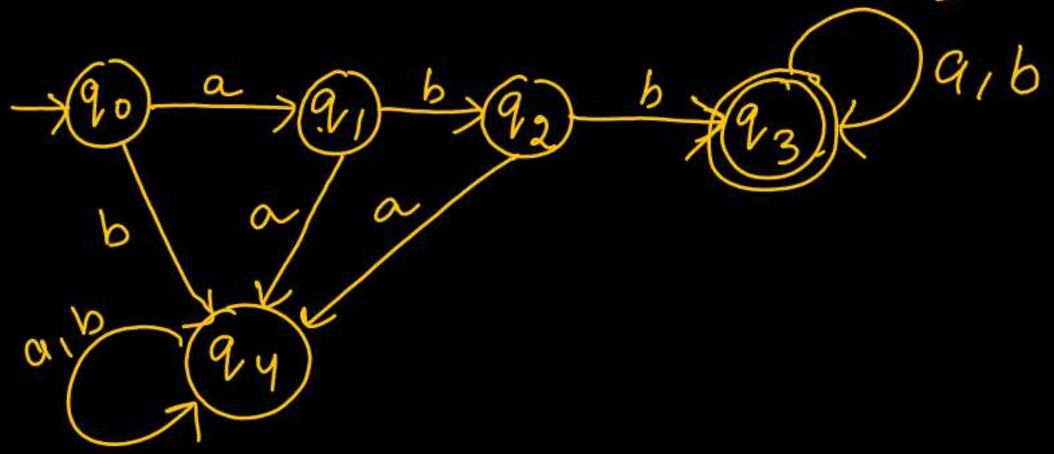
- In DFA, for each input symbol, machine transitions to one and only one state.
- DFA does not allow any null transitions, meaning every state must have a transition defined for every input symbol.

Gateway Classes

$\Sigma = \{a, b\}$

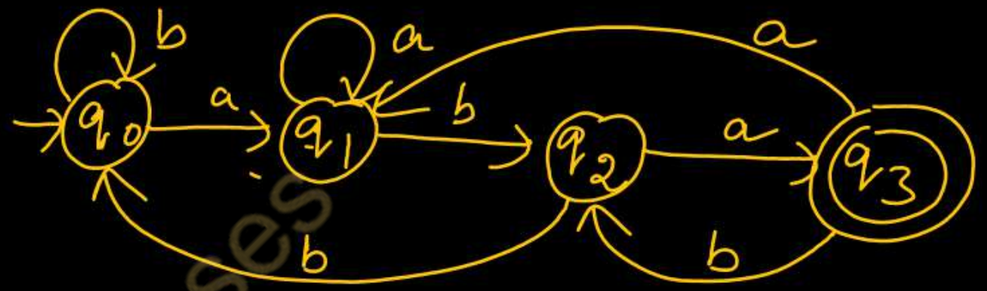
Construct DFA that accepts all strings starting with 'abb'

$L = \{ abb, abba, abbbba \ldots\ldots \}$



Construct DFA that accepts all strings ending with 'aba'.

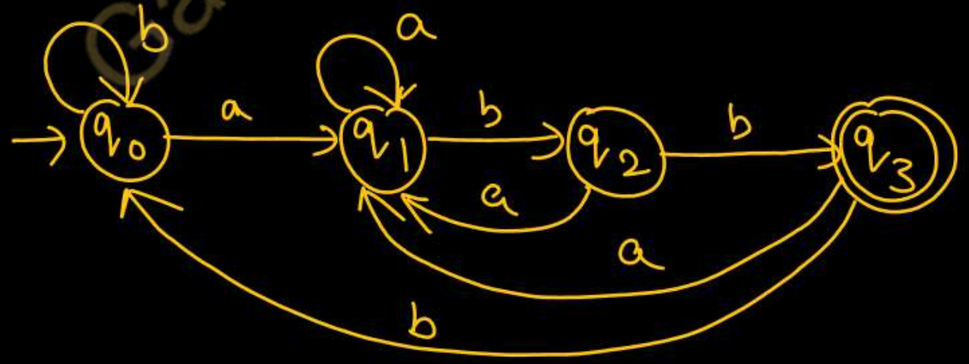
$L = \{ aba, aaba, ababa, bbaba \ldots\ldots \}$



	a	b
→q_0	q_1	q_4
q_1	q_4	q_2
q_2	q_4	q_3
*q_3	q_3	q_3
q_4	q_4	q_4

Construct DFA that accepts all strings ending with 'abb'

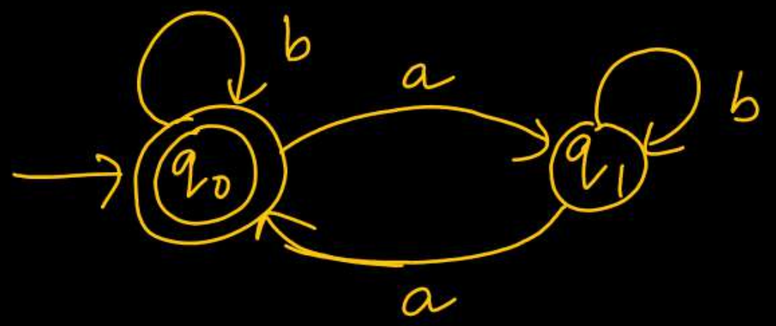
$L = \{ abb, aabb, baabb \ldots\ldots \}$



Gateway Classes

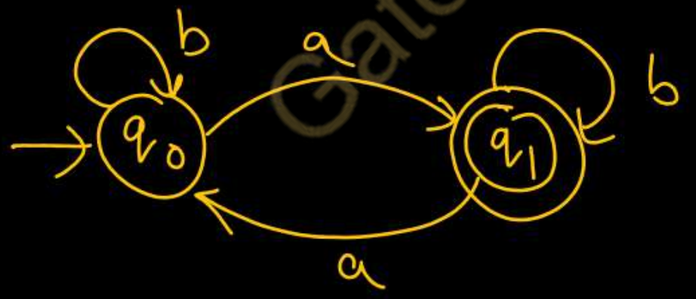
Construct DFA that accepts all strings containing even number of a.

$L = \{ b, aa, abab, aba, abba \ldots \}$



Construct DFA that accepts all strings containing odd number of a.

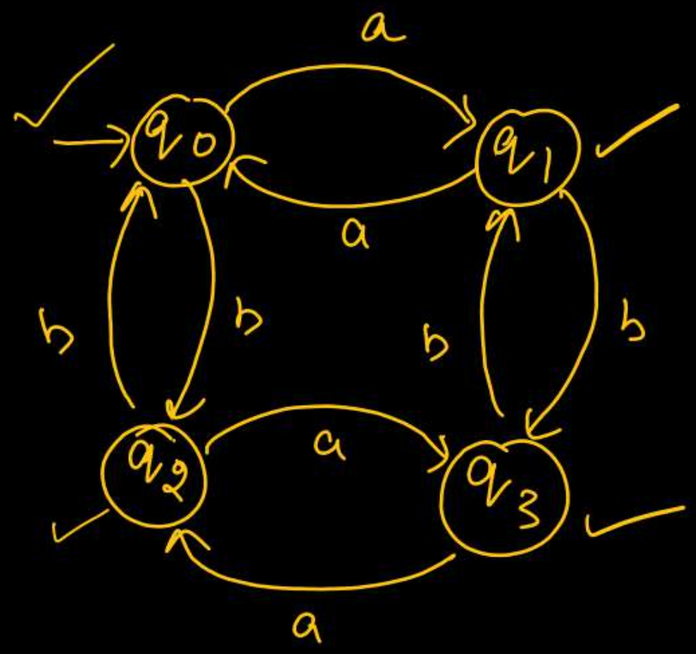
$L = \{ a, abaa, bab, babaa \ldots \}$



Gateway Classes

Q: Design a FSM that accepts all strings of *a* and *b* containing even number of a's and odd number of b's.

(AKTU 2024-25)



No of a's	No. of b's.	Final state
even	even	q_0
odd	even	q_1
even	odd	q_2
odd	odd	q_3

Gateway Classes

Minimization of DFA

- DFA minimization convert a given DFA to its equivalent DFA with minimum number of states.
- Also called as optimization of DFA.

Minimization of DFA Using Equivalence Theorem-

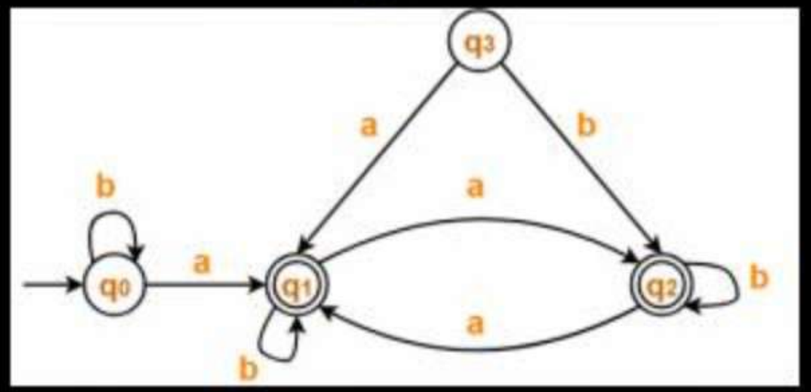
To start with:

→ Eliminate all equivalent states if any.

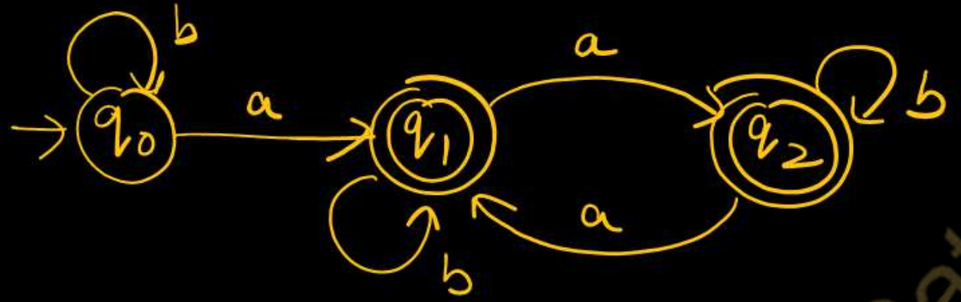
- Eliminate all the unreachable states(All those states which can never be reached from the initial state are called as unreachable states) from the given DFA (if any).

Gateway Classes

Minimize the given DFA-

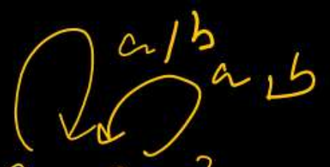


1) Remove unreachable state q_3



States	a	b
→ q_0	q_1	q_0
* q_1	q_2	q_1
* q_2	q_1	q_2

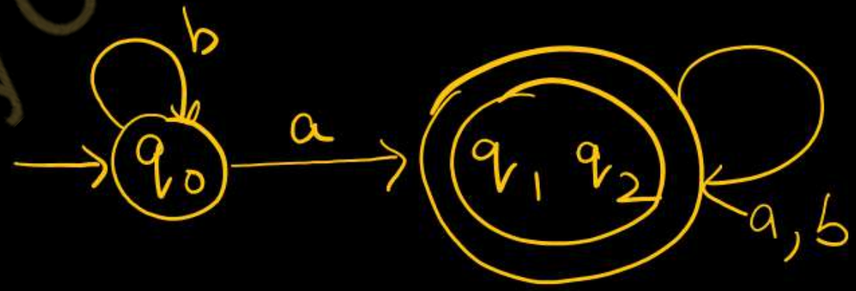
Gateway Classes



$\pi_0 = \{q_0\}\ \{q_1, q_2\}$

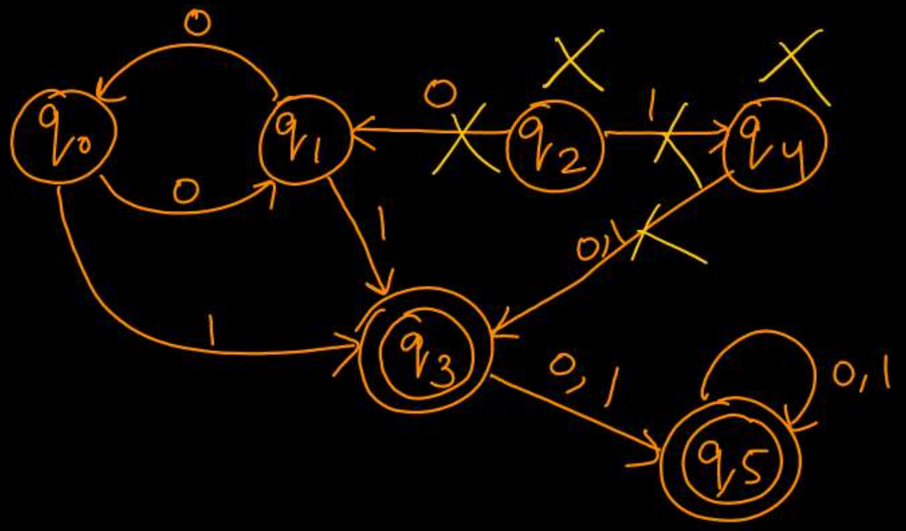
$\pi_1 = \{q_0\}\ \{q_1, q_2\}$ → states can be merged.

As $\pi_0 = \pi_1$ so stop here

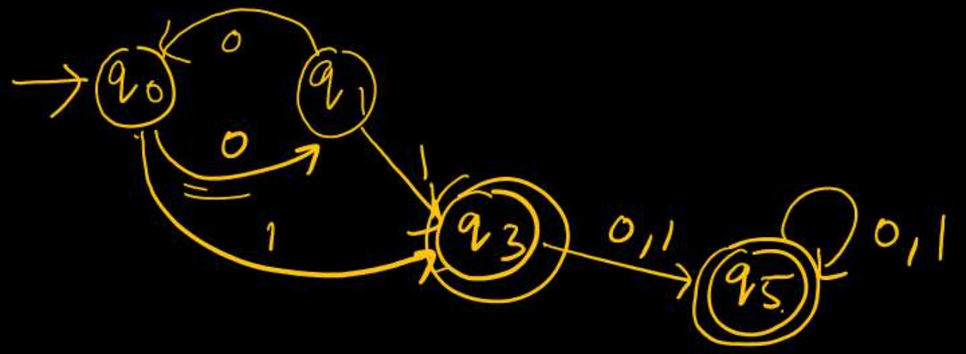


Minimized DFA

Minimize the given DFA-



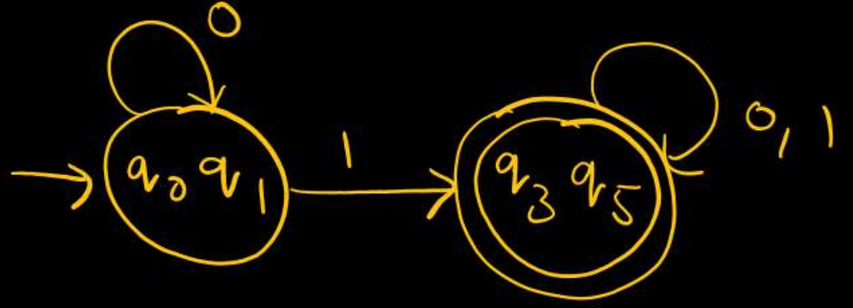
Step-1 Remove states (q_2) and (q_4) as they are unreachable from initial state (q_0)



$\pi_0 = \{q_0, q_1\}$ $\{q_3, q_5\}$

$\pi_1 = \{q_0, q_1\}$ $\{q_3, q_5\}$

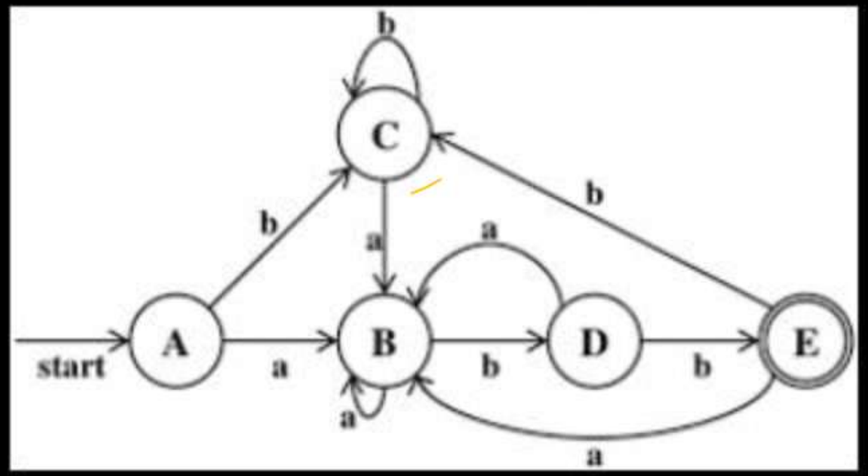
As $\pi_0 = \pi_1$



MDFA

Gateway Classes

Q: Construct minimized DFA: **(AKTU 2023-24)**



Step -1 No unreachable state

	a	b
→A	B	C
B	B	D
C	B	C
D	B	E
*E	B	C

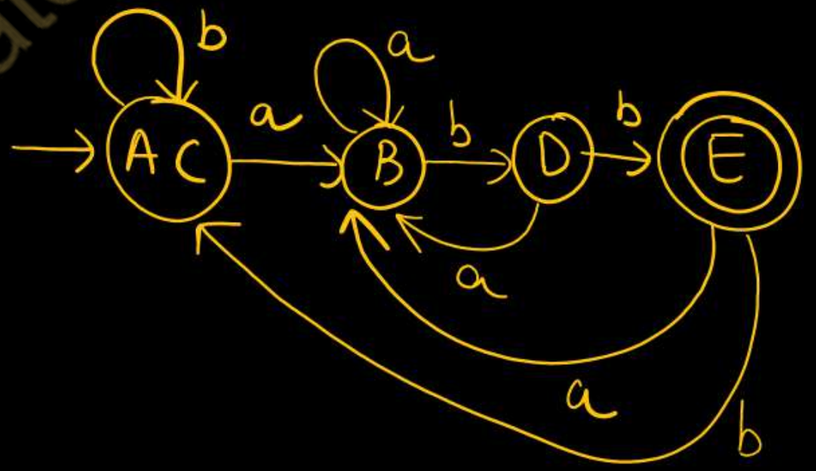
$\pi_0 = \{A, B, C, D\}\{E\}$

$\pi_1 = \{A, B, C\}\{D\}\{E\}$

$\pi_2 = \{A, C\}\{B\}\{D\}\{E\}$

$\pi_3 = \{A, C\}\{B\}\{D\}\{E\}$

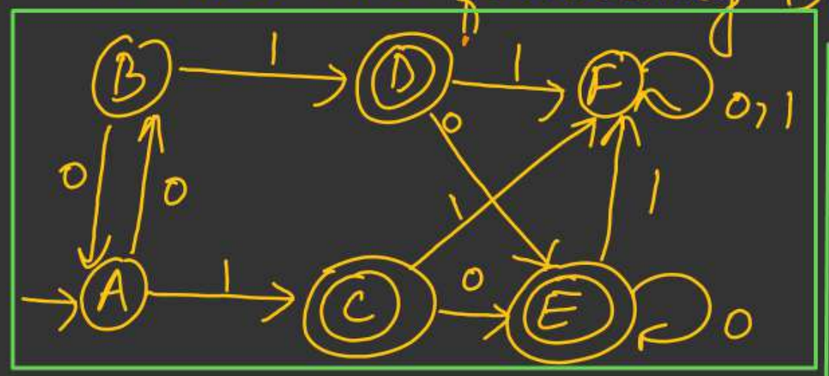
$\pi_2 = \pi_3$



MDFA

Gateway Classes

Q- Minimize the following DFA:-



	0	1
→A	B	C
B	A	D
*C	E	F
*D	E	F
*E	E	F
F	F	F

G_1 G_2

$\pi_0 = \{A, B, F\}\{C, D, E\}$

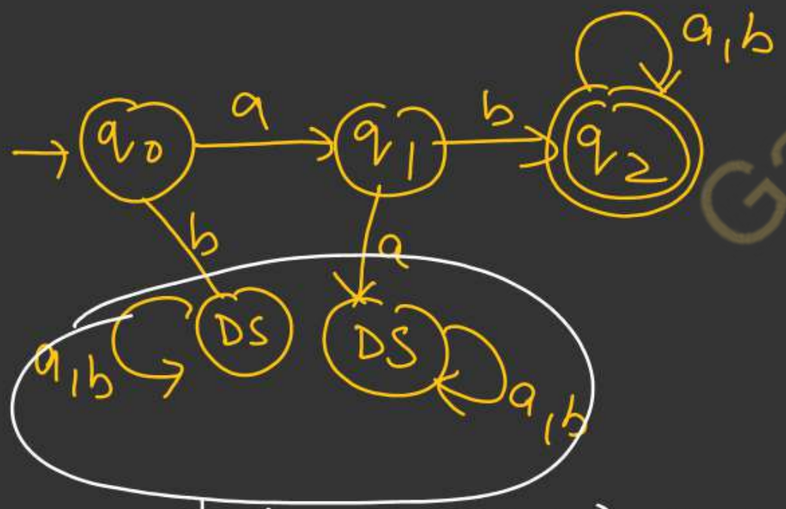
$\pi_1 = \{A, B\}\{F\}\{C, D, E\}$ (G_1, G_2, G_3)

$\pi_2 = \{A, B\}\{F\}\{C, D, E\}$

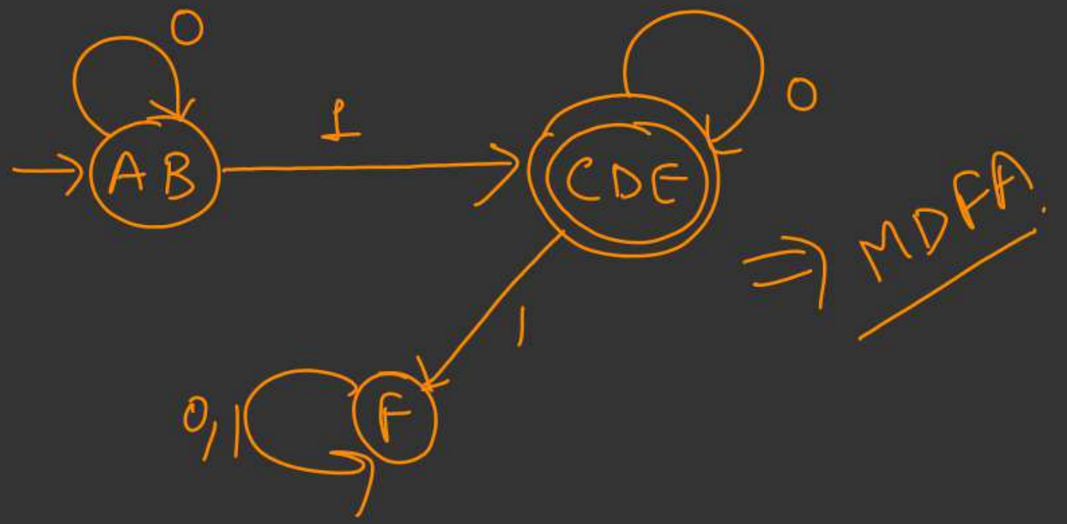
$\pi_1 = \pi_2$

Here F is a dead state, but we don't remove it as there is only one dead state.

Note



To be combined into one.



⇒ MDFA

Gateway Classes

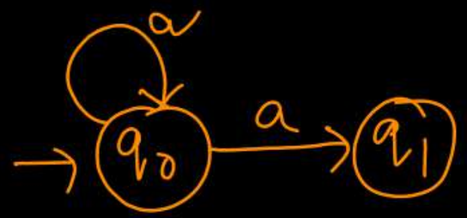
Non-deterministic Finite Automata(NDFA/NFA)

In NDFA, for a particular input symbol, the machine can move to any combination of the states in the machine (exact state to which the machine moves cannot be determined).

An NDFA can be represented by a 5-tuple $(Q, \sum, \delta, q_0, F)$ where –

- **Q** is a finite set of states.
- $\sum$ is a finite set of symbols
- **δ** is the transition function where $\delta: Q \times \sum \rightarrow 2^Q$
- $\mathbf{q_0}$ is the initial state from where any input is processed ($q_0 \in Q$).
- **F** is a set of final state/states of Q ($F \subseteq Q$).

DFA $Q \times \Sigma \rightarrow Q$

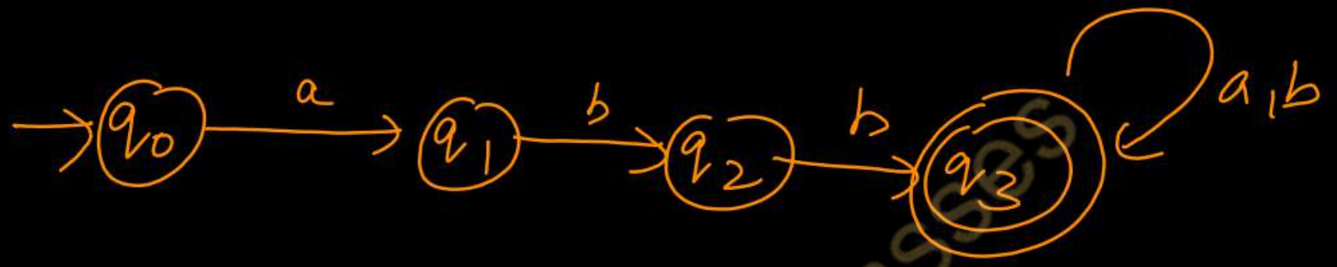


I → No transition
II → self loop
III → $q_0 \rightarrow q_1$
IV → both

Gateway Classes

Q:Construct NDFA that accepts all strings starting with 'abb'

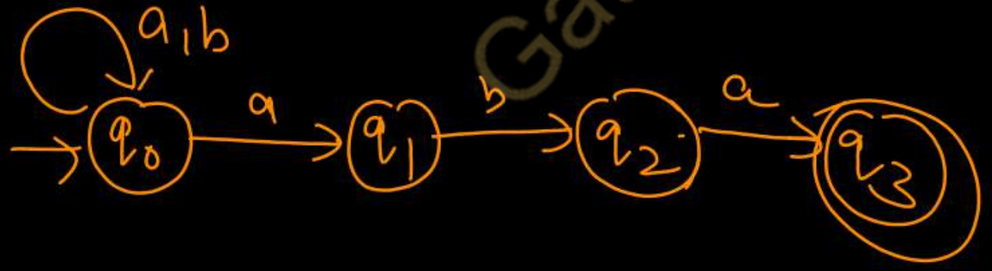
$L = \{ abb, abbba, abbaab \ldots \}$



	a	b
→ q_0	q_1	—
q_1	—	q_2
q_2	—	q_3
* q_3	q_3	q_3

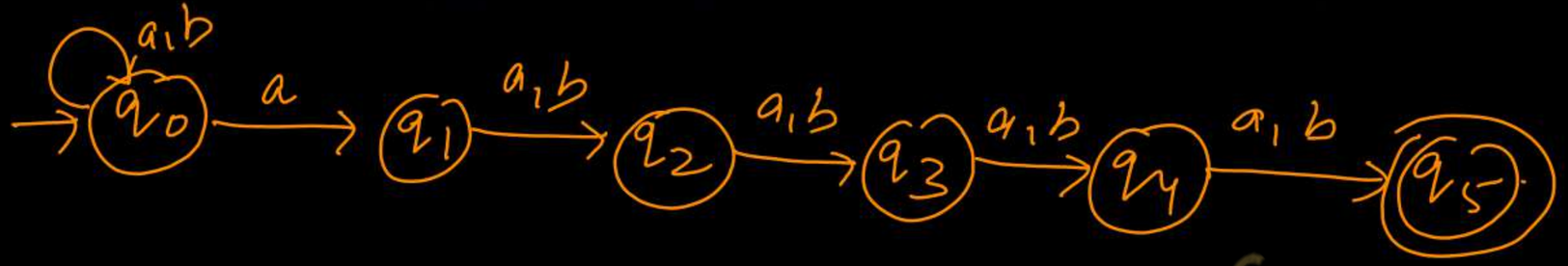
Q: Construct NDFA that accepts all strings ending with 'aba'.

$L = \{ aba, bbbaba, baba, ababa \ldots \}$

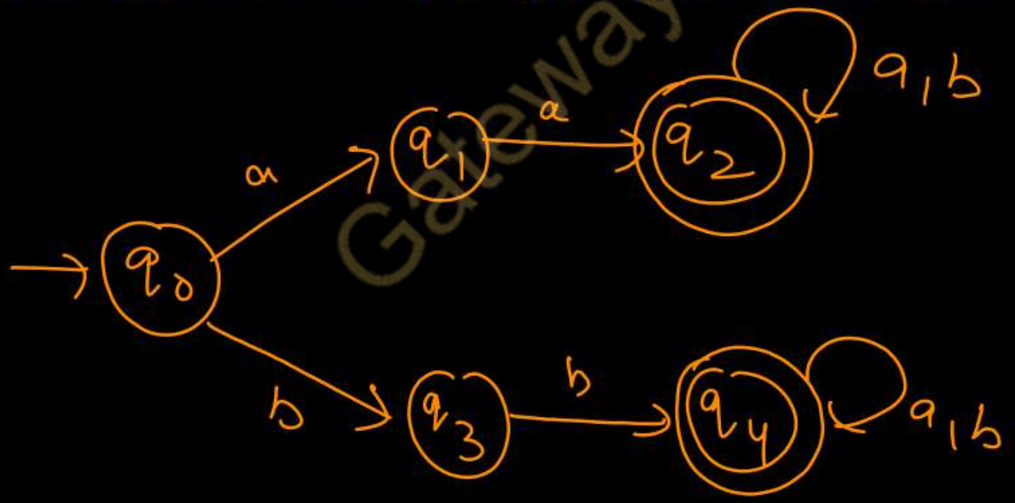


Gateway Classes

Q: Design NFA for that accepts all string over {a,b} such that fifth symbol from right is a.



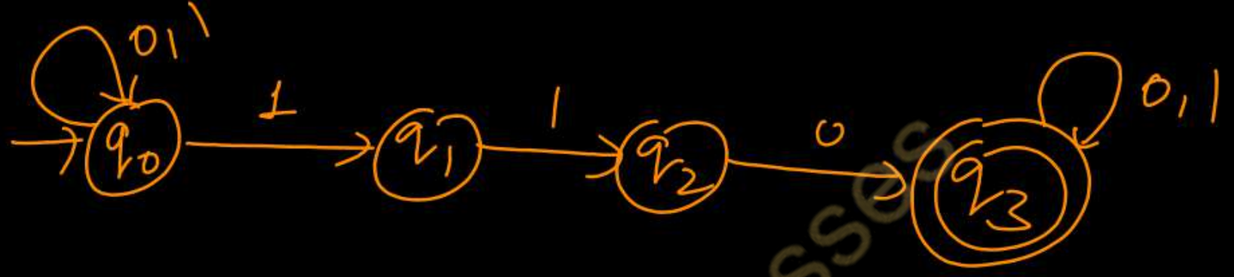
Q: Design an NFA over the alphabet Σ = {a, b} that accepts only the strings that start with "aa" or "bb"



GatewayClasses

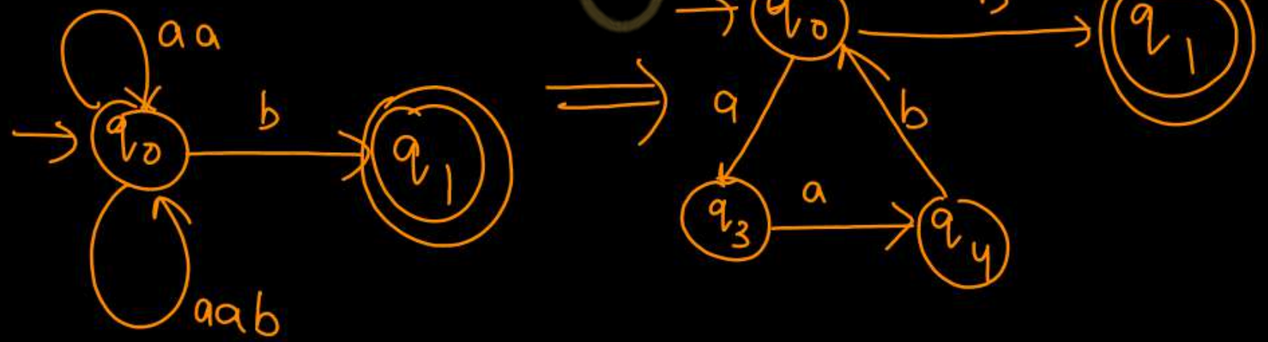
Construct an NFA with $\sum = \{0, 1\}$ in which each string must contain "double '1' is followed by ~~single '0'~~ single 0".

$L = \{110, 01100\underline{1}, 0101\underline{110}001, \text{---}\}$



Design an NFA with input alphabet $\sum = \{a, b\}$ that accepts all the strings that start with zero or more repetitions of "aa" or "aab", and end with "b".

$(aa)^*$ $(aab)^*$



a^*



$= \{\epsilon, a, aa, aaa \ldots\}$

Gateway Classes

NFA to DFA conversion



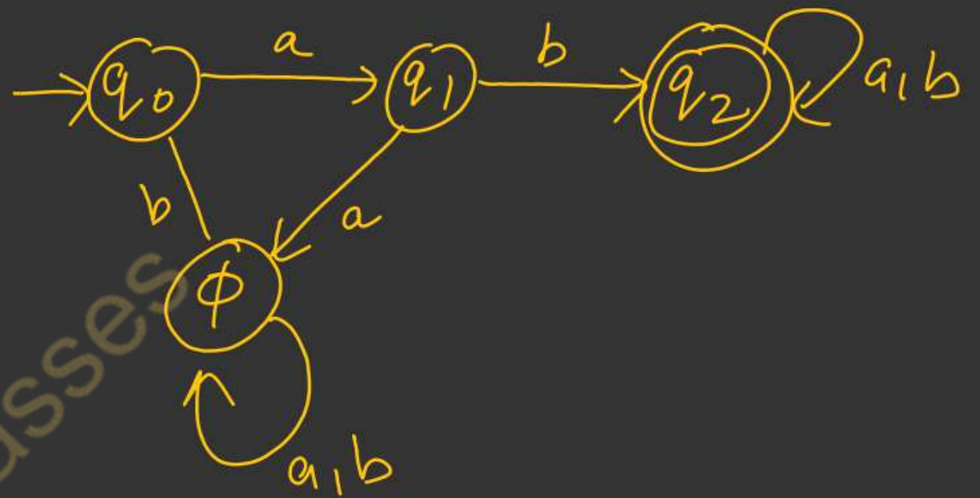
⇓

NFA that accepts all strings over $\Sigma=\{a,b\}$ starting with ab

	a	b
→ q_0	q_1	–
q_1	–	q_2
* q_2	q_2	q_2

NFA transition table.

DFA



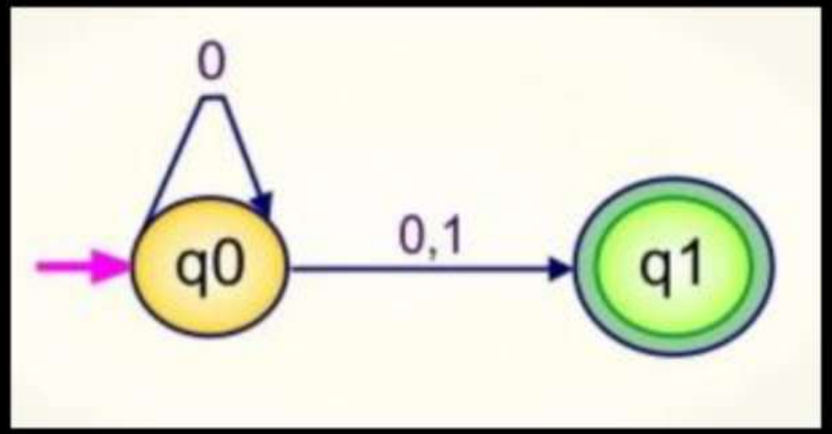
DFA Transition table

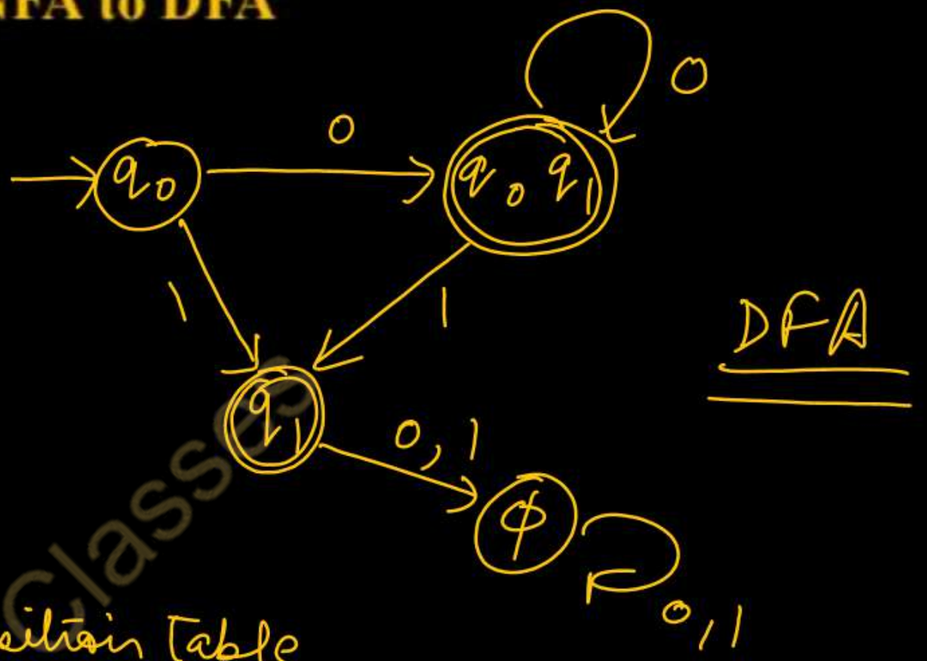
	a	b
→ q_0 ✓	q_1 ✓	ϕ
q_1	ϕ	q_2 ✓
* q_2	q_2	q_2
ϕ	ϕ	ϕ

Gateway Classes

Conversion from NFA to DFA

NFA





DFA

NFA Transition Table

	0	1
$\rightarrow q_0$	q_0, q_1	q_1
$*q_1$	—	—

$\Longrightarrow$

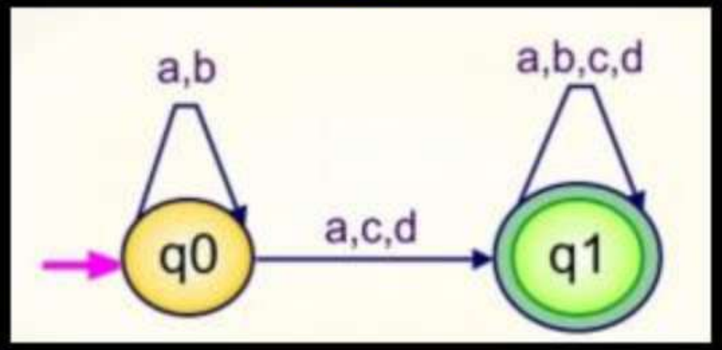
DFA Transition Table

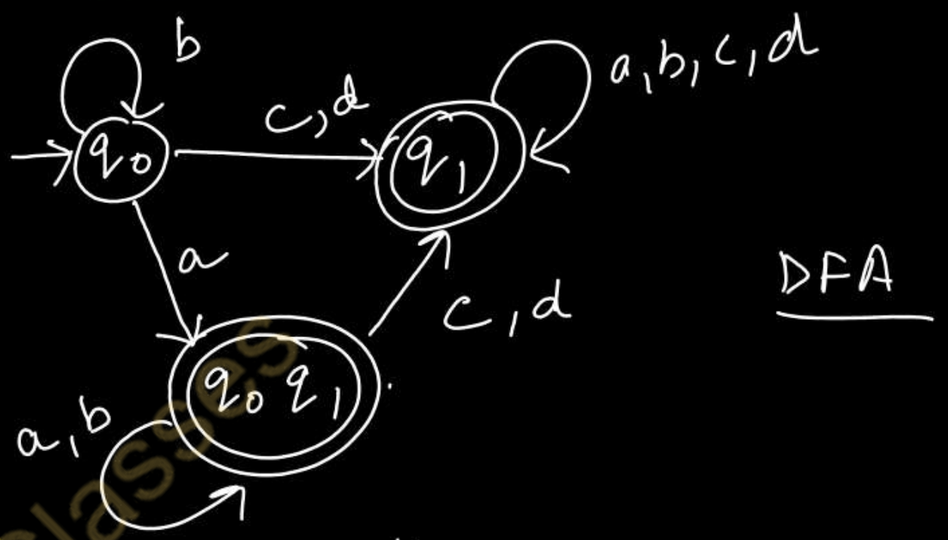
	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$ ✓	q_1 ✓
$*q_1$	ϕ	ϕ
$*\{q_0, q_1\}$	$\{q_0, q_1\}$	q_1
ϕ	ϕ	ϕ

Gateway Classes

Conversion from NFA to DFA

NFA





DFA

NFA

	a	b	c	d
$\rightarrow q_0$	$q_0 q_1$	q_0	q_1	q_1
$*q_1$	q_1	q_1	q_1	q_1

$\Longrightarrow$

DFA

	a	b	c	d
$\rightarrow q_0$	$\{q_0 q_1\}$	q_0	q_1	q_1
$*q_1$	q_1	q_1	q_1	q_1
$*\{q_0 q_1\}$	$\{q_0 q_1\}$	$\{q_0 q_1\}$	q_1	q_1

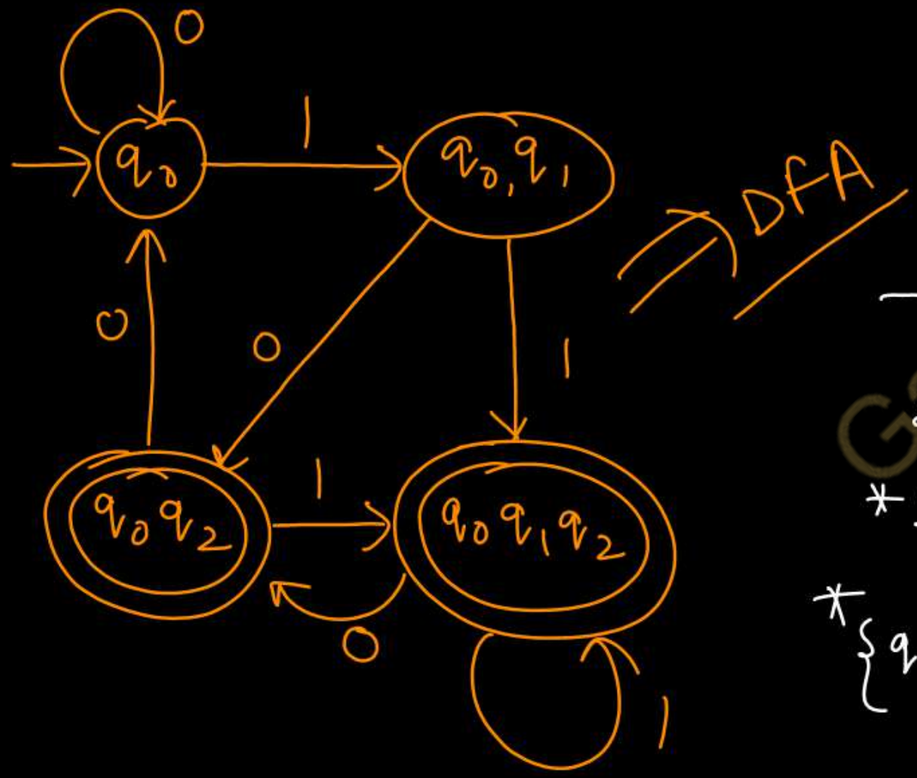
Gateway Classes

Q: Draw an NFA of all binary strings in which the 2nd last bit is 1 and convert this NFA to DFA.

$L = \{10, 11, 110, 010110, 00010 \ldots\}$



	0	1
$\rightarrow q_0$	q_0	q_0, q_1
q_1	q_2	q_2
$*q_2$	—	—



DFA

DFA.

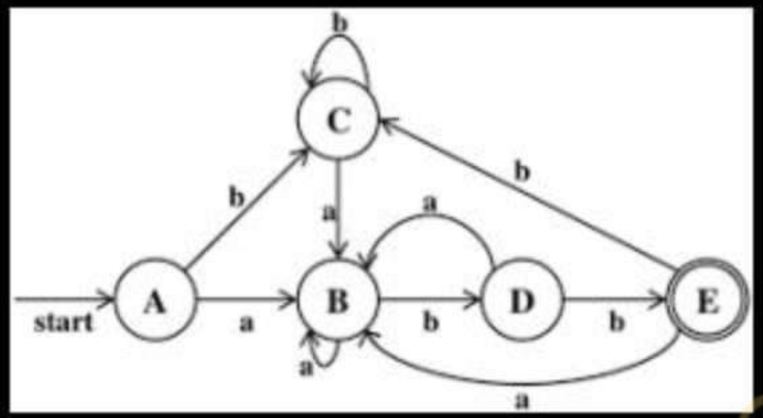
	0	1
$\rightarrow q_0$	q_0 ✓	$\{q_0, q_1\}$ ✓
$\{q_0, q_1\}$	$\{q_0, q_2\}$ ✓	$\{q_0, q_1, q_2\}$
$*\{q_0, q_2\}$	q_0 ✓	$\{q_0, q_1\}$ ✓
$*\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$ ✓	$\{q_0, q_1, q_2\}$ ✓

Gateway Classes

Compiler Design

Unit-1 : DPP-3

1. Discuss the conversion of NFA to DFA. Also discuss the algorithm. **(AKTU 2017–18)**
2. Construct minimized DFA: (AKTU 2023-24)



Gateway Classes

AKTU

B. Tech Sem-VI

CS IT & CS Allied

GATEWAY CLASSES

Compiler Design

UNIT-1 Lecture-4

Gateway Classes

Todays Target

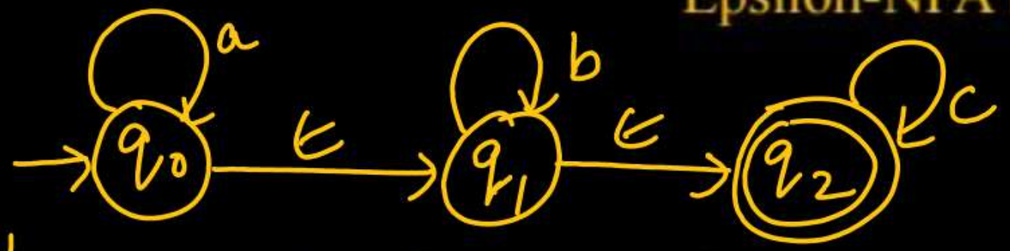
- Epsilon NFA to NFA conversion
- Regular Expression
- Writing regular expressions for languages
- AKTU PYQs



By Dr. Nidhi Parashar Ma'am

- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified

Epsilon-NFA to NFA Conversion



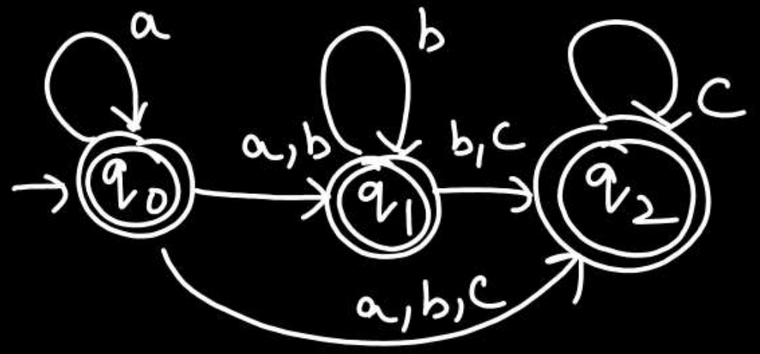
$\Sigma = \{a, b, c\}$

RE → ε-NFA → NFA → DFA

DFA → MDFA

Step - 1

	ε - closure
→ q_0	$\{q_0, q_1, q_2\}$
q_1	$\{q_1, q_2\}$
* q_2	$\{q_2\}$



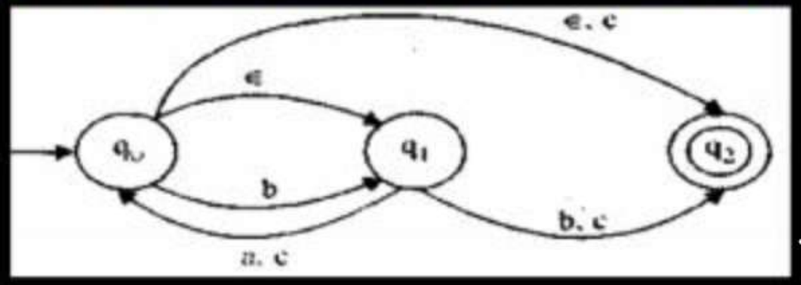
NFA

NFA Transition Table

	a	b	c
* → q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
* q_1	ϕ	$\{q_1, q_2\}$	$\{q_2\}$
* q_2	—	—	q_2

Gateway Classes

Q: Convert following NFA to equivalent DFA and minimize number of states in DFA. **(AKTU 2018-19)**

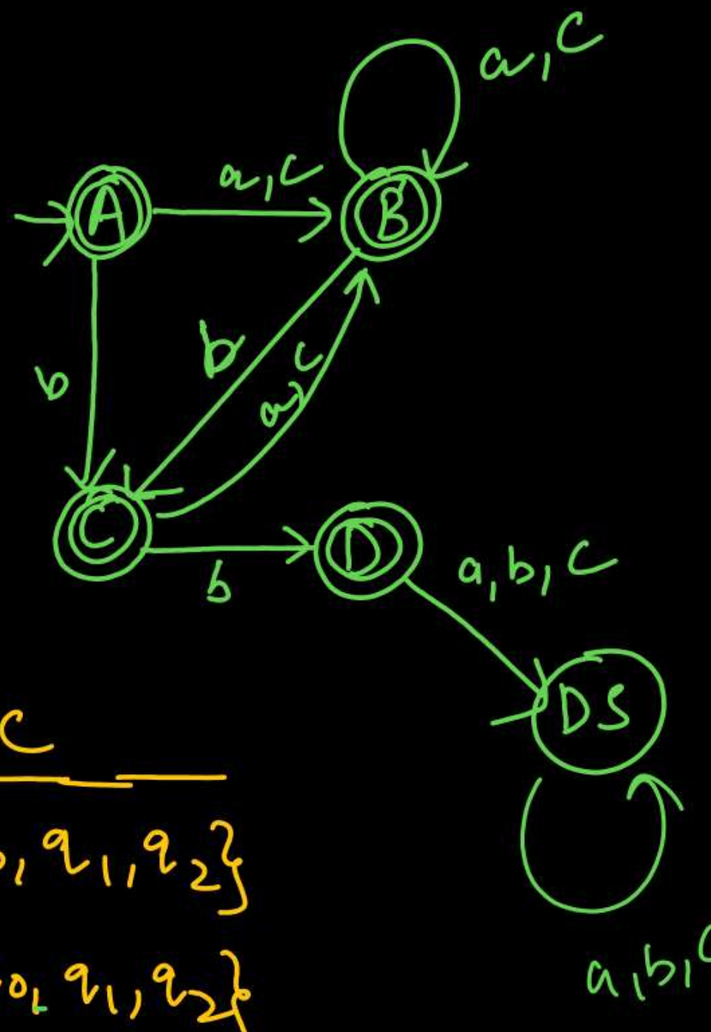


ε-closure

q_0	$\{q_0, q_1, q_2\}$
q_1	$\{q_1\}$
q_2	$\{q_2\}$

NFA Transition Table

	a	b	c
→ * q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_0, q_1, q_2\}$
q_1	$\{q_0, q_1, q_2\}$	$\{q_2\}$	$\{q_0, q_1, q_2\}$
* q_2	—	—	—

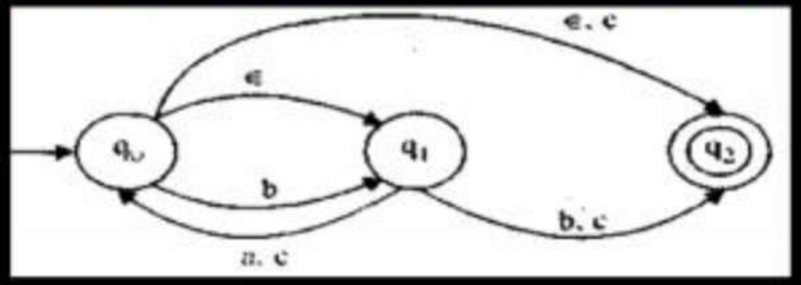


DFA

	a	b	c
A [→ * q_0	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_0, q_1, q_2\}$
* B [$\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_0, q_1, q_2\}$
* C [$\{q_1, q_2\}$	$\{q_0, q_1, q_2\}$	q_2	$\{q_0, q_1, q_2\}$
D [* q_2	ϕ	ϕ	ϕ
DS [ϕ	ϕ	ϕ	ϕ

Gateway Classes

Q: Convert following NFA to equivalent DFA and minimize number of states in DFA. **(AKTU 2018-19)**



$\pi_0 = \{A, B, C, D\}\ \{DS\}$

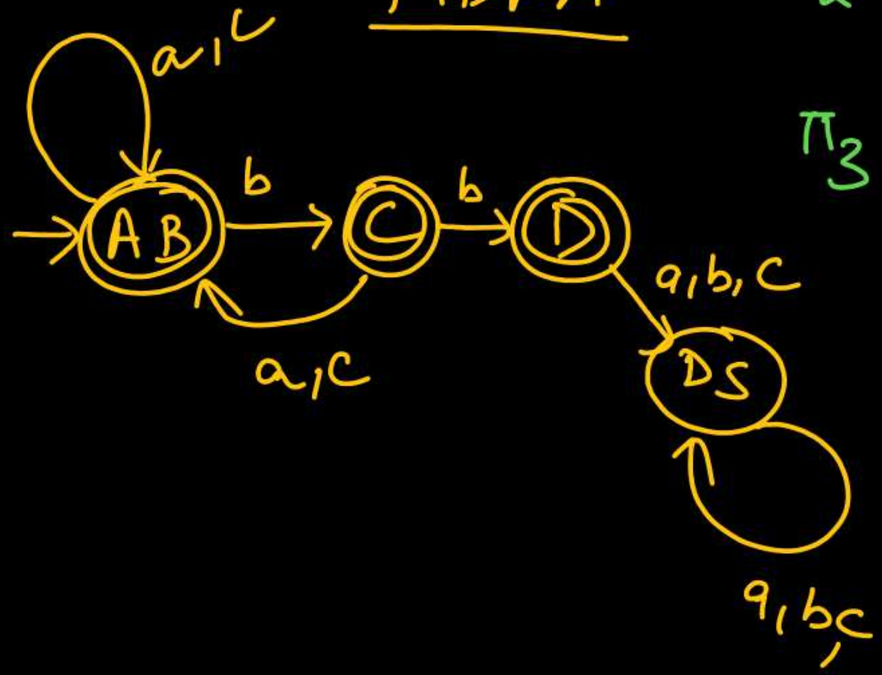
$\pi_1 = \{A, B, C\}_{G_1}\ \{D\}_{G_2}\ \{DS\}_{G_3}$

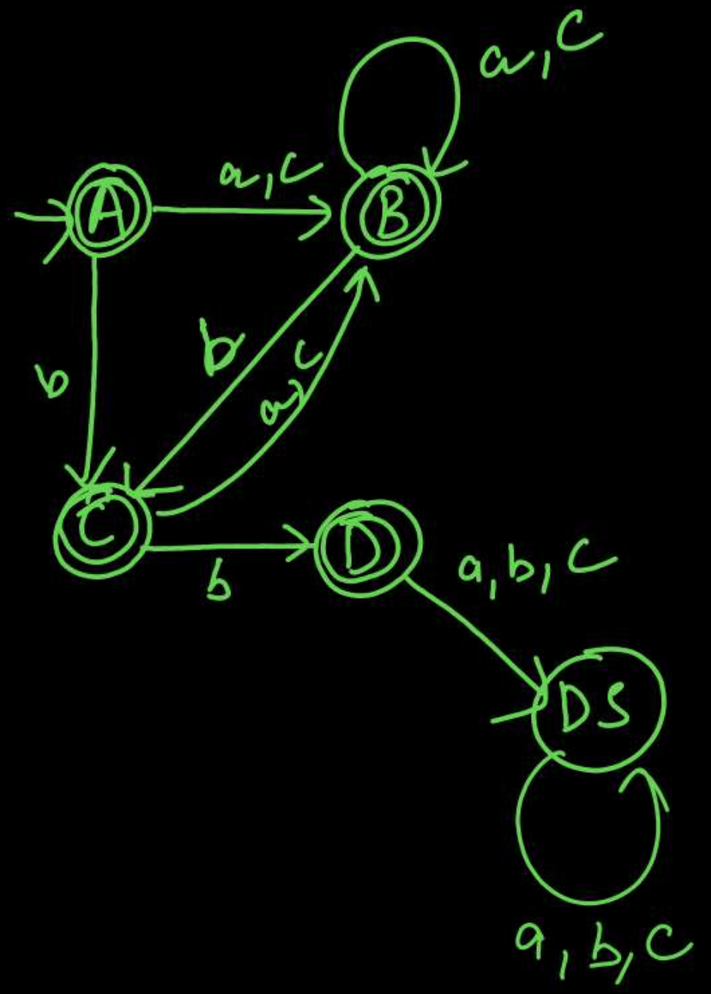
$\pi_2 = \{A, B\}_{G_1}\ \{C\}_{G_2}\ \{D\}_{G_3}\ \{DS\}_{G_4}$

$\pi_3 = \{A, B\}\ \{C\}\ \{D\}\ \{DS\}$

$\pi_2 = \pi_3$

MDFA





Gateway Classes

Regular Expression

- A regular expression over an alphabet Σ is a symbolic notation that defines a regular language using union, concatenation, and Kleene star operations.
  - A regular expression (RE) over an alphabet Σ is defined recursively as follows:
  - Base cases:
    - $\emptyset$ is a regular expression denoting the empty set.
    - ϵ(empty string) is a regular expression denoting the language {ε}.
    - For each $a \in \Sigma$, a is a regular expression denoting the language {a}.
  - Recursive cases: If R and S are regular expressions denoting languages $L(R)$and $L(S)$,then:
    - Union (alternation): $(R + S)$ denotes $L(R) \cup L(S)$
    - Concatenation: (RS) denotes $L(R)L(S)$
    - Kleene Star: (R^*) denotes $L(R^*)=(L(R))^*$, denotes zero or more repetitions of strings from $L(R)$.

Gateway Classes

Operations on Regular Expressions

- Kleene Closure (a*): $\{a^0, a^1, a^2, a^3 \ldots\} \Rightarrow \{\in, a, aa, aaa \ldots\}$
- Positive Closure (a^+): $\{a^1, a^2, a^3 \ldots\} \Rightarrow \{a, aa, aaa \ldots\}$
- Concatenation (a · b): $\{ab\}$
- Union (a + b): $\{a, b\}$ (a|b)
- (a + b)(a + b): $\{aa, ab, b, a, bb\}$ $(a+b)^2$
- (a + b)*: $\{(a+b)^0, (a+b)^1, (a+b)^2, (a+b)^3 \ldots\}$
  $\Rightarrow \{\in, a, b, aa, ab, ba, bb \ldots\}$

$(a \cdot b)^* \Rightarrow \{\in, ab, abab, ababab \ldots$

Gateway Classes

Regular Expressions and Their Languages

- $r = \varphi$ $L(r) = \{\}$ / ϕ
- $r = \varepsilon$ $L(r) = \{\in\}$
- $r = a$ $L(r) = \{a\}$
- $r = a + b$ $L(r) = \{a, b\}$
- $r = a \cdot b$ $L(r) = \{ab\}$
- $r = a + b + c$ $L(r) = \{a, b, c\}$
- $r = (a + b) \cdot b$ $L(r) = \{ab, bb\}$
- $r = a^+$ $L(r) = \{a, aa, aaa - - -\}$
- $r = a^*$ $L(r) = \{\in, a, aa, aaa - - -\}$
- $r = (a + b)(b + a)$ $L(\lambda) = \{ab, aa, bb, ba\}$

$(a+b)\cdot(a+b) = aa + ab + ba + bb$

- $r = (a + b)^2$ $L(r) = \{aa, ab, ba, bb\}$
- $r = (a + b)^*$ $\{(a+b)^0, (a+b)^1, (a+b)^2, - - -\}$

$\{\in, a, b, aa, ab, ba, bb - - -\}$

$L(r) = \{ \varepsilon, a, b, aa, ab, ba, bb, \ldots \}$

- $r = (a + b)^* \cdot (a + b)$

$L(r) = (a+b)^+$

$\Rightarrow \{a, b, aa, bb, ab, ba - - -\}$

- $r = (a \cdot b)^*$ $L(r) = \{\in, ab, abab, ababab - - -\}$

Gateway Classes

Writing regular expressions

1. Write a regular expression for strings that start with **ab**.

$$ab(a+b)^*$$

2. Write a regular expression for strings that start with **bba**.

$$bba(a+b)^*$$

3. Write a regular expression for strings that end with **abb**.

$$(a+b)^*abb$$

4. Write a regular expression for strings that contain the substring **aab**.

$$(a+b)^*aab(a+b)^*$$

5. Write a regular expression for strings that start and end with **a**.

$$a$$

$$a + a(a+b)^*a$$

Gateway Classes

Writing regular expressions

6. Write a regular expression for strings that start and end with the **same symbol**.

$$a + b + a(a+b)^*a + b(a+b)^*b$$

7. Write a regular expression for strings that start and end with **different symbols**.

$$a(a+b)^*b + b(a+b)^*a$$

8. Write a regular expression for strings of length $|w| = 3$.

$$(a+b).(a+b).(a+b) \Rightarrow (a+b)^3$$

9. Write a regular expression for strings of length $|w| \geq 3$.

$$(a+b)^3(a+b)^*$$

10. Write a regular expression for strings of length $|w| \leq 3$.

$$\epsilon + (a+b) + (a+b)^2 + (a+b)^3$$

Gateway Classes

Writing regular expressions

11. Write a regular expression for strings with **exactly two a's**

$$b^* a \; b^* a \; b^*$$

12. Write a regular expression for strings with **at least two a's**

$$(a+b)^* a (a+b)^* a (a+b)^*$$

13. Write a regular expression for strings with **at most two a's**

$$b^* + b^* a b^* + b^* a b^* a b^*$$

14. Write a regular expression for strings in which the **3rd symbol from the left is b**.

$$(a+b).(a+b)\,b\,(a+b)^*$$

15. Write a regular expression for strings in which the **10th symbol from the right is a**.

$$(a+b)^* a (a+b)^9$$

Gateway Classes

Writing regular expressions

16. Write a regular expression for strings whose length is ≡ **0 (mod 3)**.

0
3
6
9...

$[(a+b)^3]^*$ ✓

17. Write a regular expression for strings whose length is ≡ **2 (mod 3)**.

2
5
8...

$(a+b)^2[(a+b)^3]^*$

18. Write a regular expression for strings with an **even number of b's**

0, 2, 4, 8 - - -

$a^*(a^*ba^*ba^*)^*$

19. Write a regular expression for strings with the number of **a's** ≡ **1 (mod 3)**.

$b^*ab^*(b^*ab^*ab^*ab^*)^*$

20. Write a regular expression for strings with the number of **b's** ≡ **2 (mod 3)**.

$a^*b^*ba^*(a^*ba^*ba^*ba^*)^*$

Gateway Classes

Q: Construct regular expression to accept valid identifier containing letters, digits, and underscore.

(AKTU 2023-24)

[a-zA-Z_][a-zA-Z0-9_]*

union is implicit inside [ ]:

- A character class like [a-zA-Z_] already means: a | b | c | ... | Z | _

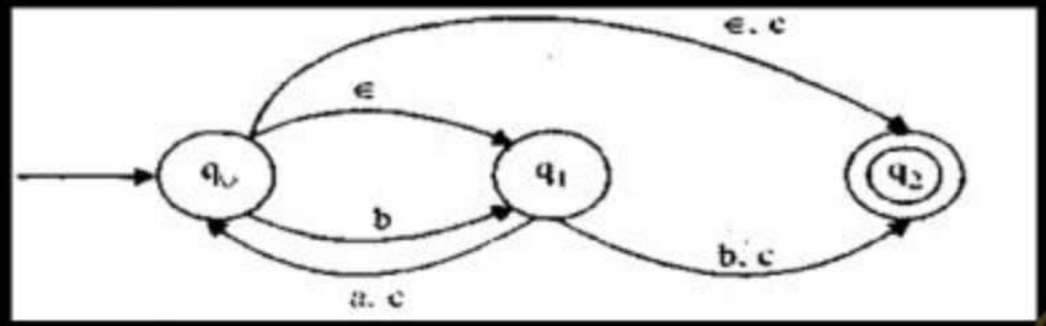
  Therefore, no explicit union symbol is required.

Gateway Classes

Compiler Design

Unit-1 : DPP-4

1. what do you mean by regular expression? (AKTU 2017-18)
2. Convert following NFA to equivalent DFA and minimize number of states in DFA. (AKTU 2018-19)



Gateway Classes

AKTU

B. Tech Sem-VI

CS IT & CS Allied

GATEWAY CLASSES

Compiler Design

UNIT-1 Lecture-5

Gateway Classes



Todays Target

- Regular Expression to NFA conversion
- Thompson's Construction
- AKTU PYQs

By Dr. Nidhi Parashar Ma'am

- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified

Q: Write down the regular expression for

1. The set of all string over {a,b} such that fifth symbol from right is a.
2. The set of all string over {0,1} such that every block of four consecutive symbol contain at least two zero.

(AKTU 2017-18)

$\Sigma = \{a, b\}$

1)

$$(a+b)^* \; a \; (a+b)\,(a+b)\,(a+b)\,(a+b)$$

$$\Rightarrow \boxed{(a+b)^* a (a+b)^4}$$

2)

$$(0 + 01 + 0011)^*$$

- No substring of length four can contain three or more 1s.
- Therefore, a single 1 must always be adjacent to a 0, which is enforced by allowing 1 only in the form 01.
- Two consecutive 1s are allowed only if they are preceded by two 0s, which is enforced by the block 0011.
- The block 0 is always safe and can appear anywhere.

Gateway Classes

Q: Write a regular expression to represent a language consisting of strings made up of odd number of a & odd number of b. **(AKTU 2020-21)**

$$(aa + bb)^*(ab+ ba)(aa + bb)^*$$

- The block (ab | ba) contributes one a and one b, making both counts odd.
- The blocks (aa | bb) add symbols in pairs, so they do not change parity (even additions).
- Repeating (aa | bb) any number of times before or after keeps the total number of a's and b's odd.
- Hence, every generated string has odd a count and odd b count.

Gateway Classes

Converting regular expression into an NFA

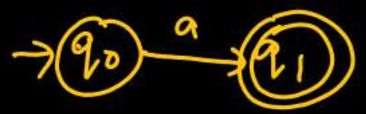
Φ



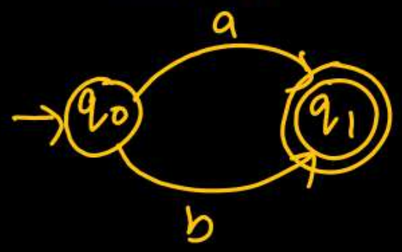
ε



a



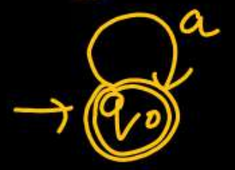
a + b



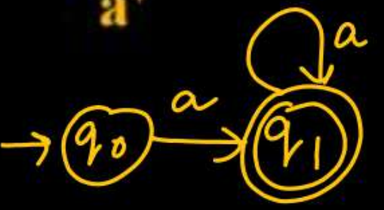
a · b



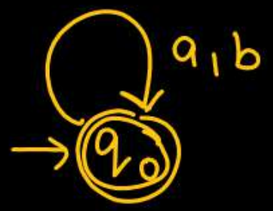
a*



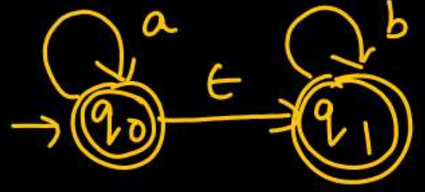
a⁺



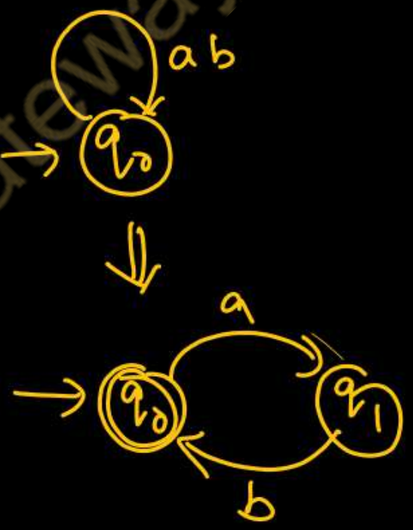
(a + b)*



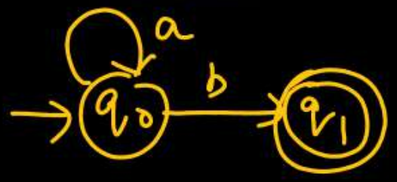
a* b*



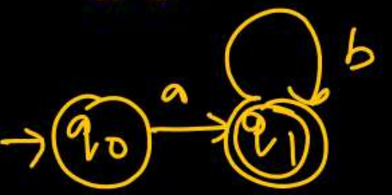
(ab)*



a* b



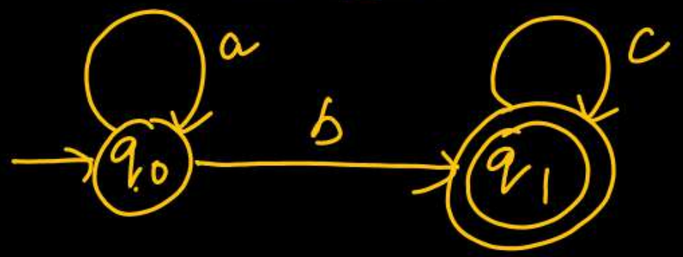
a b*



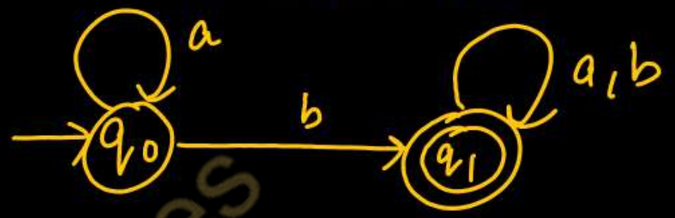
Gateway Classes

Converting regular expression into an NFA

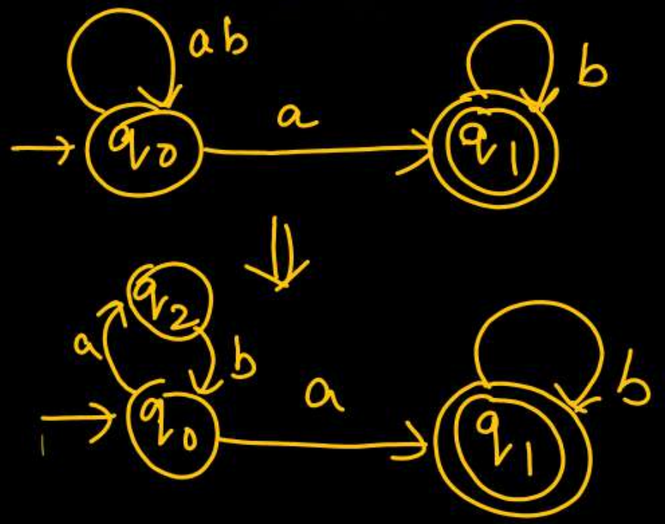
$a^* \underline{b}\, c^*$



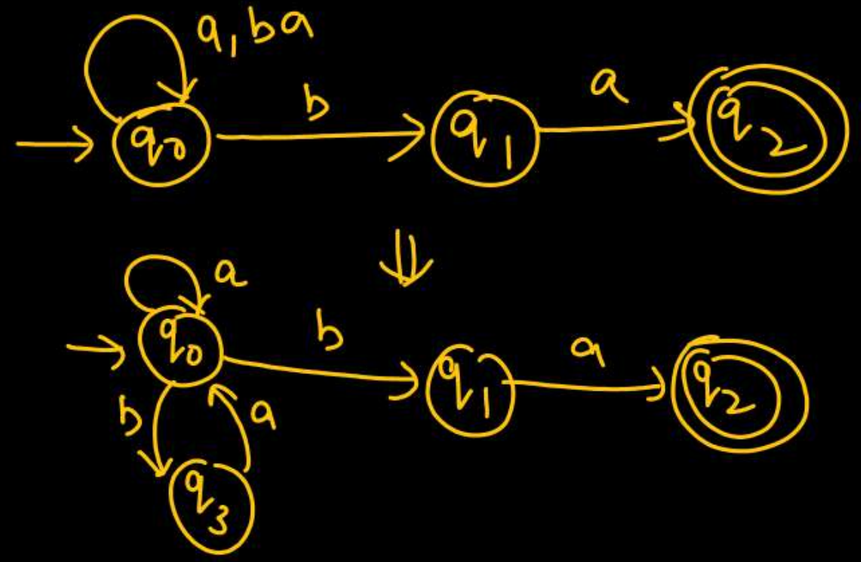
$a^*\, b\, (a + b)^*$



$(ab)^*\, a\, b^*$



$(a + ba)^*\, ba$

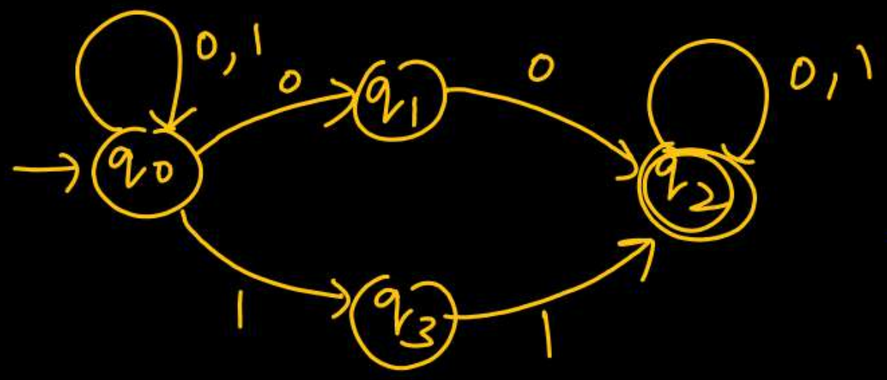


Gateway Classes

Q: Construct the NFA and DFA for the regular expression. **(AKTU 2022-23)**

$(0 + 1)^* (00 + 11) (0 + 1)^*$

NFA

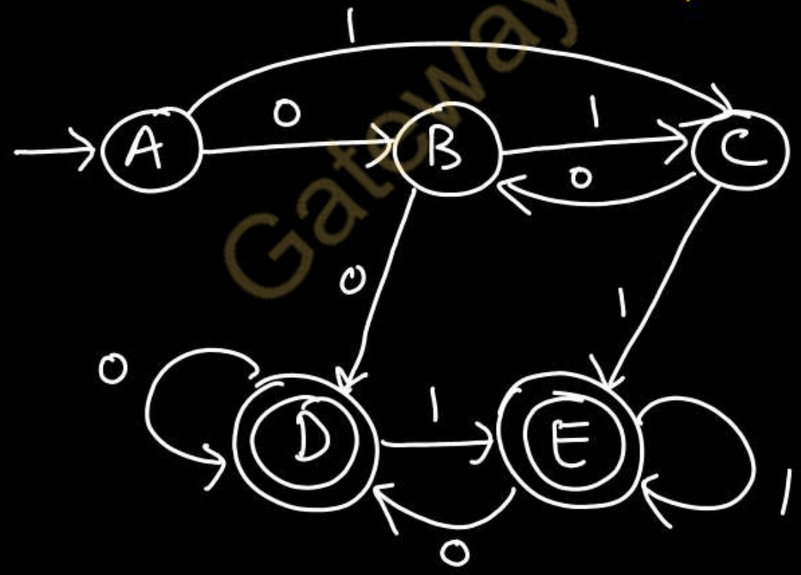


NFA

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0, q_3\}$
q_1	q_2	—
$*\ q_2$	q_2	q_2
q_3	—	q_2

DFA

		0	1
A	$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0, q_3\}$
B	$\{q_0, q_1\}$	$\{q_0 q_1 q_2\}$	$\{q_0 q_3\}$
C	$\{q_0, q_3\}$	$\{q_0, q_1\}$	$\{q_0, q_3, q_2\}$
* D	$\{q_0 q_1 q_2\}$	$\{q_0 q_1 q_2\}$	$\{q_0 q_3 q_2\}$
* E	$\{q_0 q_2, q_3\}$	$\{q_0 q_1 q_2\}$	$\{q_0 q_3 q_2\}$



Gateway Classes

Q: Construct the NFA and DFA for the regular expression. **(AKTU 2022-23)**

$$(0 + 1)^* (00 + 11) (0 + 1)^*$$

DFA Transitions

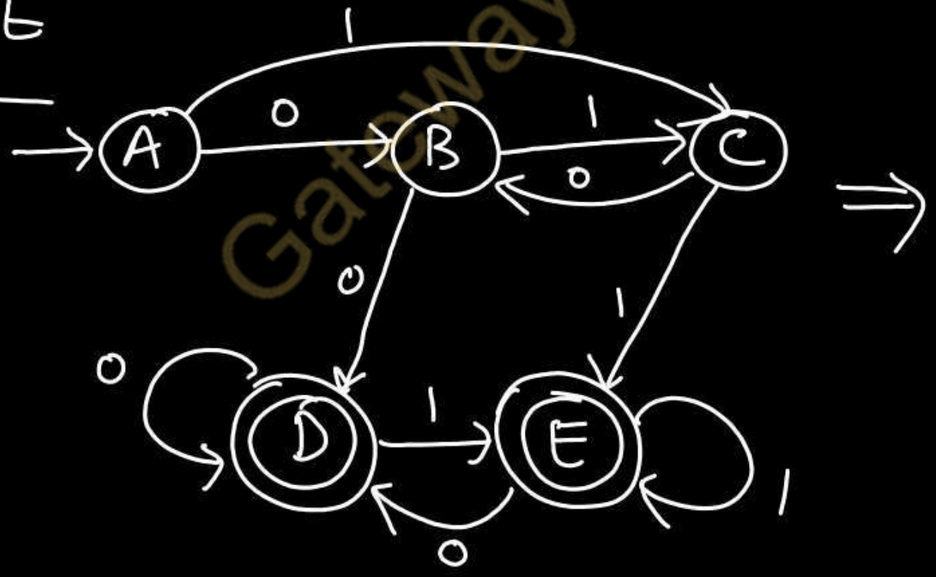
	0	1
→ A	B	C
B	D	C
C	B	E
* D	D	E
* E	D	E

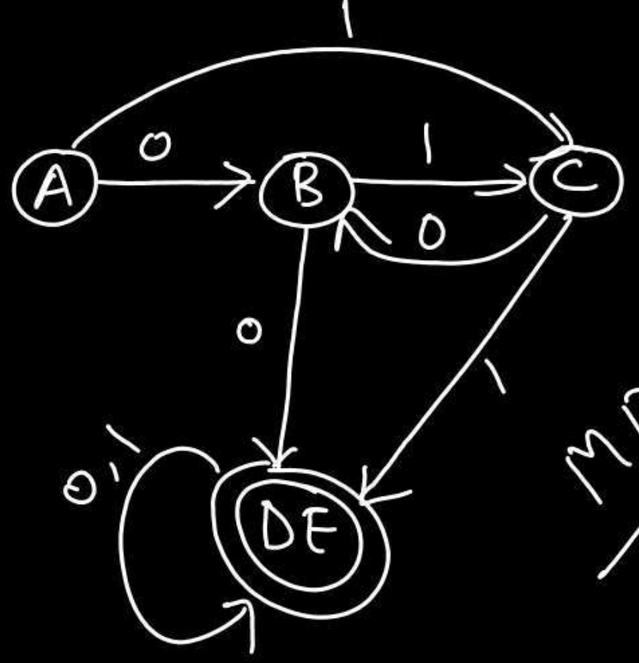
$\pi_0 = \{A, B, C\} \{D, E\}$

$\pi_1 = \{A\}\{B\}\{C\} \{D, E\}$

$\pi_2 = \{A\}\{B\}\{C\}\{D, E\}$

$\pi_1 = \pi_2$

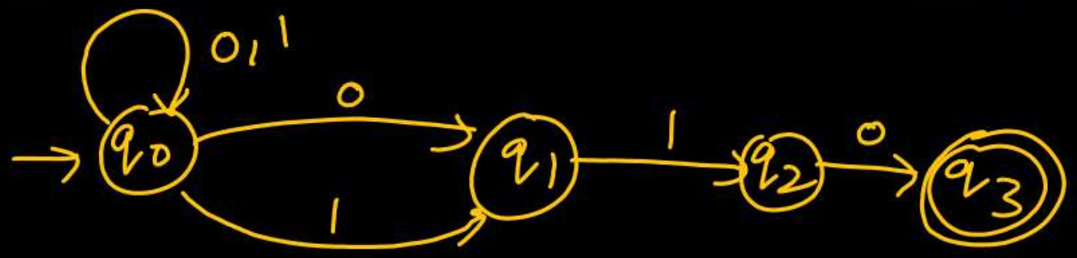


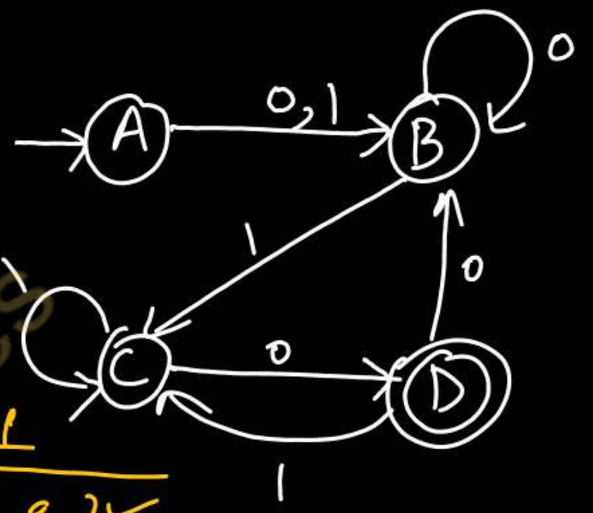


MDFA

Gateway Classes

Q: Construct the minimized DFA for the regular expression (0+1)*(0+1) 10. **(AKTU 2016-17)**





DFA

NFA

	0	1
→ q_0	$\{q_0, q_1\}$	$\{q_0, q_1\}$
q_1	—	q_2
q_2	q_3	—
* q_3	—	—

DFA

	0	1
A → q_0	$\{q_0, q_1\}$ ✓	$\{q_0, q_1\}$ ✓
B ⟹ $\{q_0, q_1\}$	$\{q_0, q_1\}$ ✓	$\{q_0, q_1, q_2\}$
C $\{q_0, q_1, q_2\}$	$\{q_0, q_1, q_3\}$ ✓	$\{q_0, q_1, q_2\}$
* D $\{q_0, q_1, q_3\}$	$\{q_0, q_1\}$	$\{q_0, q_1, q_2\}$

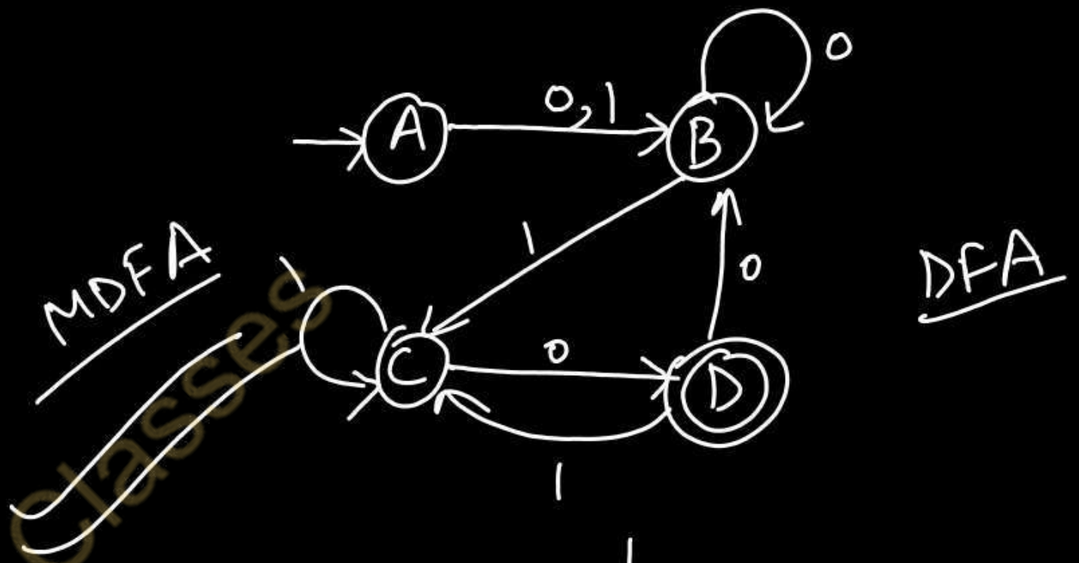
Gajendra Classes

Q: Construct the minimized DFA for the regular expression (0+1)*(0+1) 10. **(AKTU 2016-17)**

$\pi_0 = \{A, B, C\}\{D\}$

$\pi_1 = \{A, B\}\{C\}\{D\}$

$\pi_2 = \{A\}\{B\}\{C\}\{D\}$



	0	1
→ A	B	B
B	B	C
C	D	C
*D	B	C

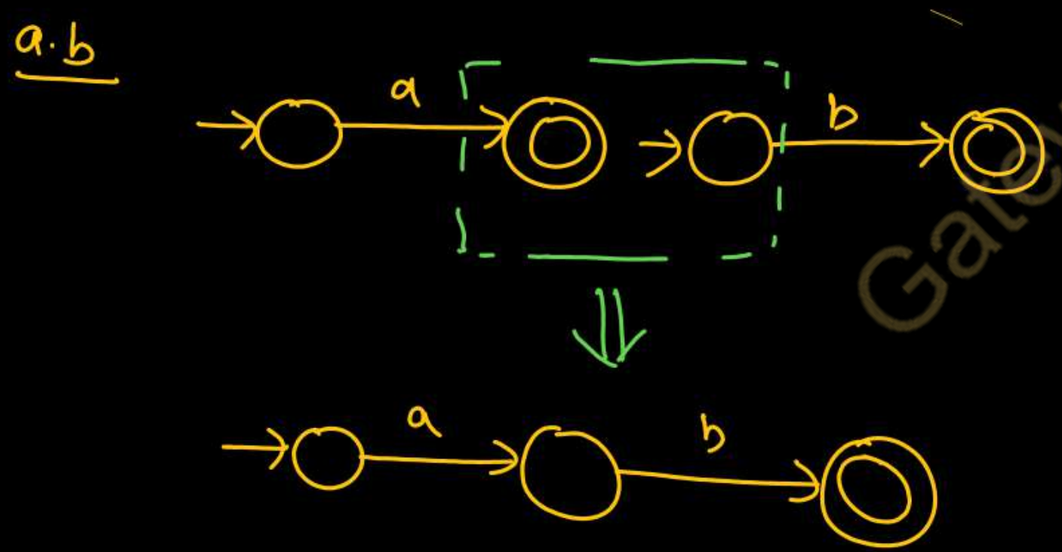
Gateway Classes

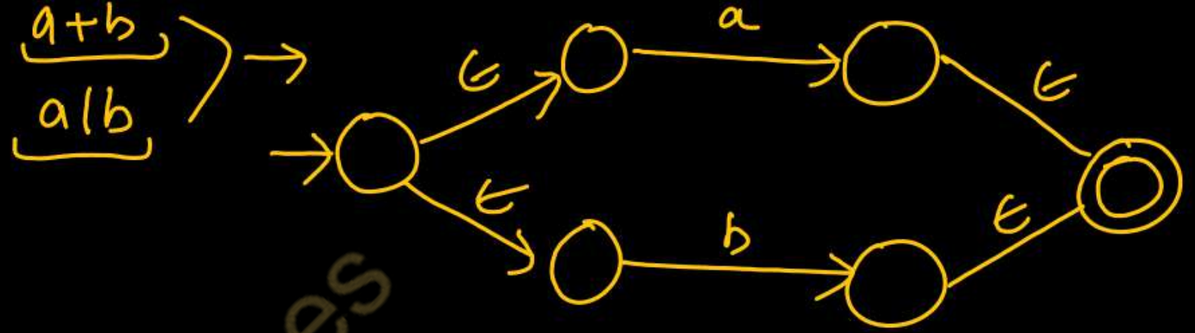
Converting regular expression into an NFA using Thompson's Construction

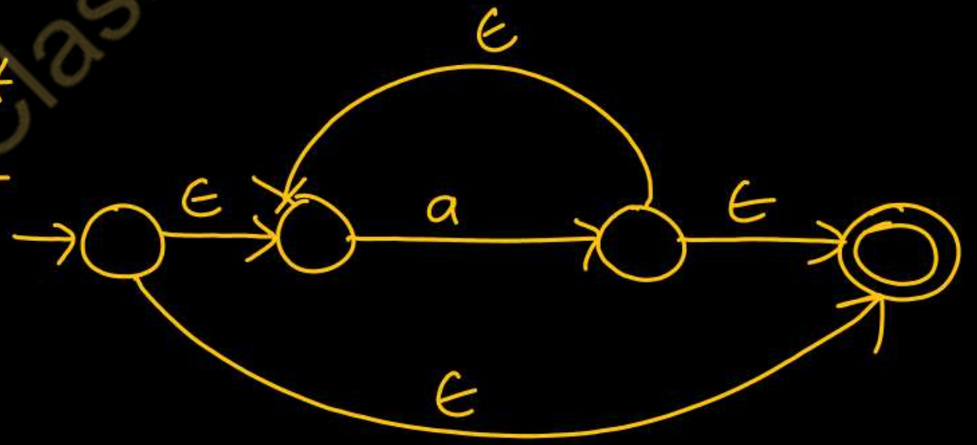








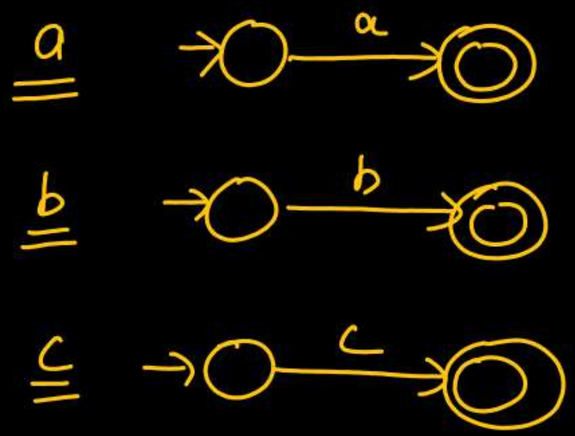


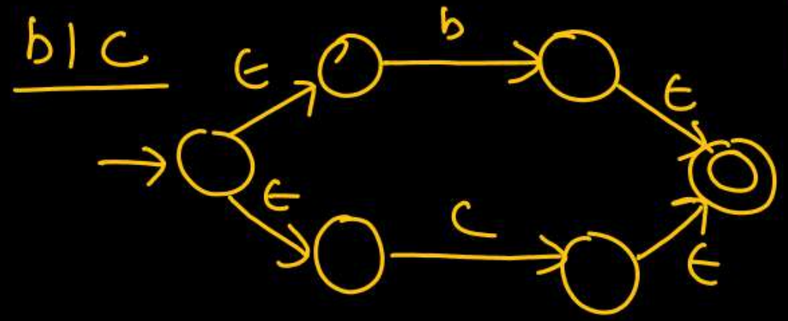


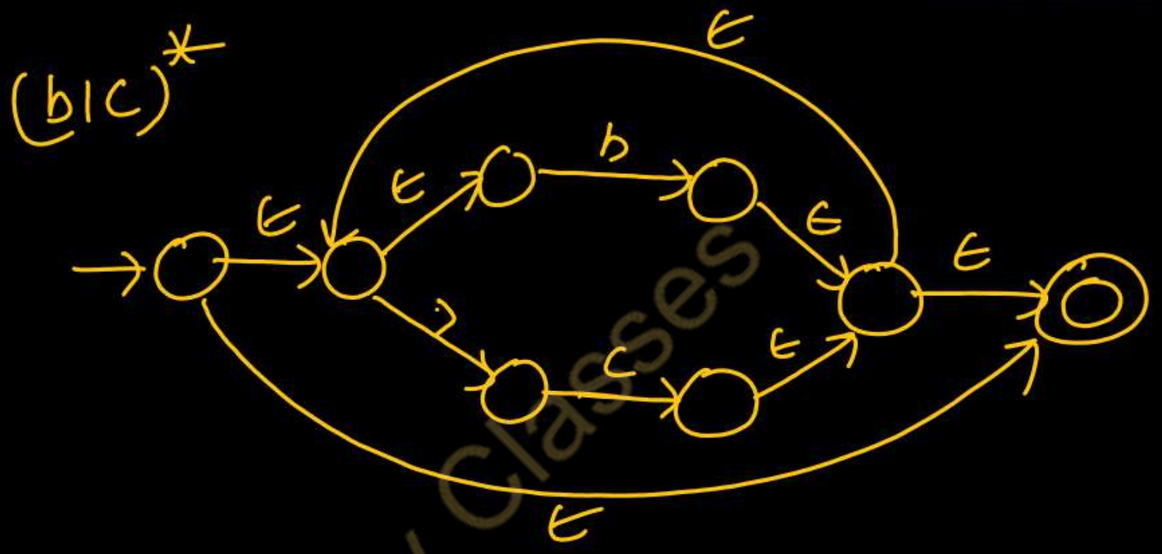
Gateway Classes

Q: Given a regular expression a(b|c)*, construct the corresponding Nondeterministic Finite Automaton (NFA) that recognizes the language described by the regular expression. **AKTU 2023-24)**

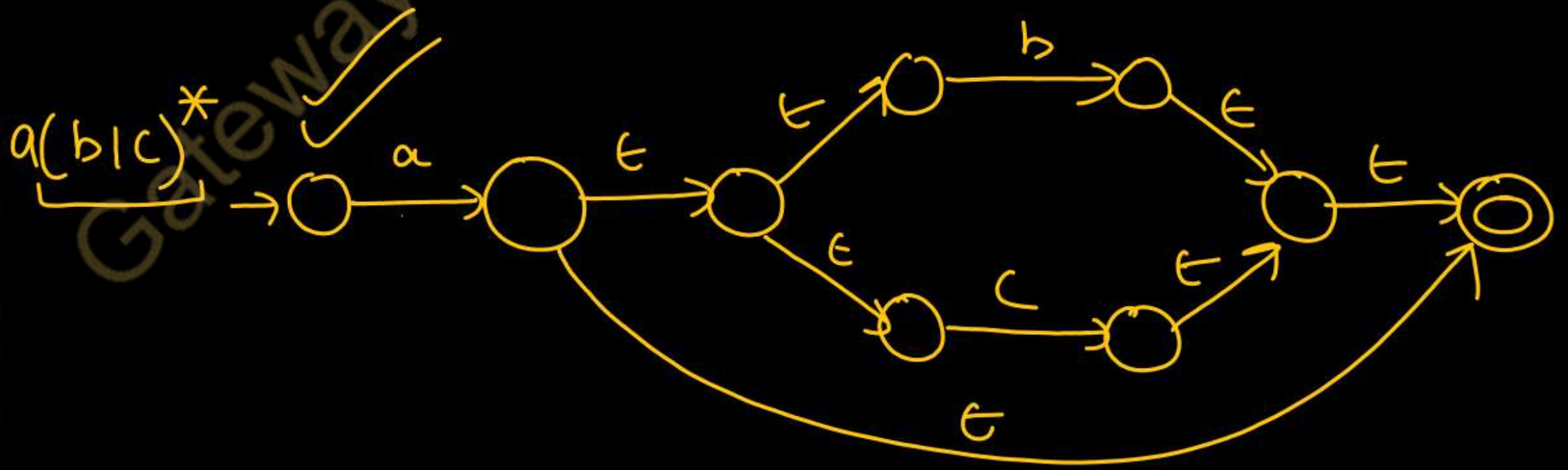
$a(b|c)^*$







ε-NFA → NFA



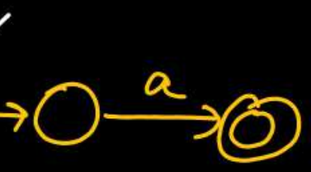
Gateway Classes

Q: Construct the NFA for the regular expression a| abb| a*b+ by using Thompson's construction.

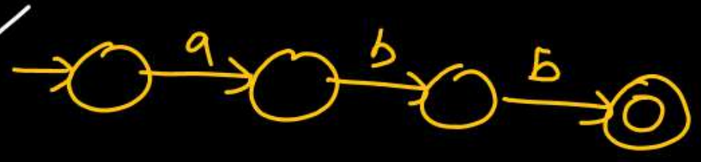
(AKTU 2017-18)

a | a b b | a* b+

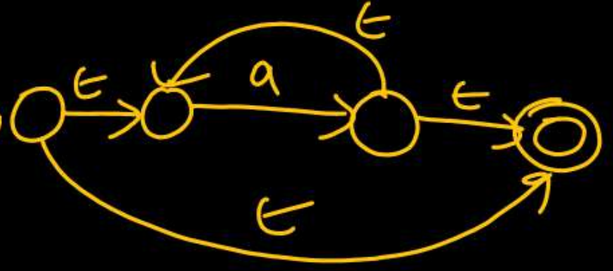
a



abb

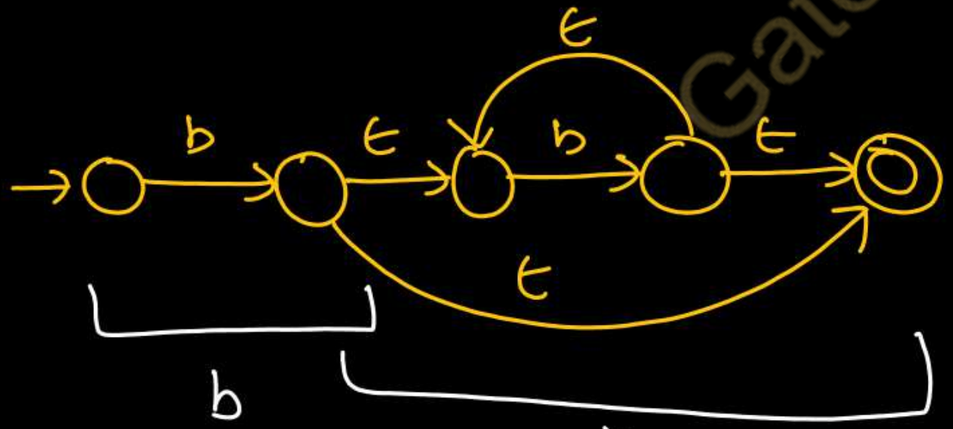


a*

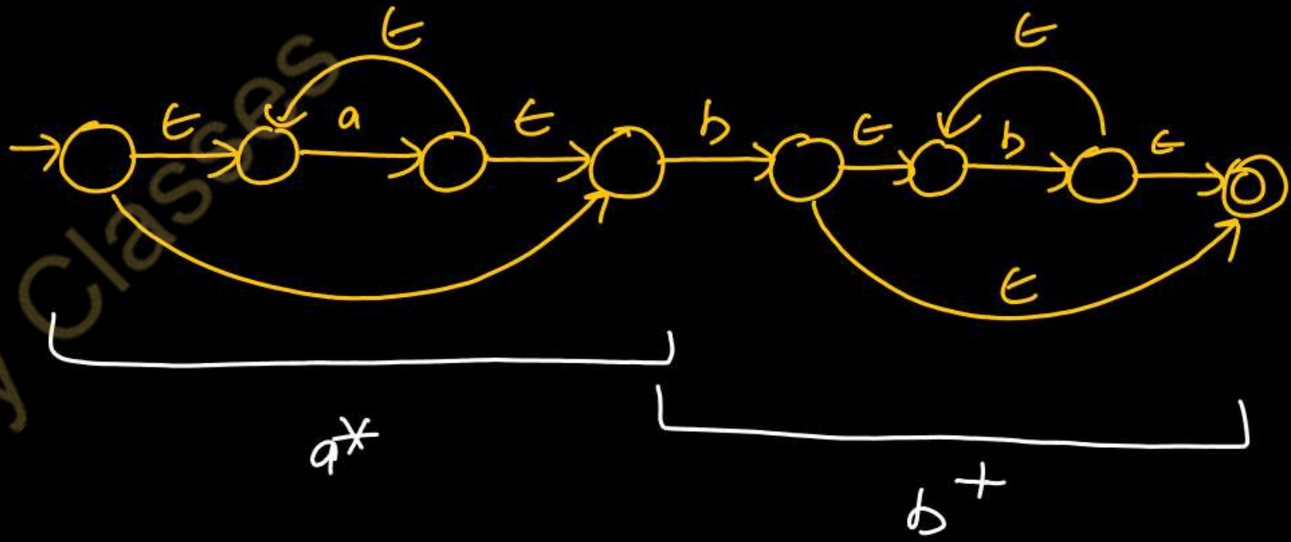


b+

b.b*



a* b+ ⇒

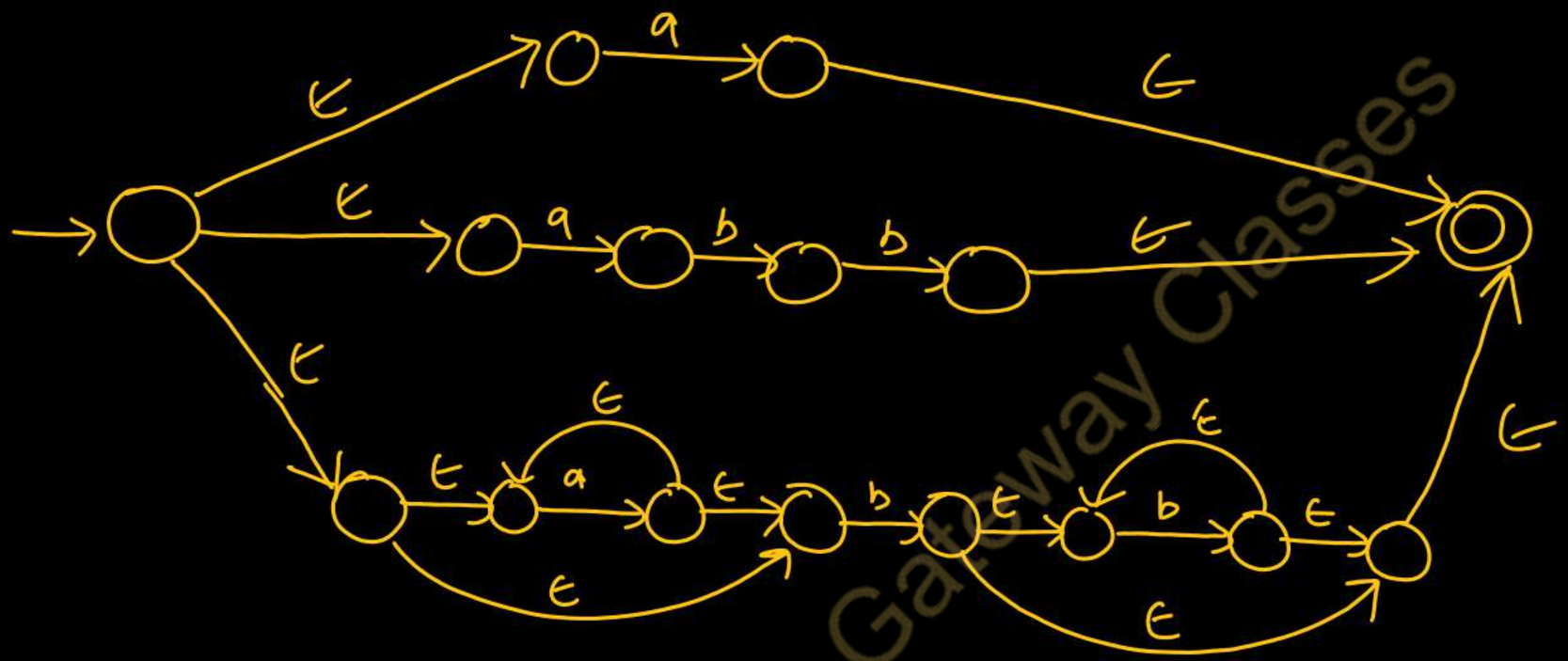


Gateway Classes

Q: Construct the NFA for the regular expression a| abb| a*b+ by using Thompson's construction.

(AKTU 2017-18)

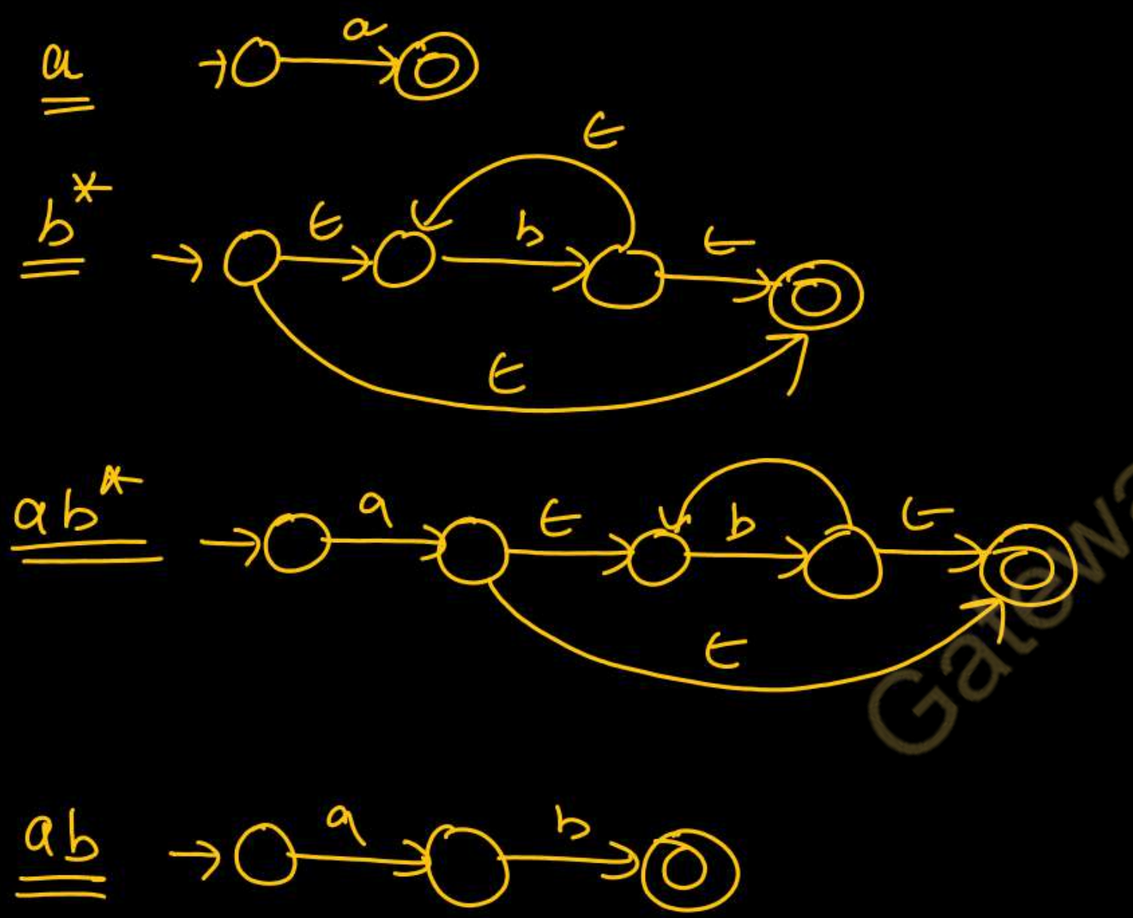
a | a b b | a* b+

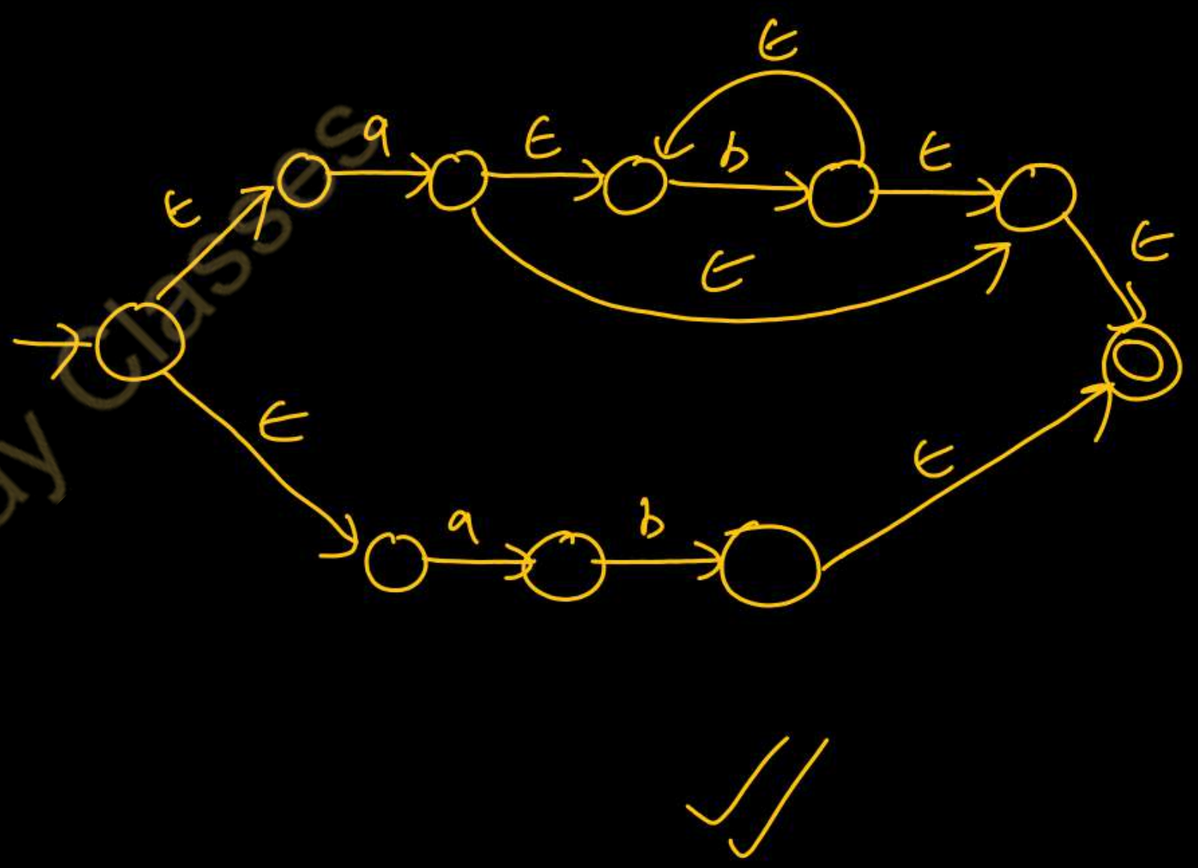


Gateway Classes

Q: Construct the NFA for the regular expression ab*|ab. **(AKTU 2016-17)**

$ab^* \mid ab$





Gateway Classes

Compiler Design

Unit-1 : DPP-5

1. Construct NFA for the regular expression a/abb/a*b+ by using Thompson's construction methodology. (AKTU 2017-18)
2. Construct the minimized DFA for the regular expression (0+1)*(0+1) 10. (AKTU 2016-17)
3. Construct the NFA and DFA for the regular expression. (0 + 1)* (00 + 11) (0 + 1)* **(AKTU 2022–23)**
4. Write a regular expression to represent a language consisting of strings made up of odd number of a & odd number of b. (AKTU 2020-21)
5. Write down the regular expression for (AKTU 2017-18)
   1. The set of all string over {a,b} such that fifth symbol from right is a.
   2. The set of all string over {a,b} such that every block of four consecutive symbol contain at least two zero.
6. Draw NFA for the regular expression ab*/ab. (AKTU 2016-17)

Gateway Classes

AKTU

B. Tech Sem-VI

CS IT & CS Allied

GATEWAY CLASSES

Compiler Design

UNIT-1 Lecture-6

Gateway Classes



Todays Target

- Lexical Analyzer Generator
- Relationship Between Finite Automata and Lex
- Formal Grammar's application to Syntax Analysis
- → YACC
- AKTU PYQs

By Dr. Nidhi Parashar Ma'am

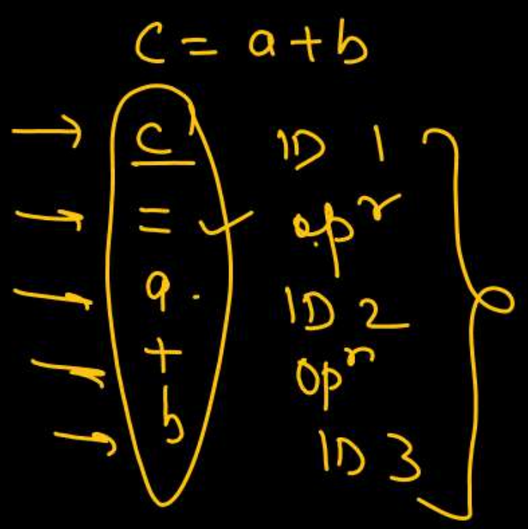
- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified

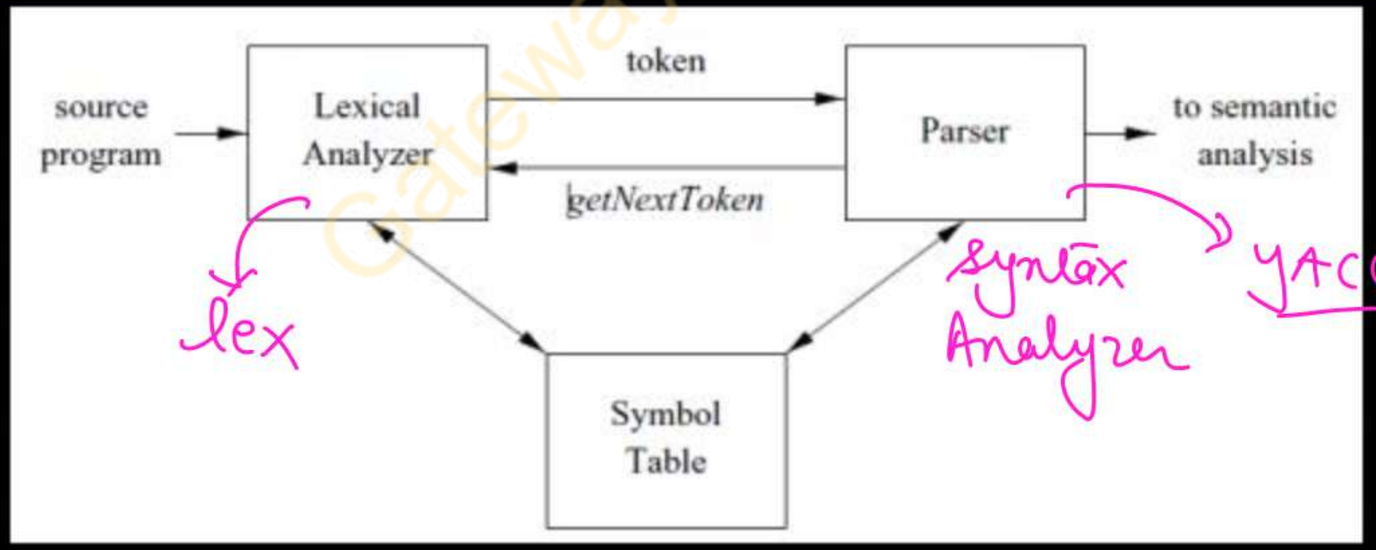
Role of the Lexical Analyzer

- Lexical analyzer read input characters of source program, group them into lexemes, and produce as output a sequence of tokens for each lexeme.
- The stream of tokens is sent to the parser for syntax analysis.
- When lexical analyzer discovers a lexeme constituting an identifier, it enters that lexeme into symbol table.

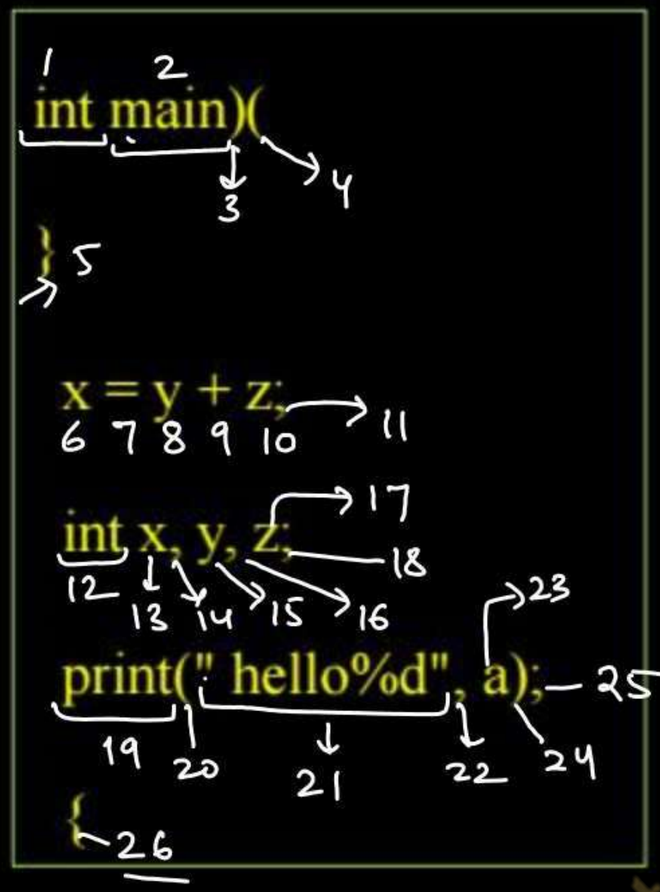
Sometimes, lexical analyzers are divided into two processes:

1. Scanning: deletion of comments and compaction of consecutive whitespace characters into one.
2. Lexical analysis: produces tokens from the output of the scanner.





Gateway Classes



Total no. of Tokens = 26

Gateway Classes

Lexical Analyzer Generator:

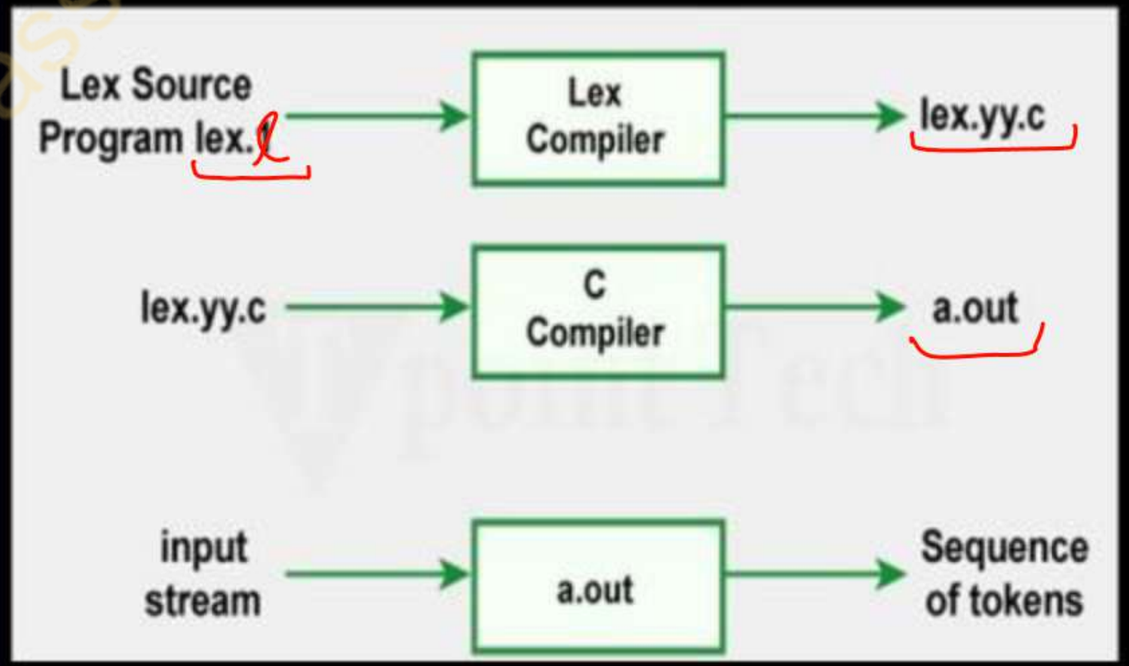
Lexical analyzer tokenize the input string and hence it is source language dependent. The lexical analyzer needs to define patterns for all programming constructs of the input language. A tool can handle these variations in source language to design the lexical analysis phase.

What is Lex ?

- Lex is a tool or a computer program that generates Lexical Analyzers.
- Lex tool takes a LEX source program as its input and produces lexical Analyzer as its output.
- Lexical Analyzer will convert the input string entered by the user into tokens as its output.

Function of Lex

1. Source code which is in Lex language having the file name 'File.l' given as input to Lex Compiler (Lex) to get the output as as C program lex.yy.c.
2. lex.yy.c is used as input to C compiler which gives the output in the form of an 'a.out' file.
3. a.out is lexical analyzer that transforms an input stream into a sequence of tokens.



Gateway Classes

The Structure of LEX

Definition section:

- This section is written **between %{ and %}**, followed by macro definitions.
- Used to define macros (regular expression shortcuts), Include header files, Declare variables.
- Any C code written here, is copied as it is into the generated source file.

%{
==
%}

- Rules section:
  - Contains **patterns (regular expressions)** and **actions (C code).**
  - When a pattern matches, its action is executed.
  - When lexical analyzer sees some text in the input matching a given pattern, it executes associated C code.

- User Subroutine section: Here, all the required procedures are defined. It contains main in which C statements and functions that are copied as it is to the generated source file. These statements contain code called by the rules in the rules section.

Gateway Classes

lex program to identify number identifier and words

Structure of lex

```
%{
    Definition section
%}

%%
    Rules section
%%

User Subroutine section
```

yytext contains the exact text (lexeme) that was matched by the current rule

123

```
%{
#include <stdio.h>
%}

DIGIT    [0-9]
LETTER   [a-zA-Z]
ID       {LETTER}({LETTER}|{DIGIT})*

%%
{DIGIT}+      printf("Number: %s\n", yytext);
{ID}          printf("Identifier: %s\n", yytext);
{LETTER}+     printf("Word: %s\n", yytext);
[ \t\n]+          ;    /* ignore spaces */
.                 ;    /* ignore other characters */
%%
int main()
{   yylex();
    return 0;
}
```

- yylex() is automatically generated by Lex
- Read the input character by character, Match the input against the rules
- When a rule matches, execute the associated action.
- Repeat until end of input

Gateway Classes

Rules Section

- %% Marks the start of rules section. %% Marks the end of rules section.
- {DIGIT}+ printf("Number: %s\n", yytext);
- {ID} printf("Identifier: %s\n", yytext); Matches a valid identifier and Prints "Identifier" and its value.
- {LETTER}+ printf("Word: %s\n", yytext); Matches one or more letters, Prints "Word" and its value
- [\t\n]+ ; Matches spaces, tabs, and newlines, ; means do nothing (ignore them)
- . ; If you encounter any single character that was not matched by previous rules, just skip it.
- User Subroutines Section: yylex(); Calls lexical analyzer generated by Lex which Starts scanning input and matching rules.

Gateway Classes

Relationship Between Finite Automata and Lex

A **Finite Automaton** is a mathematical model of computation used to **recognize patterns** (regular languages).

Finite automata are recognizers; they simply say “yes" or “no" about each possible input string.

Lex works using following pipeline:

Lex Program (lex source.l)
↓
Regular Expressions
↓
NFA construction ← Thompson’s construction
↓
NFA → DFA conversion ← Subset construction
↓
DFA minimization
↓
C code generation (lex.yy.c) (C code with DFA tables)
↓ gcc lex.yy.c -ll
a.out (Executable lexical analyzer)

Gateway Classes

Runtime Behavior of a.out: yylex() reads input character by character, moves through DFA states, Identifies tokens And executes user-defined actions.

Relationship between FSM and Regular Expressions (AKTU 2023-24)

- Finite State Machines (FSMs) and REs are equivalent formalisms used to describe regular languages.
- For every RE, there exists an equivalent FA that accepts the same language.
- Similarly, for every FA, an equivalent RE can be constructed.
- A language is regular iff it can be represented by either a RE or a finite state machine.

Use of Regular Expressions in Lexical Analysis

- During lexical analysis, source program is divided into tokens such as identifiers, keywords, operators, and constants. REs are used to specify token patterns. These expressions are converted into NFAs and then into DFAs, which are used by the lexical analyzer to recognize tokens efficiently. Tools like Lex use REs to automatically generate lexical analyzers.

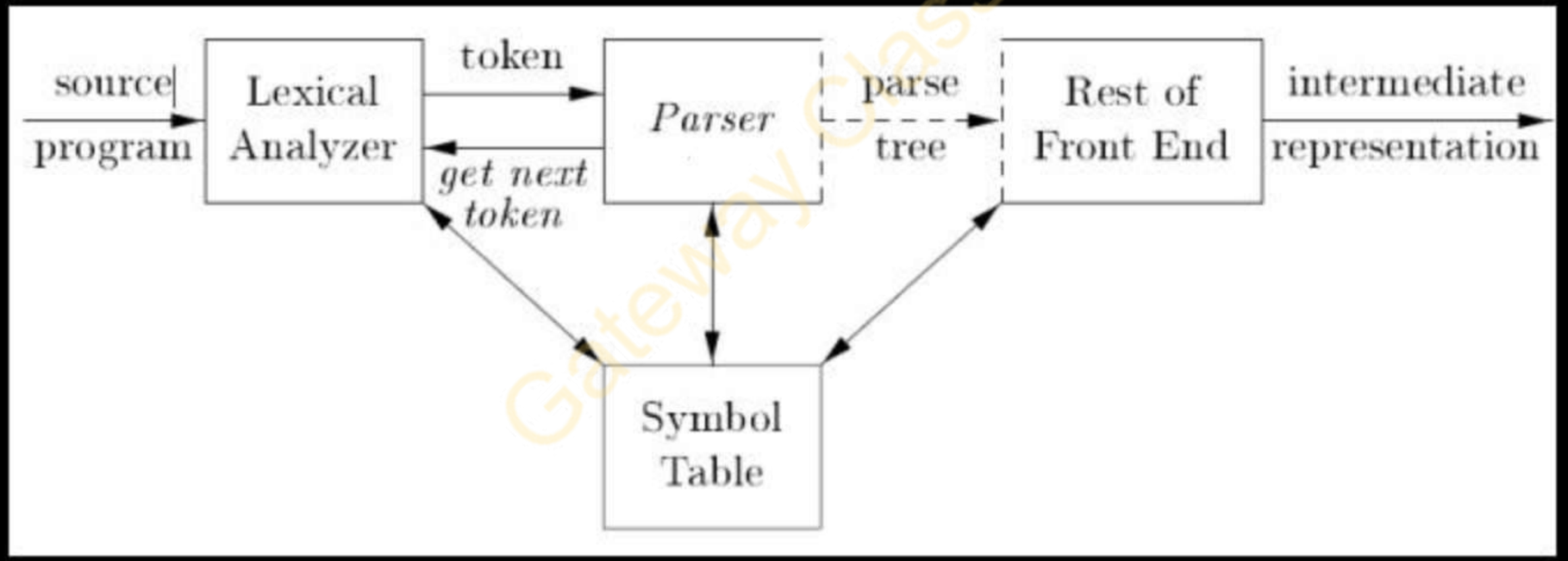
Gateway Classes

Use of Regular Expressions in Pattern Matching

Pattern matching involves searching for specific patterns within text. Internally, regular expressions are converted into finite automata, which allow efficient and fast pattern matching in linear time.

The Role of the Parser

- Parser verifies that the string of tokens coming from lexical analyzer can be generated by the grammar for the source language.
- Parser report any syntax errors and recover from commonly occurring errors to continue processing the remainder of the program(to detect subsequent errors if any).
- Parser constructs a parse tree and passes it to the rest of the compiler for further processing.



Gateway Classes

Formal Grammars and Their Application to Syntax Analysis

- Formal grammars are used to define the structure of a language. They specify the rules that define which strings of symbols (or tokens) are valid within that language. Syntax analysis applies these grammar rules to check the correctness of source code.

Grammar G can be defined as: G= (V, T, P, S)

T: finite set of terminal symbols, **V:** a finite set of non-terminal symbols

P: set of production rules, **S**: start symbol.

Non-terminals (V): Variables representing syntactic categories (e.g., Sentence, Expression) represented by uppercase letters (e.g., S, A, B).

Terminals (T): The actual symbols or tokens of language (e.g., +, id, if) that cannot be replaced further. Represented by lowercase letters (e.g., a, b, c) or specific symbols.

Production Rules (P): Rules showing how non-terminals can be rewritten (e.g., Sentence -> Noun Verb).

Start Symbol (S): A special non-terminal where derivation begins.

Gateway Classes

Formal Grammars and Their Application to Syntax Analysis

Application of grammar to Syntax Analysis (Parsing): To check if the sequence of tokens from lexical analyzer conforms to the formal grammar's rules.

Process: Takes tokens and tries to derive them from the start symbol using production rules.

Output: A parse tree.

Error Handling: If tokens can't form a valid structure, a syntax error is reported.

Context-Free Grammars (CFGs)

- CFGs define the syntax of programming languages.
- They specify rules to form valid statements and expressions.
- Parsers use CFGs to check syntax correctness. They help generate parse trees.
- A grammar is said to be CFG if every production is in form of:

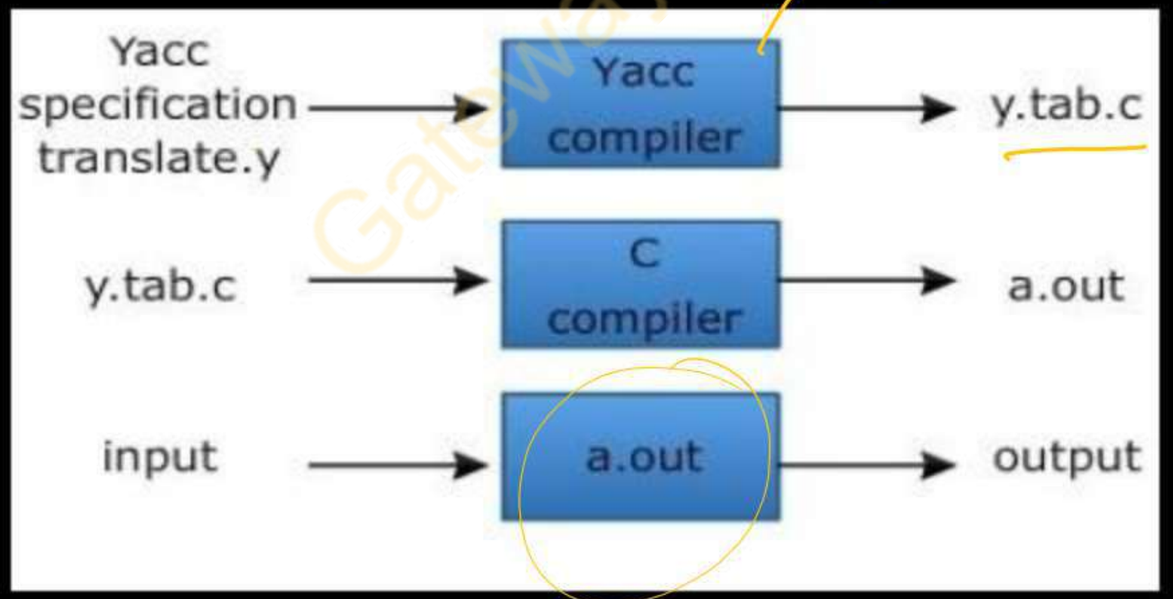
$$G \rightarrow (V \cup T)^* \text{ , where } G \in V$$

- LHS can only be a Variable. RHS can be a Variable or Terminal or combination of Variable and Terminal.

Gateway Classes

YACC (Yet Another Compiler Compiler)

- YACC compiler is a tool used to generate parsers for programming languages.
- It is commonly used in compiler construction.
- It is commonly used along with **Lex**, which performs lexical analysis.
- YACC is used in the **syntax analysis phase** of a compiler. It checks whether the sequence of tokens generated by the lexical analyzer follows the grammar of the programming language.
- It takes a grammar specification (written in a YACC file) as input and produces a C program that can parse a sequence of tokens according to that grammar rules.



LALR(1)

Gateway Classes

Workings of YACC

Step 1: YACC Specification (translate.y) describes the syntax of the language and contains:

- Grammar rules, Token declarations, Semantic actions (C code)

yyparse()

Step 2: The YACC compiler reads the grammar specification and generates a C source file called y.tab.c contains:

- Parsing tables, and Parsing logic
- Calls to yylex() for token input

Step 3: y.tab.c is compiled using a C compiler to produce an executable file, usually named a.out.

Step 4: The executable a.out is run with a source program as input.

- yyparse() starts parsing. It calls yylex() to obtain tokens
- Grammar rules are matched and Semantic actions are performed
- Result or error messages are generated

Gateway Classes

A.y (YACC source file)

```
%{
#include <stdio.h>
#include <stdlib.h>
%}
```

definition Section

```
%token NUMBER
```

Declares NUMBER as a token

Rules section

```
%%
expr : expr '+' NUMBER   { $$ = $1 + $3; }
     | NUMBER            { $$ = $1; }
     ;
%%
```

Defines grammar rules, Supports addition of numbers

pattern

action

```
int main() {
    printf("Enter an expression: ");
    yyparse();
    return 0;
}
int yyerror(char *s) {
    printf("Syntax Error\n");
    return 0;
}
```

- yyparse() starts parsing
- yyerror() handles syntax errors

user subroutine section

Gateway Classes

A.l program corresponding to A.y.

```
%{
#include "y.tab.h"
#include <stdlib.h>
%}

%%
[0-9]+      { yylval = atoi(yytext); return NUMBER; }
"+"         { return '+'; }
[ \t\n]+    ;           /* ignore spaces */
.           ;           /* ignore other characters */
%%

int yywrap() {
    return 1;
}
```

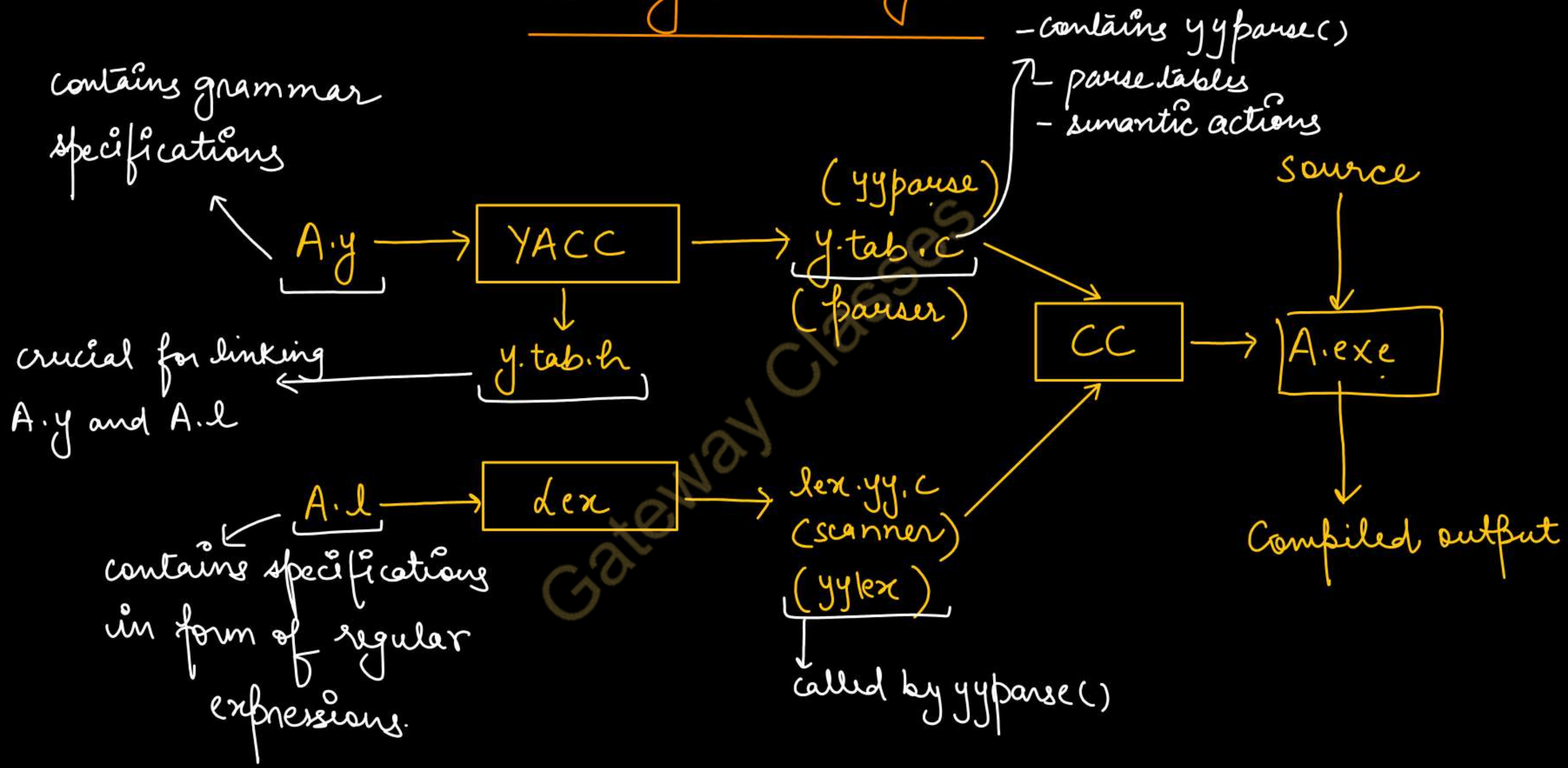
- "y.tab.h" connects Lex with YACC and imports token definitions like NUMBER
- stdlib.h is required for atoi() function
- [0-9]+ matches one or more digits
- yytext contains the matched string (e.g., "123")
- atoi(yytext) converts string → integer
- yylval stores the numeric value for YACC
- return NUMBER;
- Sends the token NUMBER to YACC

yywrap() → called by yylex() when it reaches the end of input file. It determines whether to terminate the scanning or continue with the next file.

return 1; → for terminate

Gateway Classes

Linking lex and yacc



Gateway Classes

Compiler Design

Unit-1 : DPP-6

1. Explain lexical analyzer generator and explain the different parts of the program. **(AKTU 2022-23, 2017–18)**
2. Describe the relationship between finite state machines and regular expressions. Discuss how regular expressions are used in lexical analysis and pattern matching. **(AKTU 2023-24)**
3. What is YACC? Discuss about it. **(AKTU 2020-21)**
4. Write short note on YACC parser generator. **(AKTU 2017-18)**

Gateway Classes

AKTU

B. Tech Sem-VI

CS IT & CS Allied

GATEWAY CLASSES

Compiler Design

UNIT-1 Lecture-7

Gateway Classes



Todays Target

- Derivation tree
- Ambiguous grammar and unambiguous Grammar
- AKTU PYQs

By Dr. Nidhi Parashar Ma'am

- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified

Derivation

- A Derivation is the successive application of production rules to produce the desired input string.
- The objective is to find a valid chain of production rule applications that generate the desired input string while adhering to the programming language's grammar rules.

Two types of derivation: 1. Left-most Derivation 2. Right-most Derivation

Left-most Derivation:

- The leftmost non-terminal is expanded according to production rule associated with that non-terminal. This operation continues from left to right, matching the scan of the input string from left to right.

Input: ab

S -> AB
A -> a
B -> b

S → AB [A → a]
→ aB [B → b]
→ ab

Right-most Derivation:

- The rightmost non-terminal is expanded according to production rule associated with that non-terminal. This operation continues from right to left, matching the scan of the input string from right to left.

Input: ab

S -> AB
A -> a
B -> b

S → AB [B → b]
→ Ab
→ ab

Gateway Classes

Derivation Tree

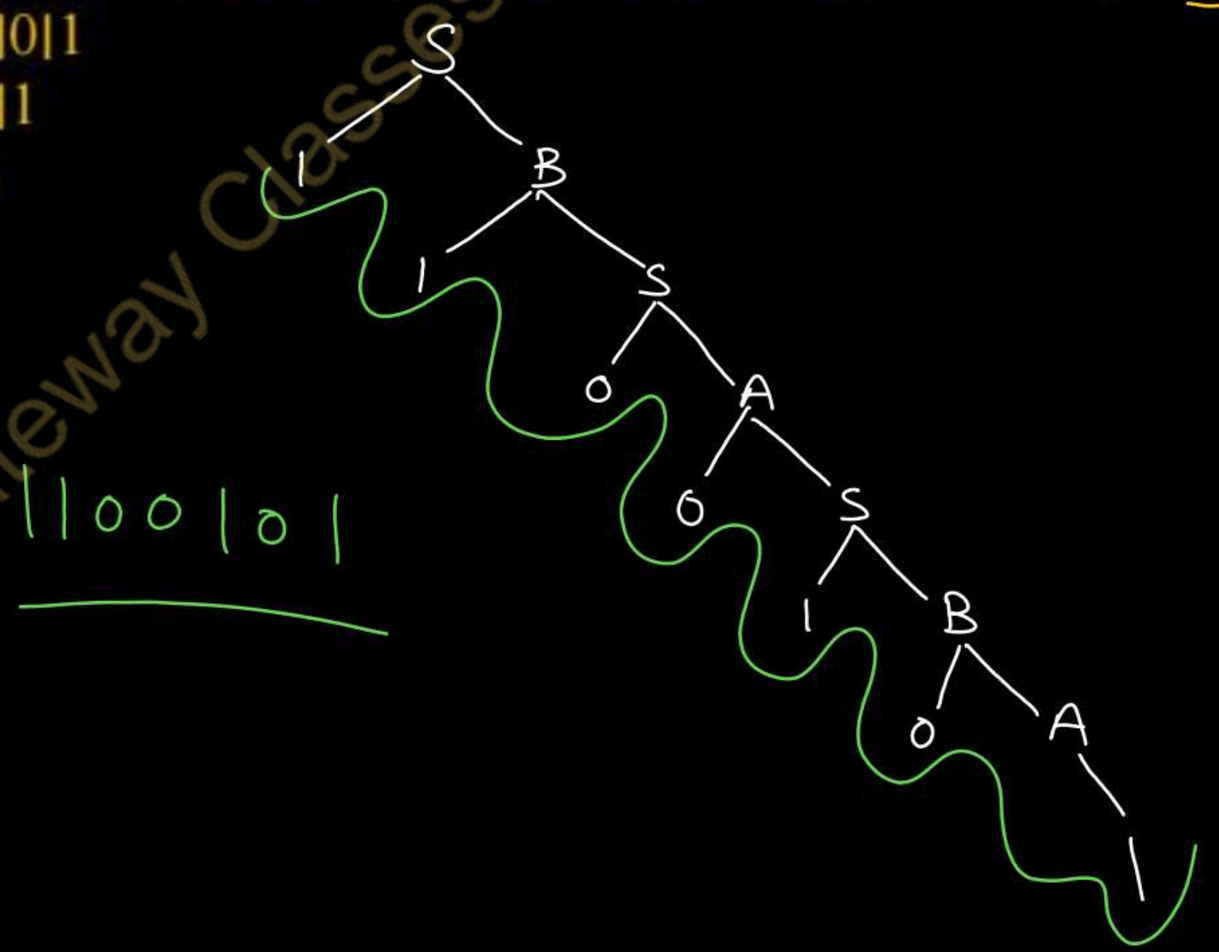
A derivation tree is a graphical representation that shows how a string is formed from the start symbol of a Context-Free Grammar (CFG) by applying production rules step-by-step. Derivation tree is also known as a parse Tree, or syntax tree.

Ex: Given the grammar: Construct the leftmost derivation (LMD) and parse tree for string: 1100101

$S \rightarrow 0A|1B|0|1$
$A \rightarrow 0S|1B|1$
$B \rightarrow 0A|1S$

1100101

$S \rightarrow 1B$ [S→1B]
$\rightarrow 11S$ [B→1S]
$\rightarrow 110A$ [S→0A]
$\rightarrow 1100S$ [A→0S]
$\rightarrow 11001B$ [S→1B]
$\rightarrow 110010A$ [B→0A]
$\rightarrow 1100101$ [A→1]



Gateway Classes

Q3. Given the grammar: $S \rightarrow (L) | a$

$L \rightarrow L,S | S$

Find the leftmost derivation, rightmost derivation, and parse tree for the string: (a,(a,a))

LMD (a,(a,a))

$S \rightarrow (L)$ [$S \rightarrow (L)$]

$S \rightarrow (L,S)$ [$L \rightarrow L,S$]

$\rightarrow (S,S)$ [$L \rightarrow S$]

$\rightarrow (a,S)$ [$S \rightarrow a$]

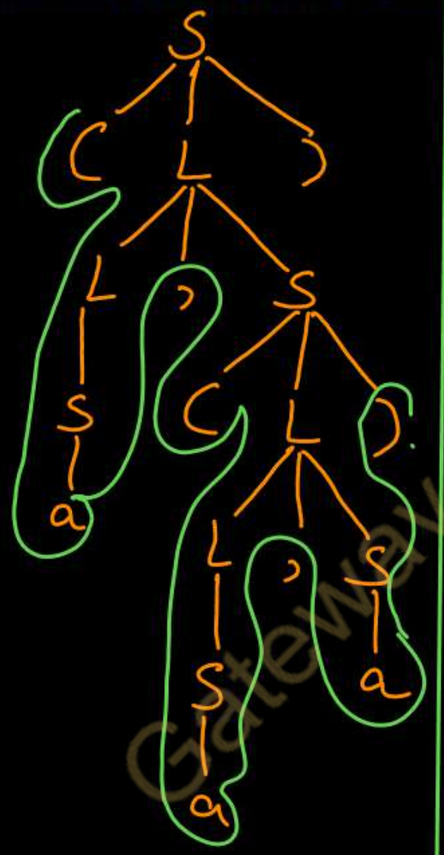
$\rightarrow (a,(L))$ [$S \rightarrow (L)$]

$\rightarrow (a,(L,S))$ [$L \rightarrow L,S$]

$\rightarrow (a,(S,S))$ [$L \rightarrow S$]

$\rightarrow (a,(a,S))$ [$S \rightarrow a$]

$\rightarrow (a,(a,a))$ [$S \rightarrow a$]



RMD

$S \rightarrow (L)$ [$S \rightarrow (L)$]

$\rightarrow (L,S)$ [$L \rightarrow L,S$]

$\rightarrow (L,(L))$ [$S \rightarrow (L)$]

$\rightarrow (L,(L,S))$ [$L \rightarrow L,S$]

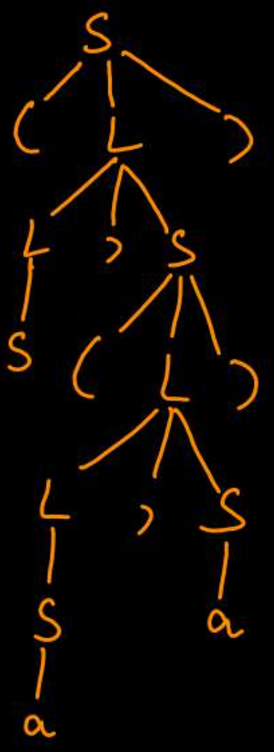
$\rightarrow (L,(L,a))$ [$S \rightarrow a$]

$\rightarrow (L,(S,a))$ [$L \rightarrow S$]

$\rightarrow (L,(a,a))$ [$S \rightarrow a$]

$\rightarrow (S,(a,a))$ [$L \rightarrow S$]

$\rightarrow (a,(a,a))$ [$S \rightarrow a$]



Gateway Classes

Q4. Derive the rightmost derivation for the string: aaabaab

Grammar: S→ aS |aSbS |ε

aaabaab

$S \rightarrow aS$ [S→aS]

$\rightarrow aaSbS$ [S→aSbS]

$\rightarrow aaSbaSbS$ [S→aSbS]

$\rightarrow aaSbaSb\epsilon$ [S→ε]

$\rightarrow aaSbaaSb$ [S→aS]

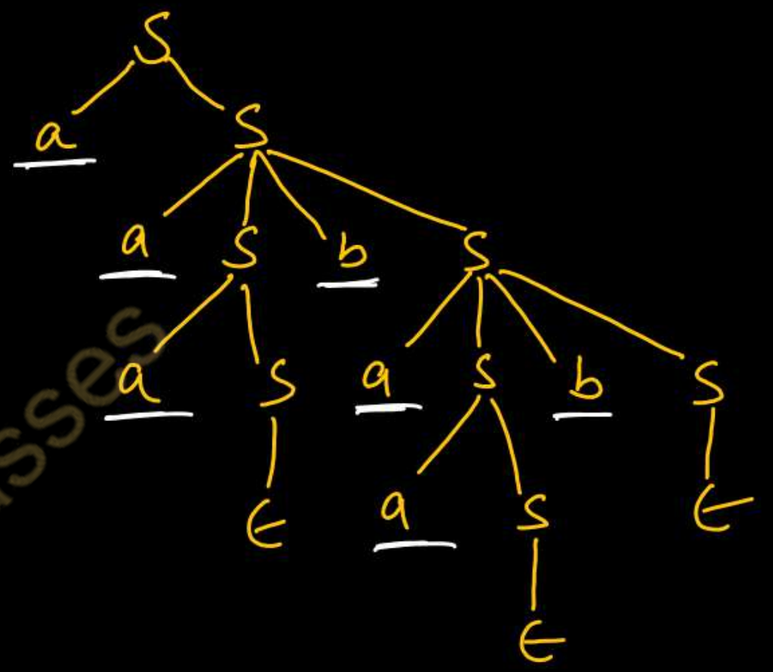
$\rightarrow aaSbaa\epsilon b$ [S→ε]

$\rightarrow aaaSbaab$ [S→aS]

$\rightarrow aaa\epsilon baab$ [S→ε]

↓

aaabaab



Gateway Classes

On the basis of No. of parse tree

Ambiguous Grammar

- A grammar is said to be ambiguous if there exists more than one left most derivation or more than one right most derivation or more than one parse tree for a given input string.
- If a grammar is ambiguous:
  - ~~Compiler~~ parser cannot decide which parse tree to choose
  - Leads to confusion in syntax analysis
  - It may cause incorrect code generation

Unambiguous Grammar

- A grammar can be unambiguous if the grammar does not contain more than one left most derivation or more than one right most derivation or more than one parse tree for any input string.

Example:

S -> AB

A -> Aa / a

B -> b

Gateway Classes

Q: Prove that the following grammar is ambiguous: **(AKTU 2021-22)**

$E \rightarrow E+E \mid E*E \mid id.$

AKTU-2023-24.

id + id * id

LMD-1.

$E \rightarrow E+E$
$\rightarrow id+E$
$\rightarrow id+E*E$
$\rightarrow id+id*E$
$\rightarrow id+id*id$

LMD-2

$E \rightarrow E*E$
$\rightarrow E+E*E$
$\rightarrow id+E*E$
$\rightarrow id+id*E$
$\rightarrow id+id*id$

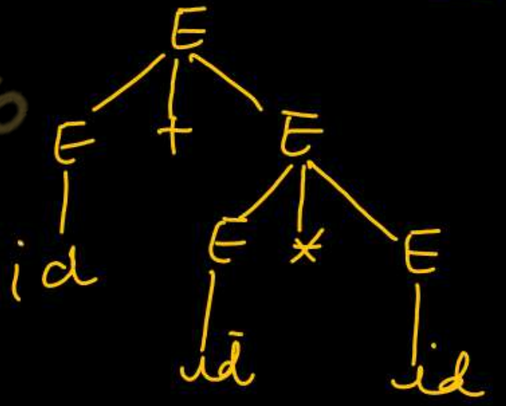
RMD-1.

$E \rightarrow E+E$
$\rightarrow E+E*E$
$\rightarrow E+E*id$
$\rightarrow E+id*id$
$\rightarrow id+id*id$

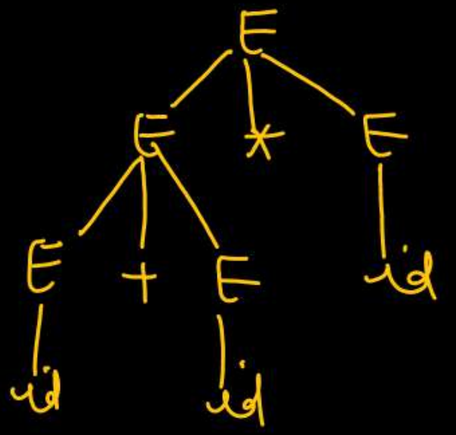
RMD-2.

$E \rightarrow E*E$
$\rightarrow E*id$
$\rightarrow E+E*id$
$\rightarrow E+id*id$
$\rightarrow id+id*id$

parse tree-1



parse tree-2.

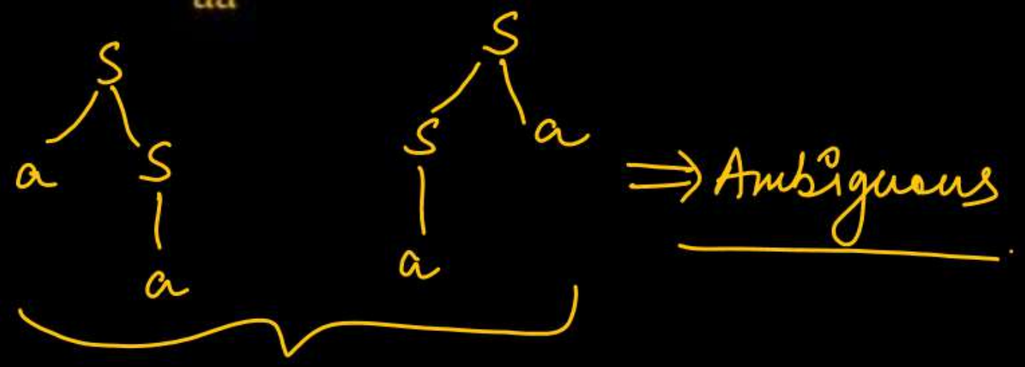


Gateway Classes

Q: Determine grammar is ambiguous or not:

$S \rightarrow aS \mid Sa \mid a$

aa

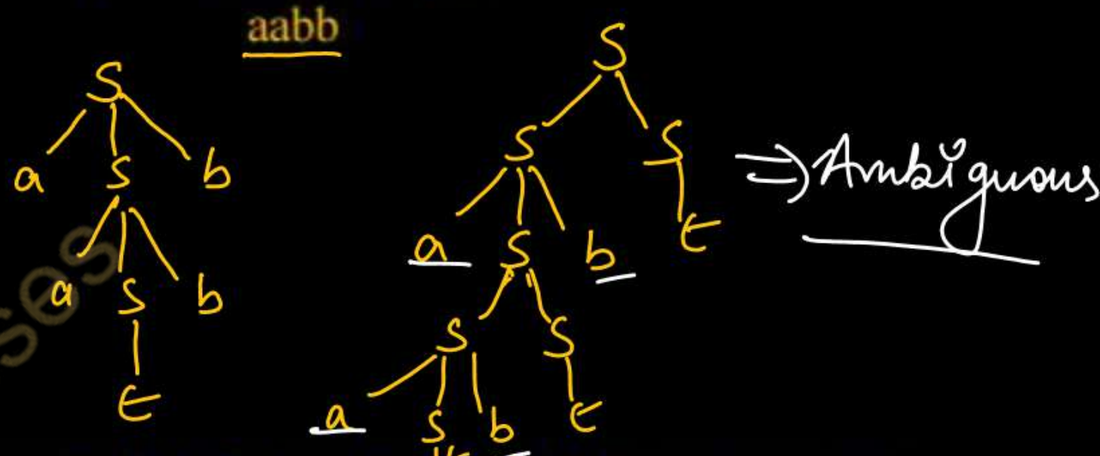


⇒ Ambiguous.

Q: Determine if grammar is ambiguous:

$S \rightarrow aSb \mid SS \mid \varepsilon$

aabb



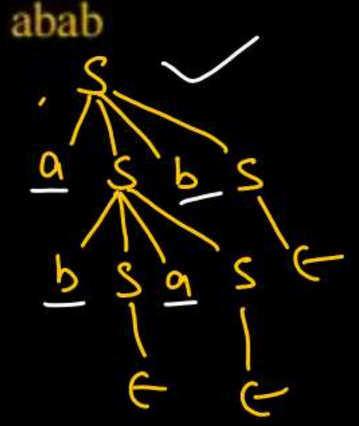
⇒ Ambiguous

Q: Check grammar is ambiguous or not:

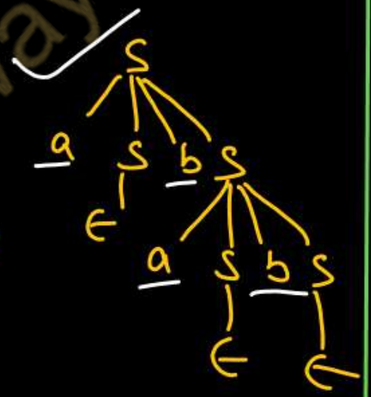
$S \rightarrow aSbS \mid bSaS \mid \varepsilon$ (AKTU 2024-25)

abab

$S \rightarrow aSbS$
$\rightarrow abSaSbS$
$\rightarrow abεaSbS$
$\rightarrow abaεbS$
$\rightarrow ababε$
$\rightarrow abab$



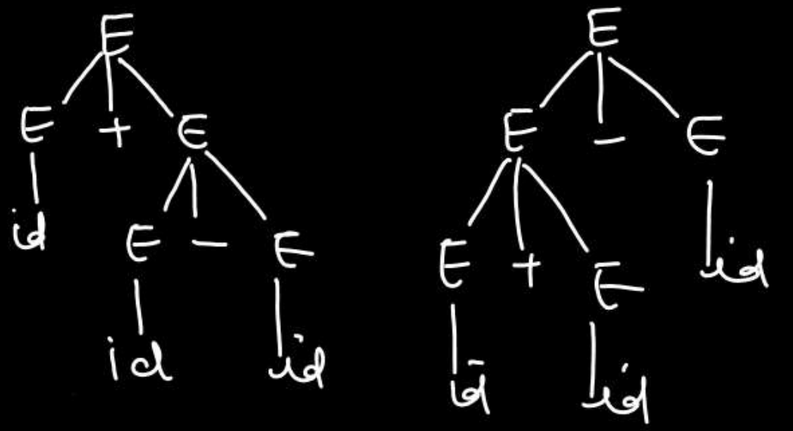
$S \rightarrow aSbS$
$\rightarrow aSbaSbS$
$\rightarrow aSbaSbε$
$\rightarrow aSbaεb$
$\rightarrow aεbab$
$\rightarrow abab$



⇒ Ambiguous.

Q: Check the ambiguity of the grammar:

$E \rightarrow E + E \mid E - E \mid id$ id + id - id



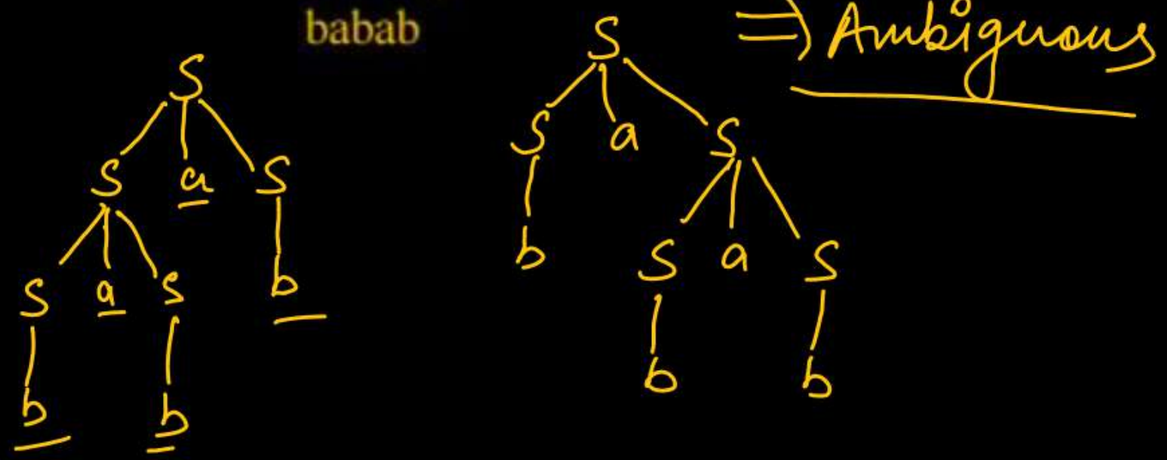
⇒ Ambiguous.

Gateway Classes

Q: Determine grammar is ambiguous or not:

$S \rightarrow SaS \mid b$

babab

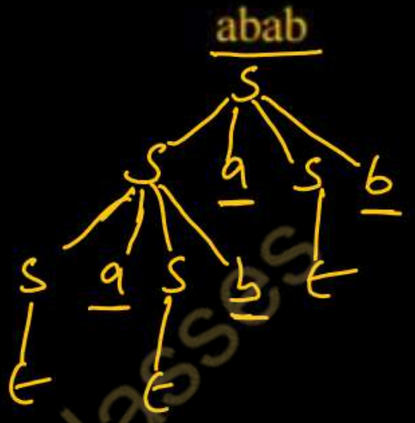


⇒ Ambiguous

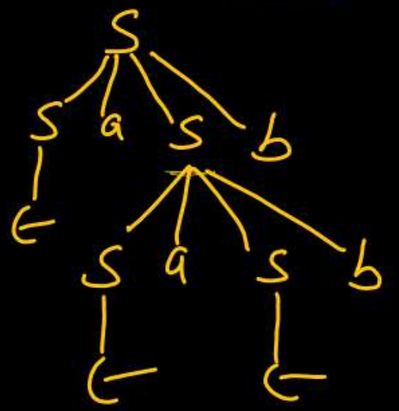
Q: Check the ambiguity of the grammar:

$S \rightarrow SaSb \mid \varepsilon.$ (Unambiguous)

abab



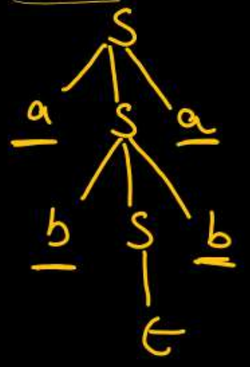
aabb



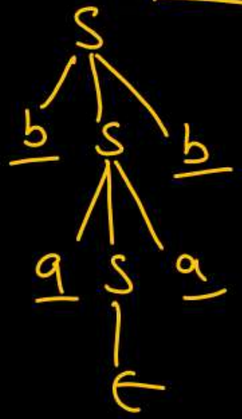
Q: Check grammar is ambiguous or not:

$S \rightarrow aSa \mid bSb \mid \varepsilon$ (Unambiguous)

abba



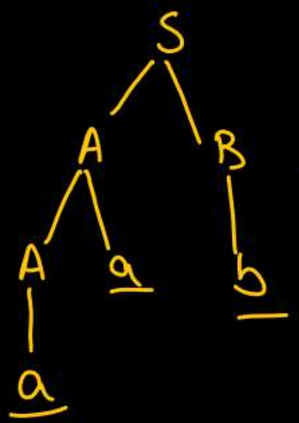
baab



Q: Check grammar is ambiguous or not:

$S \rightarrow AB,\ A \rightarrow Aa \mid a,\ B \rightarrow b.$

aab



⇒ Unambiguous

Gateway Classes

Compiler Design

Unit-1 : DPP-7

1. Using rightmost derivations or the derivation trees show that the grammar with following production rules is ambiguous. $S \rightarrow aSbS \mid bSaS \mid \varepsilon$. (AKTU 2024-25)
2. What is an ambiguous grammar? Give example. (AKTU 2021-22)

Gateway Classes

AKTU

B. Tech Sem-VI
CS IT & CS Allied

GATEWAY CLASSES

Compiler Design

UNIT-1 Lecture-8

Gateway Classes



Todays Target

- Removing ambiguity from CFGs
- Removing Left Recursion
- AKTU PYQs

By Dr. Nidhi Parashar Ma'am

- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified

Changing Ambiguous Grammar to Unambiguous Grammar

Consider the grammar: $E \rightarrow E + E \mid E * E \mid id$ → E↑E

This grammar is ambiguous because it does not define operator precedence and associativity.

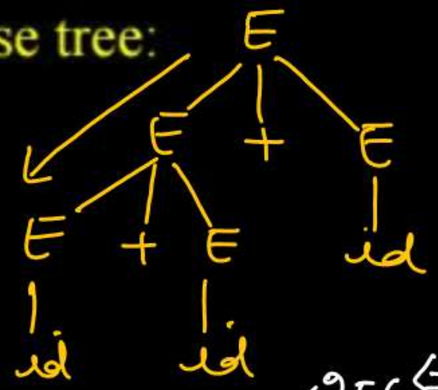
1. Ambiguity due to associativity: id + id + id

(id + id) + id → Left associative

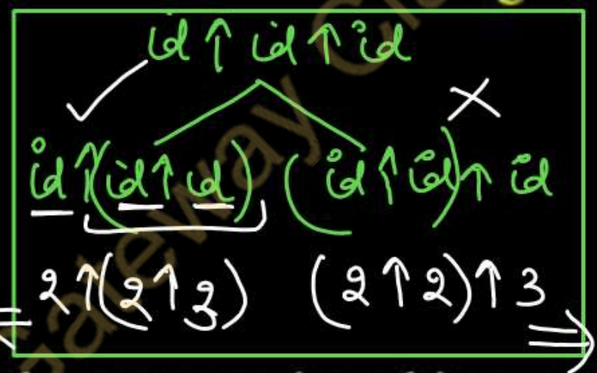
↑ ⇒ R-L associative

id + (id + id) → Right associative

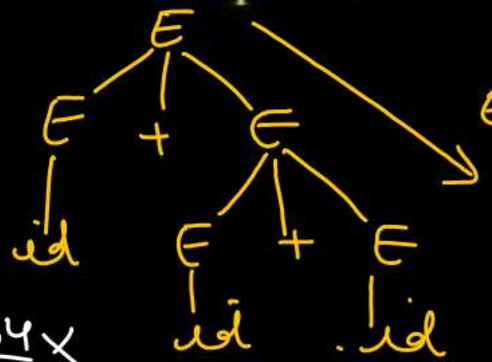
Left associative parse tree:



correct parse tree



Right associative parse tree:



Incorrect parse tree

- Since both interpretations are possible, the grammar is ambiguous. But, '+' operator is left associative.

  Therefore, the correct evaluation is: (id + id) + id ✓

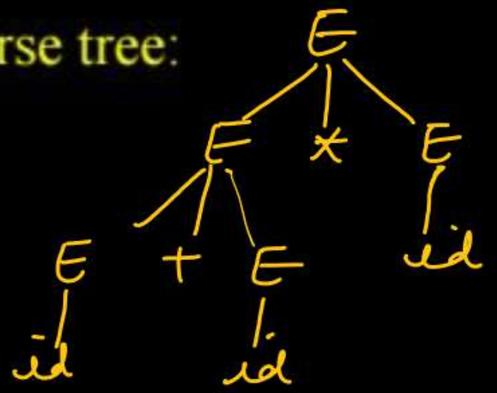
- Reason for ambiguity: Associativity is not specified in the grammar.

Gateway Classes

2. Ambiguity due to Precedence: id + id * id

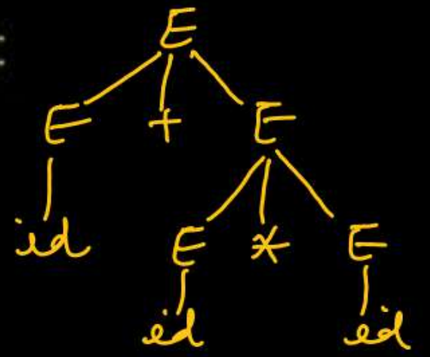
(id + id) * id → Incorrect

Parse tree:



id + (id * id) → Correct

Parse tree:



Gateway Classes

- The '*' operator has higher precedence than '+'. Therefore, multiplication must be evaluated first.
- Therefore, the correct evaluation is: id + (id * id)
- Reason for ambiguity: Operator precedence is not defined in the grammar.
- **Ambiguity can be removed by:**

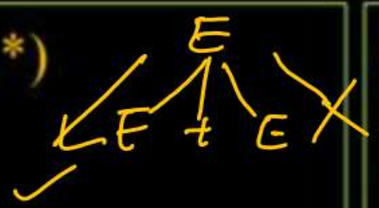
1. Defining associativity using recursion

2. Defining precedence using multiple grammar levels

Removing Ambiguity Due To Associativity

$$E \rightarrow E + E \mid E * E \mid id$$

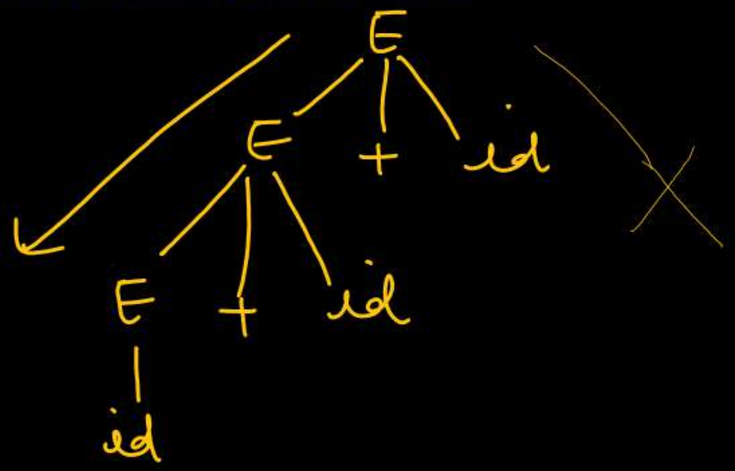
(a) For Left Associative Operators (+, *)



Grammar: $E \rightarrow E + id \mid id$

This grammar allows only left associative parse trees.

Parse Tree For id + id + id:



Left recursion ensures left associativity

Only one parse tree is possible, so ambiguity is removed.

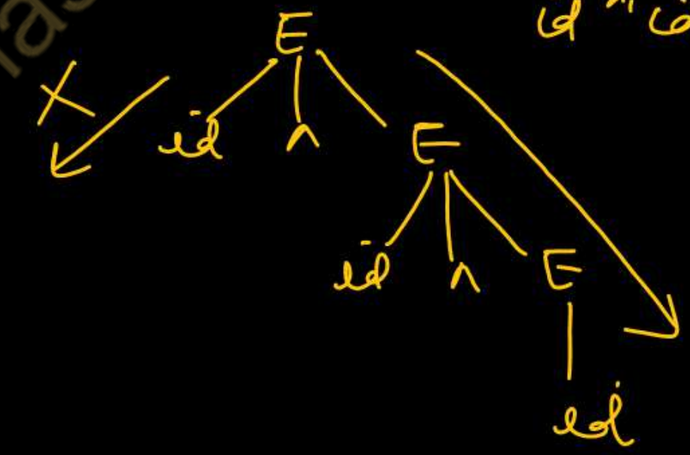
(b) For Right associative operators (^)

For string: id ^ id ^ id Correct evaluation: id ^ (id ^ id)

Grammar: $E \rightarrow id \wedge E \mid id$ ⇒ Right Recursive

This grammar allows only left associative parse trees.

id ^ id ^ id



Right recursion ensures right associativity.

Only one parse tree is possible, so no ambiguity.

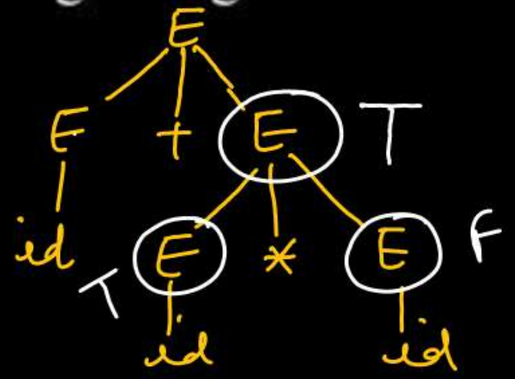
Gateway Classes

Removing Ambiguity Due To Precedence

$$E \rightarrow E + E \mid E * E \mid id$$

- Higher precedence operators should appear at lower levels in the grammar.

Unambiguous grammar using precedence levels:

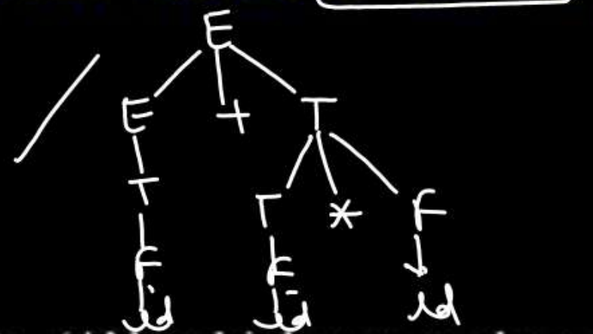


$E \rightarrow E + T \mid T$ → left recursive grammar to ensure L-R associativity.

$T \rightarrow T * F \mid F$ → Suppose we need to generate only * like id * id * id

$F \rightarrow id$ → Suppose we want to generate only id

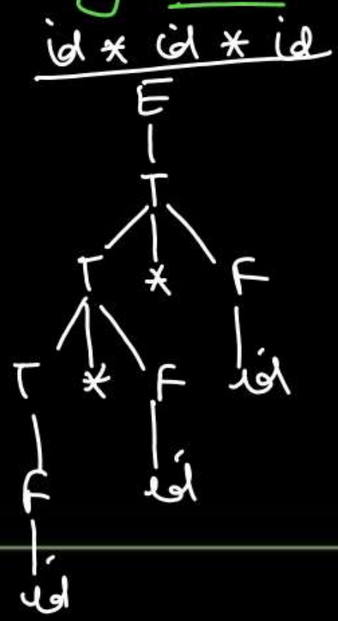
Parse Tree For id + id * id:



→ once * is generated, there is no way to generate any +







- '*' has higher precedence than '+'
- Left recursion ensures left associativity. Only one parse tree is generated

GatewayClasses

Removing Ambiguity Due To Precedence

$$E \rightarrow E + E \mid E * E \mid id$$

$E \, ^{\wedge} \, E$

Adding power operator (^):

Properties of ^: Highest precedence and Right associative

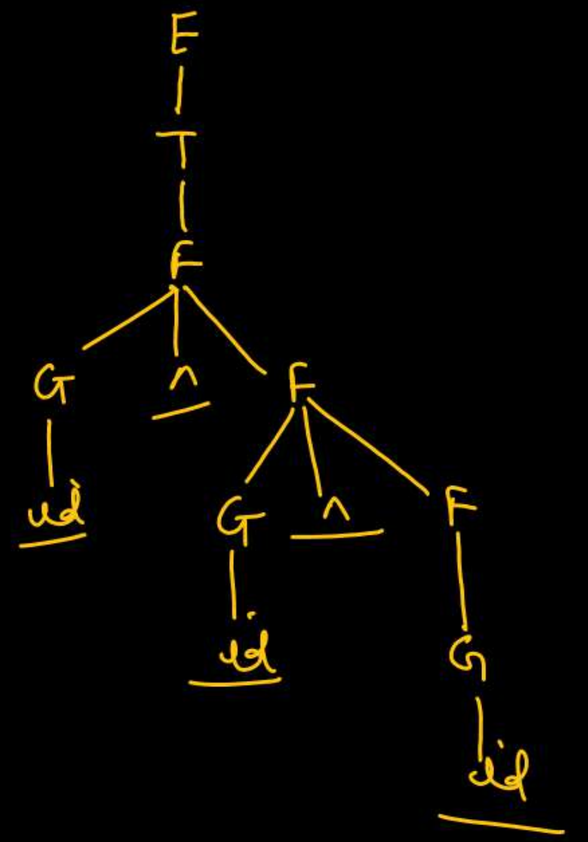
Final Unambiguous Grammar:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow G \, ^{\wedge} \, F \mid G$$
$$G \rightarrow id$$

to ensure Right associativity

Parse tree for id ^ id ^ id:

parse tree for id ^ id ^ id



GatewayClasses

Recursive Grammar

- A grammar is said to be recursive if it contains at least one production that has the same variable at both its LHS and RHS. It generates infinite number of strings.

$S \rightarrow a\,S \mid b$ $\qquad$ $S \rightarrow S\,a \mid b$ $\qquad$ $S \rightarrow a\,S\,b \mid c$

Non-Recursive Grammar

Non-recursive grammar contains no production that has the same variable at both its LHS and RHS. It generates finite number of strings.

$S \rightarrow aA / bB$
$A \rightarrow a / b$
$B \rightarrow c / d$



The language generated from this grammar is L = { aa , ab , bc , bd }

Gateway Classes

Recursive Grammar

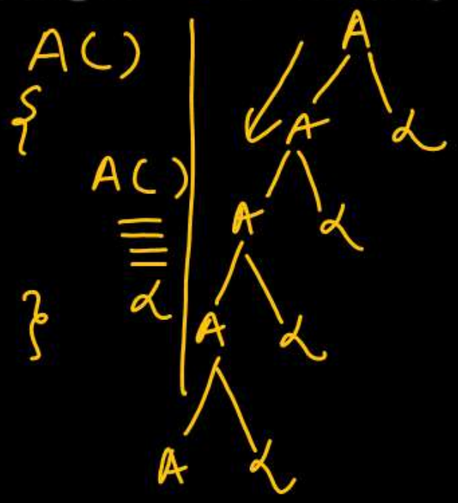
- A grammar is said to be recursive if it contains at least one production that has the same variable at both its LHS and RHS. It generates infinite number of strings.

Left Recursive Grammar:

Grammar is said to be left recursive if the leftmost variable of RHS is same as variable of LHS.

Example- $A \rightarrow A\alpha / \beta$

$\alpha, \beta \in (V \cup T)^*$



Language generated

$A \rightarrow \beta$ ✓

$A \rightarrow A\alpha$
$\rightarrow \beta\alpha$ ✓

$A \rightarrow A\alpha$
$\rightarrow A\alpha\alpha$
$\rightarrow \beta\alpha^2$ ✓

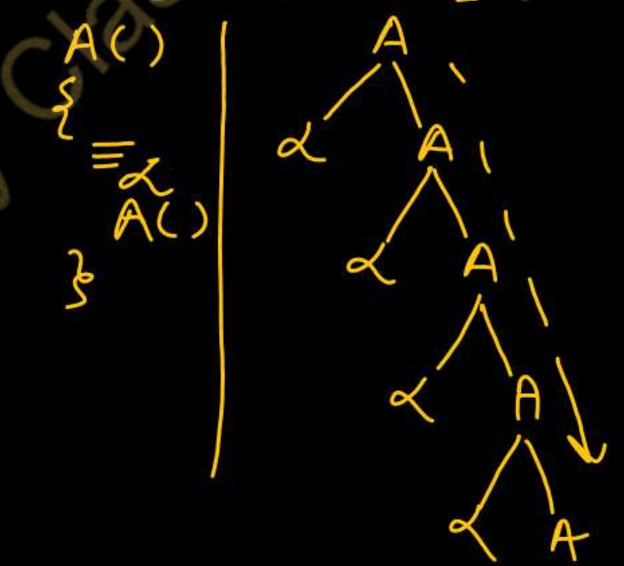
$A \rightarrow A\alpha$
$\rightarrow A\alpha\alpha$
$\rightarrow A\alpha\alpha\alpha$
$\rightarrow \beta\alpha^3$ ✓

$\Rightarrow \boxed{\beta\alpha^*}$

Right Recursive Grammar:

Grammar is said to be left recursive if the rightmost variable of RHS is same as variable of LHS.

Example- $A \rightarrow \alpha A / \beta$



Language generated

$A \rightarrow \beta$ ✓

$A \rightarrow \alpha A$
$\rightarrow \alpha\beta$ ✓

$A \rightarrow \alpha A$
$\rightarrow \alpha\alpha A$
$\rightarrow \alpha\alpha\beta \Rightarrow \alpha^2\beta$

$A \rightarrow \alpha A$
$\rightarrow \alpha\alpha A$
$\rightarrow \alpha\alpha\alpha A$
$\rightarrow \alpha\alpha\alpha\beta \Rightarrow \alpha^3\beta$ ✓

$\boxed{\alpha^*\beta}$

Gateway Classes

Note:

- The grammar which is either left recursive or right recursive is always unambiguous.

  Examples-

  $S \rightarrow aS / b$ (Unambiguous Grammar)

  $S \rightarrow Sa / b$ (Unambiguous Grammar)

- The grammar which is both left recursive and right recursive is always ambiguous.

  Example-

  $E \rightarrow E + E / E - E / E \times E / id$

- Left recursive grammar is not suitable for Top-down parsers.
- This is because it makes the parser enter into an infinite loop.
- To avoid this situation, it is converted into its equivalent right recursive grammar.
- This is done by eliminating left recursion from the left recursive grammar.

Gateway Classes

Left Recursion

Direct (immediate)Left Recursion

$A \rightarrow A\alpha|\beta$

Indirect Left Recursion

$S \rightarrow A\alpha$

$A \rightarrow S\beta$

$A \rightarrow S\beta$

$A \alpha \beta$

$A \rightarrow A\alpha\beta$

$E \rightarrow E+T \mid T$

A A α β

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

Eliminating Direct (immediate)Left Recursion

We can eliminate direct left recursion from a grammar without altering the language it generates.

$A \rightarrow A\alpha|\beta$

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \varepsilon$ → To produce β only

Gateway Classes

Eliminate left recursion from the following grammars:

$\bar{S} \rightarrow \bar{S}01S0S \mid 01$

A A α β

$S \rightarrow 0 \mid S'$

$S' \rightarrow 01S0SS' \mid \epsilon$

$S \rightarrow (L) \mid x$ → No LR present

$L \rightarrow L, S \mid S$

A A α β

$L \rightarrow SL'$

$L' \rightarrow ,SL' \mid \epsilon$

$S \rightarrow (L) \mid x$

$L \rightarrow SL'$

$L' \rightarrow ,SL' \mid \epsilon$

$E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow id$ No LR

1) $E \rightarrow E+T \mid T$

A A α β

$E \rightarrow TE'$ ✓

$E' \rightarrow +TE' \mid \epsilon$ ✓

2) $T \rightarrow T*F \mid F$

A A α β

$T \rightarrow FT'$ ✓

$T' \rightarrow *FT' \mid \epsilon$ ✓

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow id$

Gateway Classes

Removing Left recursion present in multiple productions of a single LHS

Given Grammar: $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \ldots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_n$

(where $\beta_1, \beta_2, \ldots, \beta_n$ do **not** start with A)

$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \ldots \mid \beta_n A'$

$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \ldots \mid \alpha_m A' \mid \varepsilon$

Example:

$S \rightarrow Sx \mid SSb \mid xS \mid a$

A $\quad$ A α_1 $\quad$ A α_2 $\quad$ β_1 $\quad$ β_2

$S \rightarrow xSS' \mid aS'$

$S' \rightarrow xS' \mid SbS' \mid \in$

Gateway Classes

Eliminate Indirect left recursion from the following grammars:

$S \to Aa$

$A \to Sb \mid c$

$A \to Aab \mid C$

A A α β

$A \to CA'$

$A' \to abA' \mid \epsilon$

$S \to Aa$

$S \to Aa \mid b$

$A \to Ac \mid Sd \mid f$

$S \to Aa \mid b$ ✓

$A \to Ac \mid Aad \mid bd \mid f$

A A α_1 A α_2 β_1 β_2

$A \to bdA' \mid fA'$ ✓

$A' \to cA' \mid adA' \mid \epsilon$ ✓

$S \to Aa \mid b$

$A \to bdA' \mid fA'$

$A' \to cA' \mid adA' \mid \epsilon$

Remove left recursion: **(AKTU 2017-18)**

$S \to AB$

$A \to BS \mid b$

$B \to SA \mid a$ ✓

$B \to SA \mid a$

$B \to ABA \mid a$ [$S \to AB$]

$B \to BSBA \mid bBA \mid a$

A A α β_1 β_2

$S \to AB$

$A \to BS \mid b$

$B \to bBAB' \mid aB'$

$B' \to SBAB' \mid \epsilon$

Gateway Classes

Eliminate Indirect left recursion from the following grammars:

Remove left recursion: **(AKTU 2023-24)**

$S \rightarrow Aa \mid b \mid \varepsilon$

$A \rightarrow Ac \mid SAd \mid \varepsilon$

$A \rightarrow Ac \mid SAd \mid \epsilon$

$A \rightarrow Ac \mid Aa Ad \mid bAd \mid Ad \mid \epsilon$

$A \quad A \; \alpha_1 \; A \; \alpha_2 \quad \beta_1 \quad A \; \alpha_3 \quad \beta_2$

$A \rightarrow bAd\,A' \mid \epsilon A'$

$A' \rightarrow cA' \mid aAdA' \mid dA' \mid \epsilon$

$S \rightarrow Aa \mid b \mid \epsilon$

$A \rightarrow bAdA' \mid A'$

$A' \rightarrow cA' \mid aAdA' \mid dA' \mid \epsilon$

Gateway Classes

Consider the following grammar and eliminate left recursion:

$$X \rightarrow XSb \mid Sa \mid b$$
$$S \rightarrow Sb \mid Xa \mid a$$

For $X \rightarrow \underbrace{X}_{A}\underbrace{Sb}_{A\ \alpha} \mid \underbrace{Sa}_{\beta_1} \mid \underbrace{b}_{\beta_2}$

$$X \rightarrow Sax' \mid bx'$$
$$x' \rightarrow Sbx' \mid \epsilon$$

$S \rightarrow \underbrace{Sb}_{A\ \alpha_1} \mid \underbrace{Sa}_{A\ \alpha_2}\underbrace{x'a}_{\beta_1} \mid \underbrace{bx'a}_{\beta_1} \mid \underbrace{a}_{\beta_2}$

$$S \rightarrow bx'as' \mid as'$$
$$S' \rightarrow bs' \mid ax'as' \mid \epsilon$$

$S \rightarrow Sb \mid Xa \mid a$ ✗

$S \rightarrow \underline{S}b \mid \underline{S}ax'a \mid bx'a \mid a$

$$X \rightarrow Sax' \mid bx'$$
$$x' \rightarrow Sbx' \mid \epsilon$$
$$S \rightarrow bx'as' \mid as'$$
$$S' \rightarrow bs' \mid ax'as' \mid \epsilon$$

Gateway Classes

Eliminate Indirect left recursion from the following grammars:

Remove left recursion: **(AKTU 2016-17)**

$A \rightarrow Ba \mid Aa \mid c$

$B \rightarrow Bb \mid Ab \mid d$

$A \rightarrow Ba \mid Aa \mid c$

$A \rightarrow \beta_1 \quad A\alpha \quad \beta_2$

✓ $A \rightarrow BaA' \mid cA'$

✓ $A' \rightarrow aA' \mid \epsilon$

$B \rightarrow Bb \mid Ab \mid d$ ✗

$B \rightarrow Bb \mid BaA'b \mid cA'b \mid d$

$B \rightarrow Bb \mid Ba\,A'b \mid cA'b \mid d$

$B \rightarrow \beta_1 \quad A\alpha_1 \quad A\alpha_2 \quad \beta_2$

✓ $B \rightarrow cA'bB' \mid dB'$

✓ $B' \rightarrow bB' \mid aA'bB' \mid \epsilon$

✓

$A \rightarrow BaA' \mid cA'$

$A' \rightarrow aA' \mid \epsilon$

$B \rightarrow cA'bB' \mid dB'$

$B' \rightarrow bB' \mid aA'bB' \mid \epsilon$

Gateway Classes

Compiler Design

Unit-1 : DPP-8

1. Check whether given grammar is ambiguous or not. If ambiguous then convert it into unambiguous grammar: E→E+E|E*E|id (AKTU 2023-24)
2. Construct the following grammar after removing left recursion: (AKTU 2023-24)

   S -> Aa | b | ε

   A -> Ac | SAd | ε
3. Define left recursion. Is the following grammar left recursive? (AKTU 2021-22)

   E → E+E | E*E| a| b
4. Eliminate left recursion from the following grammar (AKTU 2017-18)

   S→AB, A→BS | b, B→SA | a

Gateway Classes

AKTU

B. Tech Sem-VI

CS IT & CS Allied

GATEWAY CLASSES

Compiler Design

UNIT-1 Lecture-9

Gateway Classes



Todays Target

- Deterministic and Nondeterministic Grammar
- [redacted] Left Factoring
- BNF
- Capabilities of CFG
- Some small topics belonging to unit-1
- AKTU PYQs

By Dr. Nidhi Parashar Ma'am

- Ph.D (CSE)
- M.Tech Gold Medalist
- Net Qualified

Deterministic Grammar

- A grammar is called deterministic if, at any step of parsing, only one production rule can be chosen for a non-terminal and input symbol.
- In other words, there is no common prefix among the productions of the same LHS (non-terminal).
  At any step of parsing, the parser can choose the correct production without backtracking.

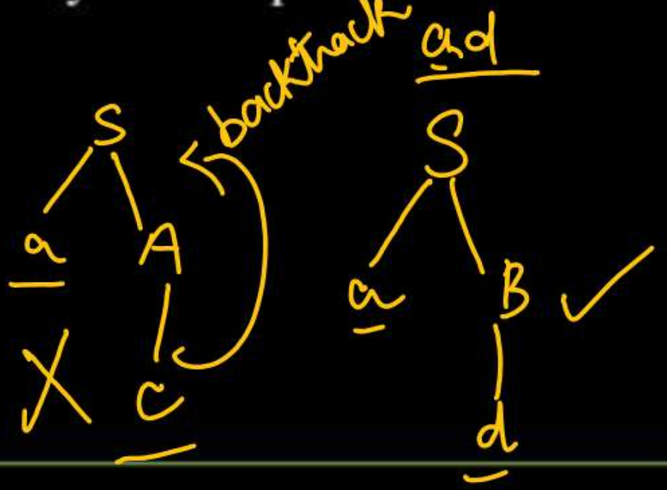
Example: $S \rightarrow aSb \mid bSa \mid \varepsilon$

Non-Deterministic Grammar

- A grammar is non-deterministic if more than one production rule can be applied for same non-terminal and input symbol.
- Two or more productions of the same LHS have a common prefix.
- Parser cannot decide immediately which production to apply.
- Backtracking is required.

Example:

$S \rightarrow aA \mid aB$
$A \rightarrow c$
$B \rightarrow d$



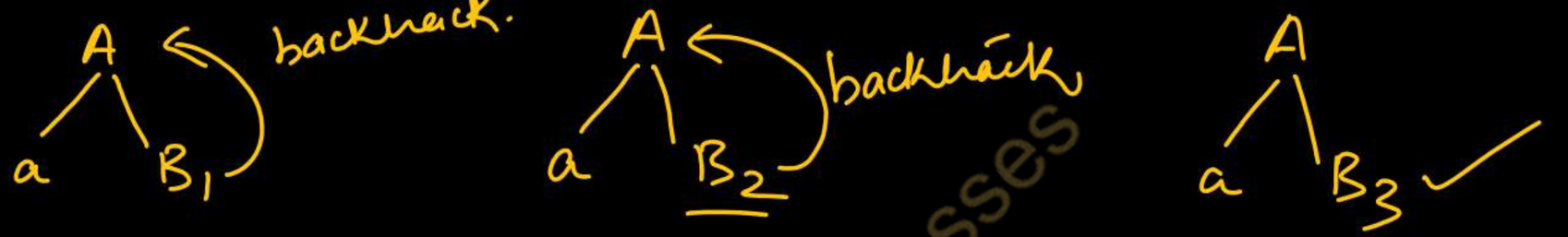
Gateway Classes

General Form of Non deterministic Grammar:

$A \rightarrow \underline{\alpha}\beta_1 \mid \underline{\alpha}\beta_2 \mid \underline{\alpha}\beta_3 \mid \ldots$ (Here α is the common prefix)

Example: $A \rightarrow \underline{a}B_1 \mid \underline{a}B_2 \mid \underline{a}B_3$

aB3 ✓



Gateway Classes

- A top-down parser cannot decide between productions if they have a common prefix.
- This causes non-determinism and requires backtracking, which is inefficient.
- Top-down parsers ~~like~~ do not allow backtracking. (LL(1) parser)
- **Left factoring** is used to remove non-determinism from a grammar by eliminating common prefixes in the productions of the same non-terminal.

Left factoring

- Left factoring is a process by which the grammar with common prefixes is transformed to make it useful for Top-down parsers.
- Suppose the grammar is in the form:

$$A \Rightarrow \alpha\beta1 \mid \alpha\beta2 \mid \alpha\beta3 \mid \ldots\ldots \mid \gamma_1 \mid \gamma_2 \mid \gamma_3 \ldots\ldots\ldots$$

- Here, A is a non-terminal and α is the common prefix.
- We will separate those productions with a common prefix and then add a new production rule in which the newly introduced non-terminal will derive those productions with a common prefix.

$$A \Rightarrow \alpha A' \mid \gamma_1 \mid \gamma_2 \mid \gamma_3 \ldots\ldots\ldots$$
$$A' \Rightarrow \beta1 \mid \beta2 \mid \beta3 \mid \ldots\ldots$$

Deterministic grammar.

Gateway Classes

Do left factoring in the following grammar-

$S \rightarrow iEtS / iEtSeS / a$.

$E \rightarrow b$

$S \rightarrow iEtSS' / a$

$S' \rightarrow \epsilon / eS$

$S \rightarrow bSSaaS / bSSaSb / bSb / a$

$S \rightarrow bSS' / a$ ✓

$S' \rightarrow SaaS / SaSb / b$ ✗

$S' \rightarrow SaA / b$ ✓

$A \rightarrow aS / Sb$ ✓

$S \rightarrow bSS' / a$

$S' \rightarrow SaA / b$

$A \rightarrow aS / Sb$

$A \rightarrow aAB / aBc / aAc$

$A \rightarrow aA'$ ✓

$A' \rightarrow AB / Bc / Ac$ ✗

$A' \rightarrow AX / Bc$ ✓

$X \rightarrow B / C$ ✓

$A \rightarrow aA'$

$A' \rightarrow Ax / Bc$

$X \rightarrow B / C$

Gateway Classes

Do left factoring in the following grammar-

$S \rightarrow aSSbS / aSaSb / abb / b$

$S \rightarrow aS' | b$ ✓

$S' \rightarrow SSbS | SaSb | bb$ ✗

$S' \rightarrow SA | bb$ ✓

$A \rightarrow SbS | aSb$ ✓

$S \rightarrow aS' | b$
$S' \rightarrow SA | bb$
$A \rightarrow SbS | aSb$

$S \rightarrow a / ab / abc / abcd$

$S \rightarrow aA$ ✓

$A \rightarrow \epsilon | b | bc | bcd$ ✗

$A \rightarrow bB | \epsilon$ ✓

$B \rightarrow \epsilon | c | cd$ ✗

$B \rightarrow cC | \epsilon$ ✓

$C \rightarrow \epsilon | d$ ✓

$S \rightarrow aA$
$A \rightarrow bB | \epsilon$
$B \rightarrow cC | \epsilon$
$C \rightarrow d | \epsilon$

Gateway Classes

Backus–Naur Form (BNF)

- BNF is a formal notation used to describe the syntax or grammar of a programming language so that compilers and interpreters can correctly analyze and process source code.
- In BNF, a language is described using terminal , nonterminal symbols and production rules.

Terminal symbols : keywords such as if while, operators like + and -, and punctuation symbols like ;.

Non-terminal symbols: represent expressions, statements, or loops usually written inside angle brackets,

Example: <expression> or <statement>.

A BNF production rule general form:

<non-terminal> ::= expansion

::= means "is defined as". The expansion on the right-hand side may contain terminals, non-terminals, or a combination of both. If there are multiple possible expansions, they are separated using the vertical bar symbol |.

Gateway Classes

- Example: <expression> ::= <number> "+" <number>

  <number> ::= <digit> | <digit> <number>

  <digit> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

This rule defines an expression as two numbers separated by a plus operator.

- Multiple productions with the same left-hand side can be written in a compact form using the | symbol.
- For example, the grammar

  $S \rightarrow aSa$

  $S \rightarrow bSb$

  $S \rightarrow c$

  can be written in BNF as: $S \rightarrow aSa \mid bSb \mid c$.

Gateway Classes

- BNF is important because it provides a clear way to define programming language syntax.
- It ensures consistency in language design, helps in building parsers and compilers

Capabilities of Context-Free Grammar (CFG)

1. Syntax Definition: CFGs are used to define the formal structure (syntax) of programming languages, making them suitable for describing hierarchical and nested constructs.

1. Automatic Parser Generation: A well-designed CFG enables tools like Yacc or Bison to automatically generate efficient top-down or bottom-up parsers.

2. Expression Handling: CFGs can effectively manage complex arithmetic and logical expressions by incorporating associativity and precedence rules.

3. Nested Structure Representation: CFGs are capable of describing nested language constructs, such as if-then-else statements, matching begin-end blocks, and balanced parentheses.

4. Syntax Tree Generation: They facilitate the construction of syntax trees (parse trees).

Gateway Classes

Q: Which Phase of Compiler Is Optional and Why? **(AKTU 2023-24)**

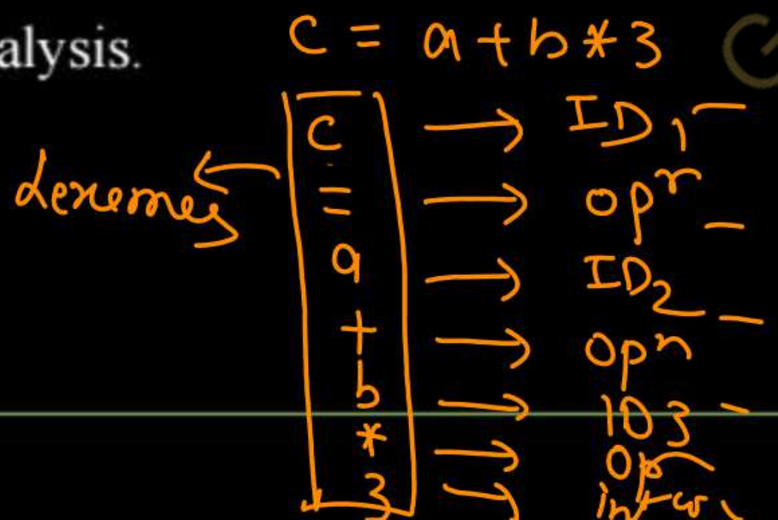
The Code Optimization phase of a compiler is considered optional.

Reason:

Code optimization improves the efficiency of the generated code in terms of execution time and memory usage, but it does not affect the correctness of the program. Even without optimization, the compiler can still generate correct target code.

Lexeme:

A lexeme is the actual character sequence in the source program that matches a token pattern during lexical analysis.



Gateway Classes

Compiler Construction Tools

These are software tools that help in the design and development of compilers by automating various phases of the compilation process.

1. The lexical analyzer generator is used to generate the lexical analyzer or scanner. It identifies tokens from the source program using regular expressions. Examples: Lex and Flex.
2. The parser generator is used to generate the syntax analyzer or parser. It checks whether the program follows the grammatical structure of the language. Examples: Yacc and Bison.
3. Syntax-directed translation tools are used during semantic analysis. They help in constructing syntax trees, performing type checking, and generating intermediate code by associating semantic actions with grammar rules.
4. Automatic code generators are used to convert intermediate code into target machine code.
5. Data-flow analysis tools are used mainly in code optimization. They analyze the flow of data values through the program to remove redundant code and improve performance.

Gateway Classes

Q: Write CFG to represent the language $L = \{a^{n+m}, b^{n} \mid m \geq 1\ n \geq 1\}$ **(AKTU 2020-21)**

That is, the number of **a's is more than the number of b's**, and there is **at least one extra a**.

How to do?

- Generate **equal numbers of a's and b's** (this corresponds to $a^n b^n$,$n \geq 1$)
- Then generate **one or more extra a's** in front (this corresponds to a^m ,$m \geq 1$)

$S \rightarrow aA$
$A \rightarrow aAb \mid ab$

$a \mid aAb$
$aaabb$

$S \rightarrow aS$
$\rightarrow aaS$
$\rightarrow aaaS$
$\rightarrow aaaaS$
$\rightarrow aaaaaA$
$aaaaaaAb$
$aaaaaaabb$

$S \rightarrow aS \mid aA$
$A \rightarrow aAb \mid ab$

$A \rightarrow aAb$
$aabb$

$A \rightarrow aAb$
$\rightarrow aaAbb$
$aaabbb$

- $A \rightarrow aAb \mid ab$ generates strings of the form $a^n b^n$,where $n \geq 1$ (equal no. of a's and b's)
- $S \rightarrow aS$ adds extra a's (for $m \geq 1$)
- $S \rightarrow aA$ ensures **at least one extra a** before switching to equal a and b generation

Gateway Classes

Compiler Design

Unit-1 : DPP-9

1. Discuss the capabilities of CFG. (AKTU 2022-23)
2. What is do you mean by left factoring the grammars? Explain. (AKTU 2021-22)

Gateway Classes

Gateway Classes

Empowering Learners, Transforming Futures

Thank You

Gateway Classes



Helpline No- 7819 0058 53 , 9259833200